

Shahid Hussain
Arif Ali Khan
Muhammad Abdul Basit Ur Rahim
Saif Ur Rehman Khan (Eds.)

Communications in Computer and Information Science 2725

SEET—Software Engineering for Emerging Technologies

First International Conference, SEET 2025
Long Beach, CA, USA, August 11–12, 2025
Proceedings

 Springer

Series Editors

Gang Li , *School of Information Technology, Deakin University, Burwood, VIC, Australia*

Joaquim Filipe , *Polytechnic Institute of Setúbal, Setúbal, Portugal*

Zhiwei Xu, *Chinese Academy of Sciences, Beijing, China*

Rationale

The CCIS series is devoted to the publication of proceedings of computer science conferences. Its aim is to efficiently disseminate original research results in informatics in printed and electronic form. While the focus is on publication of peer-reviewed full papers presenting mature work, inclusion of reviewed short papers reporting on work in progress is welcome, too. Besides globally relevant meetings with internationally representative program committees guaranteeing a strict peer-reviewing and paper selection process, conferences run by societies or of high regional or national relevance are also considered for publication.

Topics

The topical scope of CCIS spans the entire spectrum of informatics ranging from foundational topics in the theory of computing to information and communications science and technology and a broad variety of interdisciplinary application fields.

Information for Volume Editors and Authors

Publication in CCIS is free of charge. No royalties are paid, however, we offer registered conference participants temporary free access to the online version of the conference proceedings on SpringerLink (<http://link.springer.com>) by means of an http referrer from the conference website and/or a number of complimentary printed copies, as specified in the official acceptance email of the event.

CCIS proceedings can be published in time for distribution at conferences or as post-proceedings, and delivered in the form of printed books and/or electronically as USBs and/or e-content licenses for accessing proceedings at SpringerLink. Furthermore, CCIS proceedings are included in the CCIS electronic book series hosted in the SpringerLink digital library at <http://link.springer.com/bookseries/7899>. Conferences publishing in CCIS are allowed to use our online conference service (Meteor) for managing the whole proceedings lifecycle (from submission and reviewing to preparing for publication) free of charge.

Publication process

The language of publication is exclusively English. Authors publishing in CCIS have to sign the Springer CCIS copyright transfer form, however, they are free to use their material published in CCIS for substantially changed, more elaborate subsequent publications elsewhere. For the preparation of the camera-ready papers/files, authors have to strictly adhere to the Springer CCIS Authors' Instructions and are strongly encouraged to use the CCIS LaTeX style files or templates.

Abstracting/Indexing

CCIS is abstracted/indexed in DBLP, Google Scholar, EI-Compendex, Mathematical Reviews, SCImago, Scopus. CCIS volumes are also submitted for the inclusion in ISI Proceedings.

How to start

To start the evaluation of your proposal for inclusion in the CCIS series, please send an e-mail to ccis@springer.com

Shahid Hussain · Arif Ali Khan ·
Muhammad Abdul Basit Ur Rahim ·
Saif Ur Rehman Khan
Editors

SEET—Software Engineering for Emerging Technologies

First International Conference, SEET 2025
Long Beach, CA, USA, August 11–12, 2025
Proceedings

 Springer

Editors

Shahid Hussain
Penn State University
Behrend, PA, USA

Arif Ali Khan
University of Oulu
Oulu, Finland

Muhammad Abdul Basit Ur Rahim
College of Engineering
California State University, Long Beach
Long Beach, CA, USA

Saif Ur Rehman Khan
Shifa Tameer-e-Millat University
Islamabad, Pakistan

ISSN 1865-0929 ISSN 1865-0937 (electronic)
Communications in Computer and Information Science
ISBN 978-3-032-08976-2 ISBN 978-3-032-08977-9 (eBook)
<https://doi.org/10.1007/978-3-032-08977-9>

© The Editor(s) (if applicable) and The Author(s), under exclusive license
to Springer Nature Switzerland AG 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Preface

We are delighted to extend a warm welcome to the proceedings of the 1st International Conference on Software Engineering of Emerging Technologies (SEET 2025), hosted at California State University, Long Beach, CA, USA. A primary objective and characteristic of this initiative is to unite academic scientists, engineers, and industry researchers to share and exchange their experiences and research findings regarding various facets of science and social research. Additionally, it aims to facilitate discussions on the practical challenges faced and the solutions implemented. We hope you had a technically rewarding experience and used this opportunity to reconnect with old friends and make many new ones.

The SEET 2025 conference was an engaging and enlightening event, featuring an impressive lineup of keynote speakers. The program consisted of regular, short, idea and vision, and Student Research Competition (SRC) sessions and discussions with eminent speakers covering a wide range of topics related to the integration of emerging technologies and software engineering practices, and their implications. This comprehensive program offered all participants the chance to connect and engage with each other. We trust that your experience at SEET 2025 was both rewarding and enduring. With your assistance and involvement, the conference will sustain its success for an extended period.

The response to the call for papers was outstanding, originating from contributors in 21 different countries. Regrettably, many manuscripts from respected institutions could not be accepted due to the outcomes of the review process and our capacity constraints. We would like to express our heartfelt thanks and appreciation to all the reviewers who helped us maintain the high standard of the manuscripts included in these proceedings published by Springer.

We would like to thank the organization staff, the members of the program committees and reviewers. They diligently reviewed papers and provided valuable suggestions to the authors for enhancing their work. We would also like to extend our appreciation to the external reviewers for their additional assistance during the review process, as well as to the authors for their contributions of research findings to the conference. Special thanks go to Springer.

SEET 2025 aimed to create a platform for discussing the issues, challenges, opportunities, and findings associated with the integration of emerging technologies into the software development process. The constantly evolving nature and swift advancements in emerging technologies to improve software development processes give rise to new challenges and inquiries, leading to a genuine necessity for the exchange of innovative ideas and fostering a heightened awareness of this significant area of research.

We are now optimistic and full of hope about getting the proceedings of SEET 2025 covered by Springer and Scopus in due course. We acknowledge that the authors of SEET 2025 may seek to enhance the visibility of their papers. We will make every effort

to assist them in their pursuits. We extend our best wishes to all participants of SEET 2025 and hope you had a wonderful and productive experience at the conference.

August 2025

Shahid Hussain
Arif Ali Khan
Muhammad Abdul Basit Ur Rahim
Saif Ur Rehman Khan

Organization

General Chairs

Shahid Hussain	Penn State University, Behrend, USA
Arif Ali Khan	University of Oulu, Finland

Program Committee Chairs

Muhammad Abdul Basit Ur	Rahim California State University, Long Beach, USA
Saif Ur Rehman Khan	Shifa Tameer-e-Millat University, Pakistan

Track Chairs

Wen-li Wang	Penn State University, Behrend, USA
Mei-Huei Tang	Gannon University, USA
Muhammad Abid	Florida Polytechnic University, USA
Xin Qin	California State University Long Beach, USA
Tairan Liu	California State University Long Beach, USA
Muhammad Imran	Federation University, Australia
Azeem Akbar	Lappeenranta University of Technology, Finland
Souti Rini Chattopadhyay	University of Southern California, USA

Program Committee

Waqar Ali	Zurich University of Applied Sciences, Switzerland
Ghufran Ahmad	FAST-NUCES, Pakistan
Ghulam Mudassir	University of Buckingham, UK
Chong Chun Yong	Monash University Malaysia, Malaysia
Kifayat Ullah Khan	Birmingham City University, UK
Kwabena Ebo Bennin	Wageningen University & Research, Netherlands
Adnan Akhunzada	University of Doha for Science and Technology, Qatar
Pullin Agarwal	Penn State University, Behrend, USA

Mohammad Imtiaz Ullah	Ontario Tech University, Canada
Mohammad Junaid Khan	Western Washington University, USA
Mohammad Javid	Gomal University, Pakistan
Abdul Mateen	Federal Urdu University, Pakistan
Jie Zhao	Penn State University, Behrend, USA
Matteo Esposito	University of Rome “Tor Vergata”, Italy
Mahdi Fahmideh	University of Southern Queensland, Australia
Vita Santa Barletta	University of Bari, Italy
Shalli Rani	Chitkara University, India
Gastón Márquez	Universidad del Bío-Bío, Chile
Saima Rafi	Edinburgh Napier University, UK
Anibrata Pal	University of Bari, Italy
Javed Ai Khan	University of Hertfordshire, UK
Peng Liand	Wuhan University, China
Muhammad Imran	University of Florida, USA
Umer Javeed	Baymatob Pty Ltd., Australia
Khubaib Amjad Alam	Al Ain University, Dubai
Muhammad Imran	National University of Sciences and Technology, Pakistan
Mohsin Riaz	COMSATS University Islamabad, Pakistan
Javed Iqbal	COMSATS University Islamabad, Pakistan
Irum Inayat	FAST-NUCES, Pakistan
Raja Habib	Shifa Tameer-e-Millat University, Pakistan
Abdul Khaliq Shaikh	Sultan Qaboos University, Oman
Sajid Ibrahim Hashmi	University of Oulu, Finland
Md Rayhanur Rahman	University of Alabama, USA
Hassan Ali Khattak	Australian Catholic University, Australia
Abdul Wahid Khan	National University of Sciences and Technology, Pakistan
Kamran Latif	National Institute of Electronics, Pakistan

Additional Reviewers

Bashir Ahmad	Qurtuba University, Pakistan
Mohammad Imran Faisal	Shifa Tameer-e-Millat University, Pakistan
Mohammad Rafiq Mufti	COMSATS University, Pakistan
Humaira Afzal	Bahauddin Zakariya University, Pakistan
Ahmad Jan	Gomal University, Pakistan

Contents

Regular Research Paper

RF-MalDetect: Harnessing Random Forest for Malware Identification in PE Files	3
<i>Dhwanil Chauhan, Sachin Patel, Ankur Patel, Rushi Patel, Rishi Patel, Rooshikesh Bhatt, Margi Shah, and Manav Seth</i>	
Development of an Artificial Intelligence-Driven Live Patient Scheduling for Modern Healthcare	23
<i>Abdul Khalique Shaikh, Murtala Bello Abubakar, and Naresh Adhikari</i>	
Blockchain Technology in Logistics Business	31
<i>Nalyse Nakazato, Kashif Manzer, and Muhammad Abdul Basit Ur Rahim</i>	
Predicting User Affective States from Mobile Notification Interactions Using LLM-Based Machine Learning Models	45
<i>Muhammad Faizan Khan, Israr Ahmad, Muhammad Asif Khan, Ghulam Mudassir, and Gohar Hayat Khan</i>	
Responsible AI Transforms Insurance Claims via Prompt Engineering	61
<i>Shravya Kalva and Atif Farid Mohammad</i>	
Responsible AI Incorporation in Advanced Persistent Threat Landscape	68
<i>Sarthak Bhatt, Atif Farid Mohammad, and Urmita Banerjee</i>	
Enhancing Anomaly Detection in Software Logs with Bi-LSTM and Attention Mechanisms	78
<i>Zulfiqar Ali, Israr Ur Rehman, Muhammad Abdul Basit Ur Rahim, and Jonathan Witkowski</i>	
FIRMMOD: Generating API Taint Models for Firmware Analysis	90
<i>Ken Yihang Bai and Tuba Yavuz</i>	
Large Language Models Using Retrieval Augmented Generation and Prompt Engineering for AI-Driven Music Source Separation: A Literature Review	114
<i>Scott Josephson and Atif Farid Mohammad</i>	

Beyond Supervision: HyMoBY-Swin Hybrid Self-guided and Adaptive Learning Transformer for Multiclass Retinal Disease Diagnosis	125
<i>Muhammad Hammad Malik, Ghulam Mudassir, Yingying Ren, and Da-Wei Ding</i>	
STRIDE: Sports Tracking and Injury Detection Using Estimations	154
<i>Apoorva Rumale, Mahek Desai, and Marjan Asadinia</i>	
FAITH: Fault Anomaly Identification Using Machine Learning for Trusted Healthcare IoT	167
<i>Mahek Desai, Apoorva Rumale, and Marjan Asadinia</i>	
Synthetic Cognitive Augmentation Network	179
<i>Benjamin J. Kennedy, Atif Farid Mohammad, and Matthew Wyandt</i>	
Robust UAV Intrusion Detection via Federated Learning: A Comparison of NN and CNN-LSTM Models	189
<i>Sayed Muqayyad Hussain, Madiha Haider Syed, Adeel Anjum, Muhammad Javed, and Ankit Raj</i>	
Quantum Motion Sensing in an Electromagnetic Field: A Software Engineering Study of Four Techniques	213
<i>Hamed Nazari and Atif Farid Mohammad</i>	
Exploring the Configuration Space of BusyBox Vulnerabilities with CONFER	222
<i>Tuba Yavuz</i>	
Addressing Cultural Challenges During DevOps Adoption	246
<i>Javed Iqbal, Abdul Hadi Afghan, Muhammad Salih Tanveer, Muzaffar khan, Muhammad Javed, Shibi Rahul Senthil Kumar, and Mohammad Imran Faisal</i>	
Security Engineering Framework for Cyber-Physical System Product-Line	262
<i>Ademola Adejokun, Michael Siok, and LiGuo Huang</i>	
Effort Estimation in Agile Software Development Context: A Systematic Mapping Study	287
<i>Saif Ur Rehman Khan, Syed Abu Saeed, Habib Un Nisa, Muhammad Javed, and Kashif Manzer</i>	
Advancing Polycystic Ovarian Syndrome (PCOS) Detection Using Handheld Ultrasound Devices and Immunosensors: A Survey	302
<i>Kyla Harpe, Kelly Resetar, Hannah Vutuan, and Muhammad Abid</i>	

Multimodal LLM for Anomaly Detection	318
<i>Ayman Anba, Nathaniel Lethbridge, Preston Millhouse, and Muhammad Abid</i>	
A Comprehensive Survey of Computer Vision-Based Pose Estimation for Machine Learning and Deep Learning Approaches	347
<i>Mohamed Hadid and Muhammad Abid</i>	
Multi-output MLP Architecture for Machining Process Parameter Classification	371
<i>Dylan Fisher, Jonathan Liaw, and David Loker</i>	
User-Centered Software Engineering for VR Relaxation Applications	385
<i>Franceli L. Cibrian, David Zhang, Katie Ho, Tyler Kay, Hector Camarillo-Abad, and Trudi Di Qi</i>	
Can Large Language Models be Used as an Alternative for Human Annotation: A Case Study of Emotion Classification	400
<i>Nek Dil Khan, Maram Fahaad Almufareh, Javed Ali Khan, Jianqiang Li, Arif Ali Khan, and Mamoonah Humayun</i>	
A Quantum Ising Model for Solving Sudoku Puzzles	417
<i>Wen-Li Wang, Mei-Huei Tang, Shahid Hussain, and Kevin Wang</i>	
Women’s Role in Software Engineering - An Empirical Study	430
<i>Mahima Sachan, Mayank Maurya, Akash Kumar Singh, Mohammed Faizaan Lnu, Syed Mohammed Sami Abedi, and Muhammad Abdul Basit Ur Rahim</i>	
Mammo-Find: An LLM-Based Multi-channel Tool for Recommending Public Mammogram Datasets	446
<i>Raiyan Jahangir and Vladimir Filkov</i>	
Exploring Manifold-Based Clustering Techniques for Enhanced Inductive Thematic Analysis	464
<i>Jesus A. Beltran, Hanna Mofid, Harita Parikh, Jaydeep Gondaliya, Diego Guzman, Jenil Shah, Lizbeth Escobedo, and Franceli Cibrian</i>	
Detailed Cryptanalysis of “Privacy-Preserving Quantum Federated Learning via Gradient Hiding”	481
<i>Zafar Iqbal, Syed Zohaib Hassan, Jie Zhao, and Shafiya Mubeen Umme</i>	
A Systematic Mapping Study on Toolchain Support for Quantum Computing as a Service	500
<i>Maryam Tavassoli Sabzevari</i>	

A Scalable Software-Hardware Co-design FPGA Platform for Floating-Point Operations	517
<i>Cameron D. DiSomma, Dania Susanne Mosuli, Hailu Xu, and Xiaokun Yang</i>	

AI-Powered Engine Component Identifier for Vehicle Maintenance	529
<i>Aron Cruz, Yalong Wu, and Xiaokun Yang</i>	

Short Research Paper (SRP)

Lightweight and Generalizable Glioma Grading Using Hyperdimensional Computing	541
<i>Mehjabeen Tasnim, Justin Morris, and Sreedevi Gutta</i>	

muRelBench: MicroBenchmarking for Zonotope Domains	551
<i>Kenny Ballou and Elena Sherman</i>	

A Lightweight Machine Learning Pipeline for Crypto Forecasting: A Capstone Case Study in Software Engineering Education	560
<i>Lucas Norpchen, Omar Garcia, Koby Winkler, Jose Temblador, and Benyamin Ahmadnia</i>	

Solving Prime Factorization Using Quantum Ising Model	568
<i>Wen-Li Wang, Mei-Huei Tang, Shahid Hussain, and Kevin Wang</i>	

Comparative Fine-Tuning of GPT-2 on Question Answering and Dialogue Datasets for Medical Text Generation	575
<i>Caleb Nhkum, Mohammad Masudur Rahman, Tanvir Ahmed, and Md. Faisal Kabir</i>	

Idea and Vision

A Comprehensive Framework for Optimizing API Calls, CI/CD Pipelines, and Energy Testing	593
<i>Muhammad Asif Khan, Shola Oyediji, and Jari Porras</i>	

Quantum Computing: Grover's Algorithm for String Search and Its Practical Limits	606
<i>Nimit Dagli, Jose Salgado, Yusuf Usman, and Taskin Kocak</i>	

Open-Source LLMs for Technical Q&A: Lessons from StackExchange	615
<i>Zeerak Babar, Nafiz Imtiaz Khan, Muhammad Hassnain, and Vladimir Filkov</i>	

Student Research Competition (SRC)

Analyzing and Visualizing Software Quality of Code in GitHub Repositories Using AST-Based Metrics	629
<i>Dipen Rathod</i>	
ASSIST: AI Soccer Statistics and Information Systems Technology: A Computer Vision Approach to Player Tracking and Game Analysis	649
<i>Ian Weiss and Oscar Morales-Ponce</i>	
Holistic Risk Management for Next-Gen Technologies Using AI and Governance Standards	670
<i>Samuel Oakes</i>	
Timed I/O Automata for Searching in Dynamic Environments with Humanoid Robots	680
<i>Fozhan Babaeiyan Ghamsari, Stephen Martinez, Soroush Mirzaee, and Oscar Morales Ponce</i>	
An Analysis of Worldwide Language Networks and Cultural Clustering	697
<i>Fozhan Babaeiyan Ghamsari, Anh Le, Claudia Rawson, Lesly Castellanos, Salvador Sandoval, Richie Prak, and Oscar Morales Ponce</i>	
Artificial Intelligence Applications in Software Development: A Bibliometric Approach	716
<i>Syn Nguyen and Manohar Valabi</i>	
Fairness Testing: Review of the Relevance and Methodologies of Ensuring Software Fairness	733
<i>Ilmaan Zia, Ankit Raj, and Jonathan Dabu</i>	
Author Index	761

Regular Research Paper



RF-MalDetect: Harnessing Random Forest for Malware Identification in PE Files

Dhwanil Chauhan¹ , Sachin Patel² , Ankur Patel² , Rushi Patel³ ,
Rishi Patel⁴ , Rooshikesh Bhatt⁵ , Margi Shah⁶ , and Manav Seth⁷ 

¹ Purdue University, Hammond, IN 46323, USA

Dhwanil832@gmail.com

² Charotar University of Science and Technology (CHARUSAT), Anand, India
{Sachinpatel.dit, Ankurpatel.ee}@charusat.ac.in

³ Arizona State University, Tempe, USA

⁴ Indiana University Bloomington, Bloomington, USA

⁵ Conestoga College, Kitchener, Canada

⁶ University of Ottawa, Ottawa, Canada

⁷ University of Alabama Birmingham, Birmingham, USA

Abstract. In this work, an efficient machine learning-based approach is taken up: Random Forest, which identifies the efficiency of this very approach in distinguishing malware from original applications based on the content of the PE file. It deals with broad characteristics that have been taken from the PE files for training and tests by using a dataset with labeled examples of both kinds. Such may include the accuracy, precision, and recall among other metrics that shall be used through this experimentation phase, testing the efficacy of the Random Forest classifier in distinguishing the two categories. It also shows how to use the model in the Graphical User Interface to make predictions with regard to the PE files in an easily understandable way by users. These findings convey how machine learning approaches, especially Random Forest, potentially enhance cybersecurity methods through efficient detection of malware.

Keywords: Cyber Security · Malware Detection · Random Forest

1 Introduction

Malware proliferation is one of those consistent threats which cybersecurity has to put up with in the modern context. Detection systems should be firm yet flexible since the cyber threats get fancier and more complex. In this domain, machine learning has promised a great deal in proactive detection and evasion of noxious software. This paper describes the appropriate methodology to segregate malicious from actual software through a flexible ensemble learning method, Random Forest, which enjoys a high reputation among the classification methods. This paper intends to apply Random Forest to the analysis of Portable Executable files. The research work, therefore, investigates the feasibility and effectiveness of using Random Forest in identifying micro patterns in the Portable Executable file that actually distinguish between benign software from dangerous threats toward developing cybersecurity standards.

Their frequency shows that the detection methods of malware attacks have to provide timely identification of the progress of an attack and eliminate emerging new threats. The continuous evolution of the malware landscape keeps on making it hard for the classical signature-based methods to keep pace with it. There is a strong need for innovative techniques concerning automatic detection and adaptation to novel attack vectors. A promising paradigm change that machine learning approaches make use of includes native algorithms that can evaluate enormous volumes of data to find complicated patterns. In this regard, considering the adaptability, robustness, and high-dimensionality data handling capability, the Random Forest method will keep being in strong competition. Therefore, the subsequent study shall, by the power of Random Forest, enhance the ongoing debate on the effective ways of malware detection. In particular, PE files, among the most common executable file formats occurring under Windows systems, will be reviewed in some detail to give a close look at the characteristics important for their precise categorization.

2 Literature Survey

2.1 ML Integrated with Cybersecurity

It is fighting the ever-changing panorama of threats by including Machine Learning approaches. By automatically finding patterns and behaviors from gigantic datasets with algorithms, ML provides one with proactively enabling ways whereby one can identify and mitigate any potential dangers in real time. Machine learning works in cybersecurity because it identifies complex and dynamic patterns in data; hence, it is easy to find anomalies, hostile activities, and breaches missed by traditional rule-based systems. It is due to the extensive flexibility of machine learning techniques in responding to changes in threats with great capacity for handling a wide variety of different data types of high dimensionality.

Machine learning introduces a barrier that is kinetic-the introduction of machine learning into cybersecurity to advanced cyber threats. Advanced malware is really agile, keeps changing for evasion, and has conventionally been very hard for cybersecurity mechanisms depending on signature-based detection or rule-based systems. Machine learning models, however, transcend such limitations by finding much more subtle patterns and behaviors common to malicious activities. This has occasioned an growing number of research studies that spur on many machine learning algorithms and techniques suitable for particular cybersecurity needs. This literature review, therefore, sets out to undertake a critical review of the literature previously conducted on the use of machine learning techniques, with particular emphasis on their role in enhancing malware detection and strengthening cyber defense tactics. The use of ML in cybersecurity creates a dynamic barrier against sophisticated cyber threats. In nature, modern malware is extremely agile; it changes every single day to avoid detection. The classic cybersecurity mechanisms are based on either signature-based detection or rule-based systems, unable to cope with such frenetic rhythms. Opposite to this, the model of machine learning comes out unscathed by tracing minute patterns and behaviors present in malicious activities. This evolution has accordingly called for an increased number of research

studies focusing on different machine learning algorithms and methods relevant to specific cybersecurity-related needs. The purpose of this literature review, therefore, is to put in perspective the key findings of previous research works that explore the contribution made by machine learning techniques in providing elements with respect to enhancing malware detection and improving cyber-defense strategies.

2.2 Decision Trees in Machine Learning

Because decision trees provide an intuitive way to depict decision-making processes, they are a fundamental method in machine learning. The most important feature is represented by the root node of this tree-like structure created by these models, decisions based on features are represented by the branches, and the final classification or prediction result is shown by the leaf nodes. It does so by iteratively partitioning the feature space to attain maximum information gain or minimum entropy, hence segmenting data into subsets of similar characteristics. Besides this, the main reasons why Decision Trees gained fame within many industries are due to their interpretability and versatility in data exploration, decision insight, Regression Analysis, and categorization (Fig. 1).

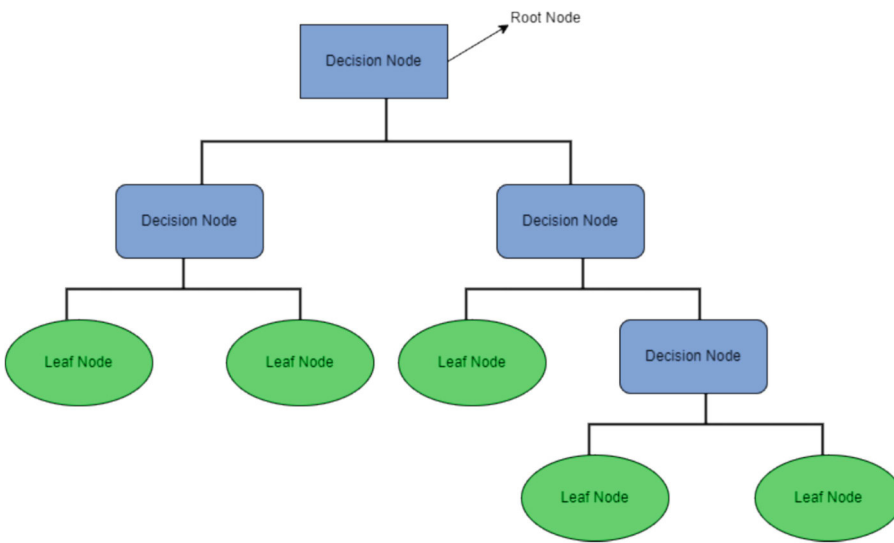


Fig. 1. Working of decision tree

ID3 Algorithm for Decision Trees

Among machine learning techniques, decision trees are one of the fundamental ways due to the naturalness of representation when modeling decision-making processes. In a tree-like structure developed by these models, the most significant characteristic forms the root node, the branches denote decisions or choices based on features, and the final results of classification or prediction appear as leaf nodes. The feature

space is segmented recursively, aiming directly for maximum information gain or reducing entropy, the segmentation of data is also done into subgroups with similar characteristics, and it shows a tree-like structure. They are highly interpretable, flexible models, hence finding wide applications in many industries. They can be used for the exploration of data, gaining insight into decision-making, and performing both regression analysis and classification.

$$H(S) = - \sum_{i=1}^n p_i \log_2(p_i) \quad (1)$$

$$\text{Information Gain (IG)} = \text{Entropy}(S) - [(\text{Weighted average}) * \text{Entropy}(\text{each feature})] \quad (2)$$

CART Method for Decision Trees

The CART method overcame several limitations of ID3: its ability to be used for both regression and classification problems. CART constructs binary decision trees by splitting the data into subsets using attribute thresholds and seeking the optimal threshold that maximizes the purity of each subset. For classification, CART uses the Gini impurity criterion for the goodness of a split. It uses MSE in regression problems as a method for finding the best splits for continuous outcomes. The range was massively large by capability: to take both numerical and categorical features, to deal well with missing values, and to build binary trees. It is such flexibility that allowed CART to find applications in a great many machine learning and data analysis tasks.

$$\text{Gini Index} = 1 - \sum_j p_j^2 \quad (3)$$

2.3 Random Forest

ID3, concerning some of the deficiencies overcome by the technique of CART, the most important of them relate to its adaptability and suitability whether for classification or regression problems. In CART, binary trees are created by thresholding on attributes for data division into subgroups and impurity optimization from each. Amongst other measures, Gini impurity used in this algorithm assists in measuring the goodness of splits with regards to classification with the aim of misclassification reduction. In the case of continuous outcomes, the best splits for it are obtained by the use of MSE in regression situations. Handling of missing data by forming binary trees is a strong ability of CART and it accepts both numerical and category features substantially. It thus came to be an indispensable tool in several data analysis and machine learning domains.

It is important to notice that the Random Forests overcome several drawbacks of solitary decision trees. The methodology decreases the risk of over-fitting of the training data for the individual trees by including many trees into one; hence, much stronger generalization to unknown data is allowed. The deliberate introduction of randomness

during feature and sample selection maintains a heterogeneous ensemble with low inter-tree correlation, hence helping the model to capture complicated patterns in data. Apart from that, Random Forest is also resistant to noise within the dataset and outliers. They provide regular and dependable results on quite diverse sets of areas. The said technique has wide applications within the highly critical sectors of bioinformatics, finance, and healthcare. Therefore, it also presents the importance of producing reliable, accurate, and flexible predictive models from a wide range of real-world applications.

2.4 Isolation Forest

Isolation Forest has been one of the innovative, effective anomaly detection techniques with the aim of finding anomalies or outliers in datasets. Unlike traditional techniques that rely on the normalcy assumption of data, the approach of Isolation Forest is to isolate anomalies, rather than building a profile of normal cases. The general idea is constructing random decision trees where pathways leading to anomalies are shallower since anomalies are unique and can be easily isolated. This differs from normal data points, which tend to expose anomalies sooner since they require extra splits in order to make them different from others.

The underlying philosophy of Isolation Forest is to build multiple random trees to achieve the necessary anomaly isolation. Indeed, it very fast identifies anomalies in the sense of their features being different from all others by a further partition of data with randomness in features and splitting points. This reveals stunning performance and scalability on quite large datasets or high-dimensional data conditions. Isolation Forest has been especially helpful in recognizing outliers from otherwise varied datasets across domains, from cybersecurity to fraud detection, by identifying abnormalities without large preprocessing or normality assumptions.

2.5 Mondrian Forests

The changed ensemble learning technique of Mondrian Forests is quite scalable and flexible, having a combination of decision trees and random forests. Since it is a sort of input space division in discontinuous rectangles through random projections, every rectangle is representing a decision tree leaf. Though classical decision trees depend on feature space splits based on impurity or information gain metrics, Mondrian Forests depend on grid-based splitting with random projections to have a scalable and efficient learning process.

A key intuition behind the Mondrian Forests is that the methodology is able to generate an astronomically large number of Mondrians or decision trees composing an ensemble of predictions. Each of those Mondrians concentrates on another region of the input space, informed by the signal in that particular partition. The Mondrian Forests also adaptively change their structure during learning, with the capability to be updated and improved continuously given new data. With this flexibility and scalability, Mondrian Forests are especially well-placed for high-dimensional data or streaming data problems, where traditional approaches become computationally infeasible. The domains that show huge interest in Mondrian Forests include online advertising, recommendation systems, and time-series forecasting, where this power of efficiency and

agility in handling enormous datasets with responses against shifting data distributions is experienced.

2.6 XGBoost Algorithm in Machine Learning

XGBoost stands for eXtreme Gradient Boosting, another efficient and popular ensemble learning technique. It's highly praised due to its performance in accuracy and speed for classification, regression, and ranking problems. The algorithm creates a sequential ensemble of weak learners, typically decision trees, but in many ways, it most closely resembles Random Forests since it's also an ensemble-based algorithm. But, unlike Random Forests, XGBoost is a boosting technique where the mistakes from the previous trees get corrected by the next tree. The model keeps on improving its prediction skills by increasing the weights for cases that are misidentified during training.

The key features that distinguish XGBoost include the new gradient boosting framework with regularization approaches to prevent overfitting, and the customized goal function, which can be used with a variety of problem types in optimizing model performance. The explanation goes in more detail with missing values in a collection, parallel processing supported, and scaling across huge datasets for maximum efficiency. Furthermore, by supplying feature relevance ratings, XGBoost allows for the identification of important variables, enabling one to glean insight into the patterns that underlie the data.

With its widespread use in industry segments like online advertising, healthcare, and finance, XGBoost is of great importance and applicability in creating reliable prediction models. It has become the go-to algorithm for practitioners seeking high predicted accuracy, efficiency, and interpretability in their models due to its outstanding results in many data contests and practical applications. Although XGBoost is not implemented in this study, the mention of it brings up the different ways to approach similar tasks using modern machine learning—thanks to its adaptability and diversity.

2.7 Gradient Boosting Machines (GBM)

Gradient Boosting Machines (GBM) are powerful ensemble learning techniques and have recently gained popularity due to great predictive performance in many fields. Unlike Random Forest, which builds trees on their own, GBM builds trees in a sequential way where each tree tries to correct the errors made by its preceding trees. In principle, GBM iteratively minimizes a pre-defined loss function by adding decision trees. This is an iterative approach that focuses successive trees on those areas where the current model is poor. It improves the accuracy over time, as it gradually refines its predictions. GBM reiterates past errors in order to improve its predictive power over time by pinpointing complex patterns in the data.

GBM works by creating shallow decision trees, or “weak learners,” which are then added together to create a robust model by boosting. That is how, through an iterative process, total prediction performance can be continually improved with adjustments derived from residual errors of previous models. It also has regularization techniques to prevent overfitting, control tree complexity, or add penalties for model complexity in order to make sure of correct predictions on new data. Coupled with its exceptional

accuracy, GBM is very flexible in handling different kinds of data sources and problem domains, which makes it among the top choices for any regression or classification tasks—be it in the field of natural language processing, finance, or healthcare.

2.8 AdaBoost (Adaptive Boosting)

AdaBoost, Adaptive Boosting, is an ensemble learning approach designed for constructing a strong and accurate model by combining weak learners in order to improve their predictive power. In contrast to the standard approach in ensemble methods, AdaBoost assigns different weights to individual learners, therefore providing more concentration on misclassified cases in subsequent rounds. These weak learners are trained iteratively: each following learner is trained more on examples that were wrongly classified in previous rounds. The flexibility in AdaBoost helps it to continuously improve its predictions by combining several weak learners into one strong model.

Essentially, AdaBoost adapts its strategy based on the complexity in data. If examples are misclassified, it increases their weights so that the next learners in the sequence are urged to pay more attention to correctly classifying those. Combining all these classifiers, AdaBoost creates a strong model for complex datasets—capable of picking up subtle trends that may be missed by individual classifiers. With its ability to adapt to different base classifiers and deal with high-dimensional data, it fits perfectly for classification tasks in those fields where accurate predictions are a must: image recognition, text analysis, and bioinformatics. However, AdaBoost can be sensitive to noisy data or outliers; still, its ability to boost weak learners toward a strong ensemble model classifies it among the important machine learning tools for the solution of different classification problems.

2.9 Gradient Tree Boosting

Gradient Tree Boosting, also known as Gradient Boosted Decision Trees (GBDT) or Gradient Boosting Machines (GBM), is a more advanced ensemble learning technique that combines the efficacy of boosting algorithms with that of decision trees. It works by adding trees to an ensemble one after another, where each tree tries to correct errors made by its predecessor. Gradient tree boosting builds decision trees one by one, learning from the mistakes of the previous trees, constantly improving the prediction, whereas it is not an independent mechanism unlike the regular decision trees. The basic idea is to add one tree at a time, trying to minimize the pre-set loss function by concentrating on the residuals or errors of the previous ones.

Gradient tree boosting can be seen as an iterative process for constructing a strong predictive model out of weak learners, usually shallow decision trees. That is, it works by fitting a number of trees, which each correct errors produced by the group of trees that came before it. It is then made to learn and improve by way of using the gradients of the loss function or residuals, in an iterative process. Gradient tree boosting is a very effective way to create a strong model—one that can capture intricate interactions within the data by slowly lowering these errors. Gradient tree boosting is one of the options that really stand out in machine learning applications, based on their possibility of being applied to any type of data and dealing with both regression and classification tasks. This

is especially true for the fields of natural language processing, finance, and healthcare, in which accuracy and interpretability are critical.

2.10 Joblib Library

Joblib is mainly a lightweight Python pipelining library. It provides a simple and efficient way of doing parallel computing, particularly for CPU-bound tasks such as training models. Most important, though, it provides an efficient way to serialize Python objects, including complex data structures, to disk. This becomes very handy when dealing with large datasets or machine learning models. This library provides a set of tools to cache function results by reusing previously computed results when the inputs are unchanged, hence reducing the computation cost.

Joblib makes parallel computing in Python very accessible, with simple functions and classes that enable a user to parallelize a task over several cores or processors without much hassle. This library is especially useful in the domain of machine learning, where model training and testing could get quite tedious. Using Joblib, tasks can be divided and processed simultaneously to yield large speedups in computing. This comes in handy for quicker computations down the line: due to its serialization, there is no need to recalculate intermediate results or trained models. To save memory, Joblib's cache caches the output of functions for any given input parameters by avoiding redundant calculations.

Joblib has certainly helped a lot in making the training and assessment stages of your models easier; that is, parallel data handling would likely have sped up feature selection, model training, and cross-validation, generally improving the efficacy of your research pipeline. This serialization can also be helpful in saving and loading intermediate results or trained models during the research process to fasten experimentation and iterations.

2.11 Numpy

The ndarray is the multi-dimensional array class for changing data processing. NumPy is a fundamental Python module for numerical computing. Scientific computing is constructed on top of this basic framework, elaborating an array-oriented computing technique to accelerate data manipulation, statistic analysis, and mathematical operations. This versatility is realized from the fact that the ndarray efficiently handles homogeneous datasets with multiple dimensions, allowing operations like slicing, reshaping, and broadcasting. The usefulness of NumPy is increased in scientific fields due to its wide range of mathematical functions, which include statistical operations, random number generation, and linear algebra. NumPy is the basic tool in data manipulation for research and development of applications because it interfaces well with C/C++ and Fortran code and further enhances the performance.

NumPy is an essential tool in the manipulation and preparation of data for malware detection using the Random Forest technique. Most likely, the library made it easier since it converted the properties of the datasets into arrays that can be used in the training model by utilizing the operations on arrays and mathematical capabilities in NumPy. The reliable array-based operations probably made feature engineering jobs easier, hence allowing the users to manage the numerical data required for model validation effectively. The ability of NumPy in handling big datasets and to perform mathematical operations

fast is what probably has made the dataset preparation and model implementation stages run smoother and fast.

2.12 PEfile Library

PEfile is a Python module, and without it, analysis and understanding of the Port-able Executable (PE) files—the default executable format in Windows systems—would be impossible. Using the impressive list of features that PEfile offers, one can dive deep into the inner workings of PE files and decipher minute facts hidden in headers, sections, and resources. This library exposes important metadata, such as header details, section properties, import and export tables, and resource information. The user is able to extract much data from PEfile, which is very essential for deep analysis and understanding of the PE file structure. It is a very important area in the field of Windows executable files.

PEfile was an integral part in breaking down PE files and extracting key metadata to further the goal of classifying malware using the Random Forest technique. Its features probably ease the extraction of complex information, including machine types, section sizes, entrance points, and file properties that were needed for strong feature sets or engineering features fit for training of a machine learning model. Furthermore, by using PEfile, one can automate the extraction of crucial resource data, such as manifest details, version information, or embedded resources, to simplify the overview of characteristics and actions of executable files. Your research may have developed a deeper understanding of PE file structures by using the capabilities of PEfile. This expertise could then be convincingly used to propose the detection and analysis of malicious behavior in Windows executables.

2.13 Scikit-Learn

Scikit-Learn is a crucial Python package for the machine learning environment. For scholars and practitioners in the subject, it acts as an extensive set of tools and methods, providing a flexible toolset. A variety of machine learning methods, including Random Forests, Decision Trees, and Support Vector Machines, are easily integrated with this toolkit due to its friendly user interface. In our study on malware detection using Random Forest, Scikit-Learn was a necessary package that helped in efficient model deployment and model assessment. From feature scaling to handling missing values in preprocessing, training, evaluation, and visualization of the models, it had it all.

The flexibility and usefulness of Scikit-Learn go beyond simply providing an implementation for a variety of algorithms but cover the whole machine learning process. Scikit-Learn did ease our tasks of preparing and formatting datasets for our research, ensuring they are ready to be trained by our Random Forest classifier. Its collection of visualization and assessment tools made it easier to assess how well our model performed in terms of identifying malicious software. The contribution of Scikit-Learn went beyond mere implementation; it went to the building, assessment, and deployment of models. Its broad functionality and ease of use made it possible to streamline the usage of machine learning techniques, which greatly contributed to creating and assessing our malware detection model.

2.14 Tkinter

Tkinter is a simple Python GUI (Graphical User Interface) toolkit that offers a simple yet powerful set of tools for building graphical desktop applications. It acts as a wrapper to Tcl/Tk, allowing Python programmers to build attractive and user-friendly interactive graphical user interfaces. Tkinter makes it easier to create a window, a button, a menu, and so on with the extensive collection of widgets and tools it has. In the arena of Python, user-friendly programs are most preferably produced by Tkinter due to its versatility and simplicity of usage on multiple platforms.

Owing to its simplicity and neat integration with Python, Tkinter is one of the best options for desktop application development. With its great set of toolkits and widgets, developers can create graphical user interfaces that are not only aesthetic but also interactive. The modularities and flexibility of Tkinter enable developers to create their own unique UI elements and handle events from users. As Tkinter has cross-platform features, the applications developed with it run seamlessly on many operating systems without additional modification.

Tkinter was important in our research project for developing an intuitive user interface to deal with the malware detection system. We developed an easy-to-use interface using the features of Tkinter that would enable users to navigate, enter file locations, and receive immediate feedback about the validity of executable files. We were able to develop an interactive application interface quickly and create an interactive application that provided users interacting with the malware detection system with a seamless experience, thanks to Tkinter's flexibility and ease of use. The development of an accessible interface by Tkinter encouraged a more user-centered strategy for using machine learning in cybersecurity applications.

3 Related Work

(See Table 1).

Table 1. Related works

Sr. No.	Title	Abstract	Conclusion	Result/Key Findings
1	Android Mobile Malware Detection Using Machine Learning: A Systematic Review [1]	Proposes a deep learning-based malware detection system using CNNs.	Highlights the effectiveness of CNNs in achieving high detection rates.	Achieved an accuracy of 98.9% and outperformed traditional methods.
2	Deep Neural Network Based Malware Detection Using Two-Dimensional Binary Program Features [2]	Utilizes machine learning for dynamic malware analysis, focusing on API call sequences	Emphasizes the effectiveness of machine learning in dynamic analysis	Demonstrates accurate detection and classification of malware based on API sequences

(continued)

Table 1. (continued)

Sr. No.	Title	Abstract	Conclusion	Result/Key Findings
3	A Novel Intrusion Detection System based on Machine Learning for Internet of Things (IoT) Devices [3]	Presents a malware classification approach using machine learning with a focus on static features	Suggests that machine learning on static features provides accurate malware classification	Achieved high accuracy in classifying malware based on static features
4	Intelligent Vision-Based Malware Detection and Classification Using Deep Random Forest Paradigm [4]	Provides a survey of various machine learning techniques employed in malware analysis	Summarizes the strengths and weaknesses of different machine learning approaches in malware analysis	Offers insights into the diverse applications of machine learning in malware analysis
5	Intelligent Malware Detection using Oblique Random Forest Paradigm [5]	Proposes an oblique random forest ensemble learning technique for malware detection	Concludes that the proposed model outperforms existing decision tree models	Achieved high accuracy and low false positive rates in malware classification
6	Machine Learning Approach for Malware Detection Using Random Forest Classifier on Process List Data Structure [6]	Develops a machine learning approach for malware detection using a random forest classifier	Reports a prediction accuracy of 90.9% using process list data from a Linux environment	Achieved 90.9% accuracy in predicting malware using process list data
7	Malware classification for the cloud via semi-supervised transfer learning [7]	Proposes a semi-supervised transfer learning model for malware detection in cloud computing	Highlights improved detection accuracy using semi-supervised transfer learning	Improved detection accuracy from 94.72% to 96.9% with the proposed model
8	Malware Detection using Machine Learning and Deep Learning [8]	Investigates malware detection using machine learning and deep learning, comparing random forest with deep neural networks	Concludes that random forest outperforms deep neural networks for opcode frequency-based malware detection	Random forest achieved higher accuracy (99.7%) compared to deep neural networks

(continued)

Table 1. (continued)

Sr. No.	Title	Abstract	Conclusion	Result/Key Findings
9	Malware Detection Using Machine Learning [9]	Proposes a versatile framework for malware detection using different machine learning algorithms	Acknowledges the need for deterministic exception mechanisms to achieve a zero false positive rate	Achieved a significant increase in total detection rate, especially with cascade one-sided perceptrons
10	Opcode-sequence-based Semi-supervised Unknown Malware Detection [10]	Our main target was to come up with a machine learning framework that generically detects as much malware samples as it can, with the tough constraint of having a zero false positive rate	Empirical validation with varying percentages of labeled instances	Results include True Positive Ratio, False Positive Ratio, Accuracy, and AUC
11	Robust Malware Detection Models: Learning from Adversarial Attacks and Defenses [11]	Investigates robust malware detection models against adversarial attacks	Highlights vulnerability of ML models to attacks, proposes defense strategies	Achieved high fooling rates against detection models, proposed defense strategies
12	AIMED-RL: Exploring Adversarial Malware Examples with Reinforcement Learning [12]	Focuses on generating functional adversarial examples in malware domain	Introduces AIMED-RL, a method for generating adversarial examples in malware classification	Achieved evasion rates using RL-based agent, emphasized importance of penalty technique, shorter sequences of transformations for effective evasion
13	Two Semi-supervised Approaches to Malware Detection with Neural Networks [13]	Compares semi-supervised algorithms for malware detection with neural networks	Application of semi-supervised learning to improve neural network performance in malware detection	Improved accuracy with low ratios of labeled data using semi-supervised methods

(continued)

Table 1. (continued)

Sr. No.	Title	Abstract	Conclusion	Result/Key Findings
14	Ransomware Detection using RandomForest Technique [14]	Proposes static analysis-based ransomware detection from raw bytes, achieving 97.74% accuracy	Highlights optimal detection with 100 trees and seed 1 in 1.37 s. - Compares classifiers' performance	Emphasizes RandomForest's efficiency in byte-level analysis. - Outperforms opcode-based methods
15	Integrated static and dynamic analysis for malware detection [15]	Proposes an integrated static and dynamic analysis method for malware detection	Combined method enhances detection accuracy over standalone methods	Improved accuracy: Static - 95.8%, Dynamic - 97.1%, Integrated - 98.7%

4 Proposed System

4.1 Dataset Description

This data contains a wide range of attributes that have been extracted from PE files. Portable Executables are the building blocks of dynamic link libraries and executable programs that are mainly used with operating systems of the Windows family. This dataset offers a wide range of characteristics, and it explores internal details of those PE files. Some of the important header data that reflect the structural composition of such files are machine type, size of optional header, characteristics, and linker versions.

In addition, the dataset contains essential size metrics that give an idea of the physical arrangement and resource allocation of the files, which include size of code, size of initialized and uninitialized data, image base, details of alignment, and check-sum. It also includes vital version information such as operating system and image versions, details of subsystems, and header sizes-a pretty exhaustive rundown of the technical and evolutionary features of the PE files.

This dataset represents a comprehensive and diverse collection of PE files, which include executable programs and dynamic link libraries due to the diversity of the file formats with extensions like.exe and.dll. The analysis of this dataset allows detecting all kinds of patterns, trends, and correlations that different file types have, which is going to make their features, functions, and place in the Windows environment clearer. This is a dataset that could be very valuable in malware detection, software analysis, and system security research and practice in Windows environments (Fig. 2).

	Name	md5	Machine	SizeOfOptionalHeader	Characteristics	MajorLinkerVersion	MinorLinkerVersion	SizeOfCode	SizeOfInitializedData	SizeOfUninitializedData	Res
0	memtest.exe	631ea3556652b470744b4427b5b8	332	224	258	9	0	361884	115712	0	...
1	one.exe	9d1099a6712e28bacc5641e3a7eab	332	224	3330	9	0	130560	19968	0	...
2	setup.exe	4d52518527353cdd8ba70d5d390	332	224	3330	9	0	517120	621568	0	...
3	DW20.DXE	a41e5248d40074b078050f0c9b12	332	224	258	9	0	585728	369152	0	...
4	dwimg20.exe	c87e56125828650ce9998f643a731	332	224	258	9	0	294912	247296	0	...
5	arappinstaller.exe	e6e3abab3b1a77127c5c4a29b237823	332	224	258	9	0	512	46592	0	...
6	Acrobat.exe	d87490772077e1e45b13ec973d89	332	224	290	9	0	222720	67072	0	...
7	Acrobat2.exe	540c5194c378c721c3e4883a348e	332	224	290	9	0	823808	650240	0	...
8	Acrobat3.exe	9a1c4c0d955d4d33a4d025233e	332	224	290	9	0	496	7168	0	...
9	Acrobat4.exe	b6d71a9e448558d8c9540d11b4	332	224	290	9	0	28896	10880	0	...
10	Acrobat5.exe	1b0d3c0e7a855557b3c4c8d4d33	332	224	290	9	0	917504	316828	0	...
11	Acrobat6.exe	155620411728b0ac5d56d04b4b2	332	224	290	9	0	53248	34816	0	...
12	LogTransport2.exe	a802a36f77780b0e158a0b7f522b	332	224	258	9	0	206448	102400	0	...
13	readme_x.exe	d99520a552085d8bdc4e43435e4d	332	224	258	9	0	14848	14336	0	...
14	Acrobat9.exe	0e9de49a847d6195da80a0bdeed3b	332	224	258	9	0	178688	134144	0	...

Fig. 2. Dataset sample

4.2 Developing the Model

The model developed in this study using the Random Forest Classifier will be reliable and effective in classifying Portable Executable files. The development process included preprocessing the dataset, dividing it into training and testing sets, and training the Random Forest Classifier.

Preprocessing involved deleting columns not relevant to the training of the model, such as “Name” and “md5”, while leaving other features that are taken from PE files. To represent the validity of PE files, the data was split into feature vectors X and matching target labels y .

The Scikit-Learn library’s train test split function was then used to divide the dataset further into training and testing sets for the purpose of training and evaluating the model. To show the ability of the Random Forest Classifier to assess accuracy using previously unseen data during training, it was instantiated with 100 estimators and an out-of-bag score.

Then, test set, X_{test} and y_{test} , was used to measure the accuracy of the classifier, which was previously trained on X_{train} and y_{train} . On the basis of this model, the results obtained showed that, with an accuracy as high as 99.53%, a model could pretty precisely distinguish between malicious and normal PE files.

After a thorough test, including a confusion matrix analysis, the final model was found to have very low rates for both false positives and false negatives, indicating that it is dependable in differentiating between malicious and legitimate PE files.

Also, using the Random Forest, a decision tree was plotted to visually assess the trained model. The tree allowed this by showing the hierarchical representation of decision boundaries made according to various features taken from PE files.

Finally, the very powerful Random Forest Classifier was serialized using the Python package pickle to save the trained model for future use. The excellent performance of the serialized model persisted after loading, verifying its consistency and reliability.

The produced model showed outstanding results when performing the classification of PE files, underlining its efficiency in cybersecurity applications for executable files detection that could be harmful.

4.3 Employing the Trained Model

The trained model's application is a critical step in the file classification pipeline that guarantees reliable threat detection in a dataset. The Random Forest classifier model was trained extensively on a wide range of datasets that included different attributes that were taken from Portable Executable (PE) files. It was chosen because it could manage high-dimensional feature sets well and showed impressive accuracy—during evaluation, it obtained an accuracy of about 99.53%. The learned information is encapsulated in the stored model, which is serialized using the pickle library as `model.pkl`, allowing for its reuse in various contexts or applications.

The trained model can be used to evaluate unfamiliar PE files with the `test.py` script. In order to extract detailed characteristics from a specific folder containing PE files, such as structural properties, header information, resource data, and version information from each file, this script interfaces with the `pefile` library. It loads the serialized model into memory and uses the extracted features to forecast whether each PE file in the specified folder is “malicious” or “legitimate.” Additionally, the script simplifies batch analysis and categorization by providing an optional graphical user interface (GUI) for user-friendly interaction.

This streamlined approach streamlines file analysis and classification, making it an essential tool for cybersecurity frameworks or threat detection systems. `Test.py` integration enables quick, precise classification of unknown files, enabling efficient decision-making and timely reactions to possible security threats (Fig. 3).

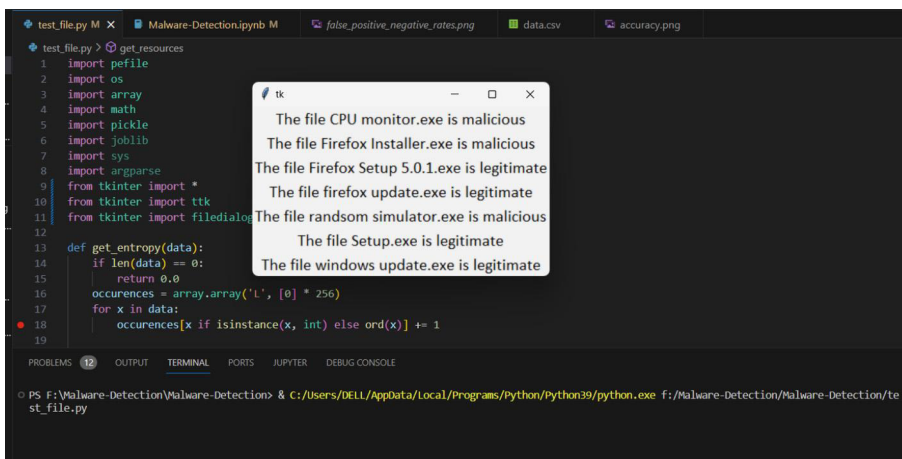


Fig. 3. GUI for the model

The `test.py` script opens a GUI and prompts the user to select a folder directory that contains the Portable Executable (.exe) files to be analyzed. The selected interface opens a window to mark each file's classification by trained Random Forest classifier predictions for its classification as either harmful or not. Feature extraction technique helps the script to gather inherent attributes of the designated.exe files. By exploiting

these features, the model will be able to predict the classification of each file with accuracy. These predictions are conveyed through the GUI, which enables proactive threat mitigation by highlighting potentially harmful files and allowing decisions to be made effectively within the cybersecurity framework (Fig. 4).

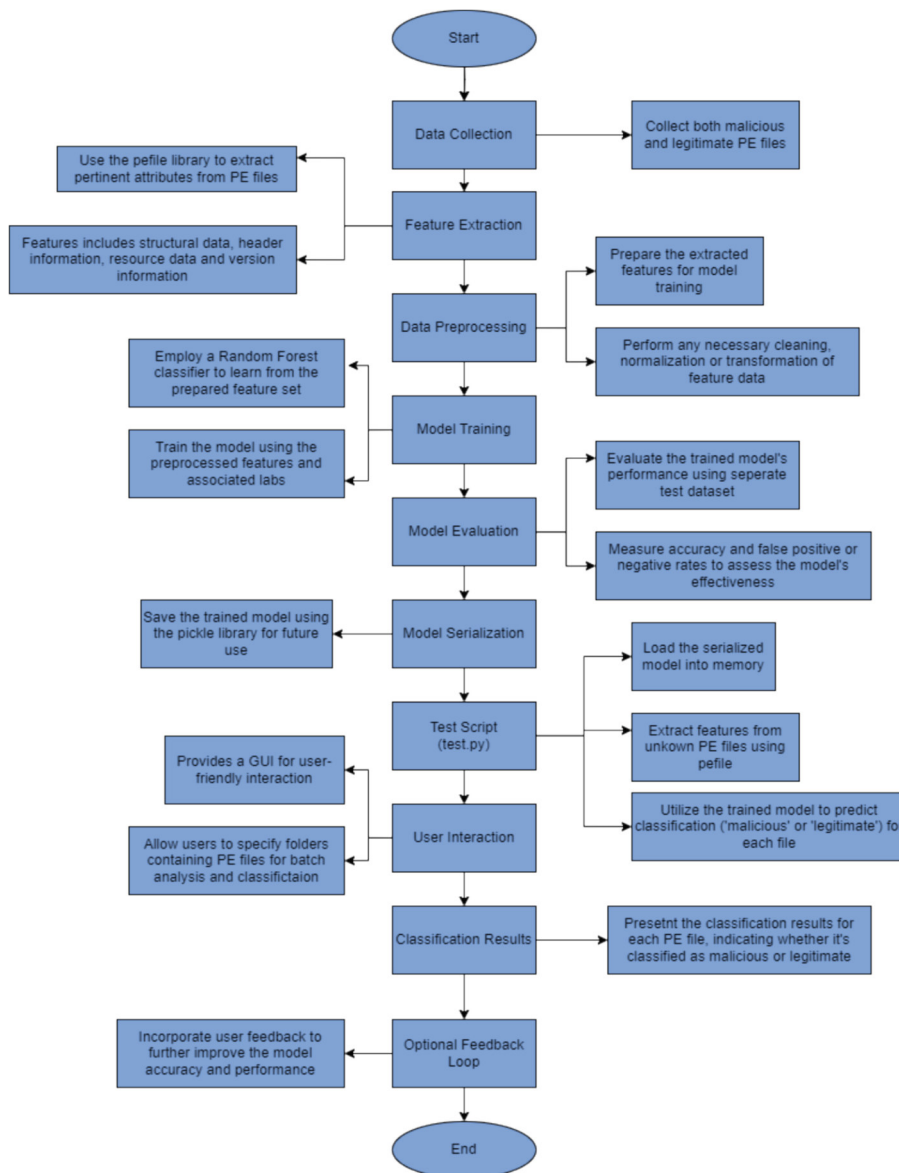


Fig. 4. Flow of the model

5 Results

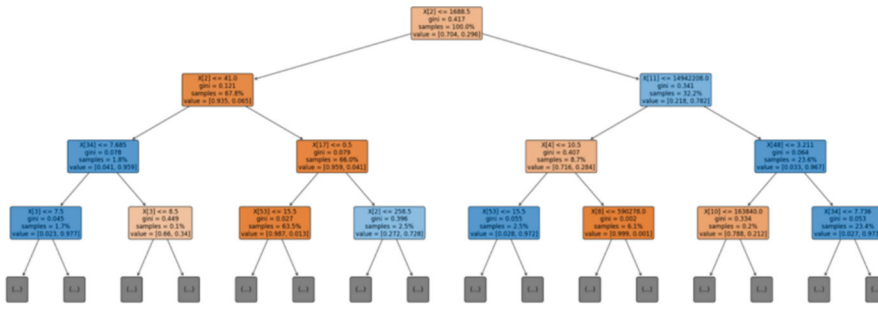


Fig. 5. Decision tree followed by the model

To provide insight into our model’s decision-making process, we present the decision tree that was followed to reach the results. The model uses a complex hierarchical structure and logic, going through features in cascaded steps to make predictions; the decision tree provides a visual representation of these processes. The decision points, or nodes, in the tree are where the model evaluates particular traits; the branches, on the other hand, stand for potential outcomes or subsequent decision points. Understanding the model’s input processing and ultimate prediction-making process is made easier by this methodical approach. The decision tree provides more information on critical nodes, split criteria, and terminal leaves, illuminating the logic and interpretability of the model’s ability to make decisions (Fig. 5).

99.53% accuracy is attained by the trained Random Forest model, demonstrating strong performance. The high accuracy highlights the effectiveness of the model in identifying harmful from normal Portable Executable (PE) files.

The model’s accuracy in dividing PE files into “malicious” and “legitimate” categories is seen in this confusion matrix. 19198 malicious files and 8283 legal files were accurately recognized by it. Furthermore, it only misclassified 64 malicious files and 65 benign files, indicating a low rate of incorrect predictions (Fig. 6).

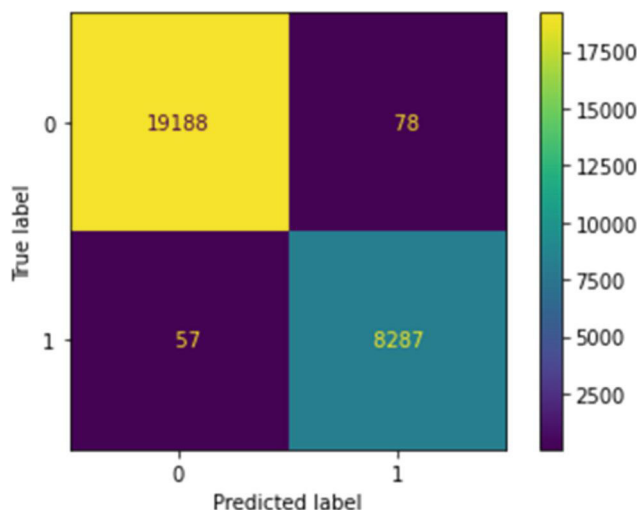


Fig. 6. Confusion Matrix

The model's excellent accuracy and the confusion matrix's balanced distribution confirm its dependability and appropriateness for demanding threat detection and file classification tasks inside cybersecurity frameworks.

6 Conclusion

The focus of this work is malware detection, where we employed a Random Forest classifier on multiple PE file attributes. The goal was to develop a robust and efficient model that could identify files that were potentially dangerous, whereas others were not, and further assist in cybersecurity efforts.

First of all, some important features had been extracted from the properties of PE files, including but not limited to structural, header, resource, and version information. Then it used these features as the base to train a strong ensemble learning method called the Random Forest classifier.

With an astounding accuracy of almost 99.53%, the final model performed exceptionally well. The aforementioned high accuracy highlights the effectiveness of the trained model in correctly classifying PE files, indicating its potential for practical implementations in cybersecurity frameworks.

The key features that influenced the classification process were highlighted by the decision tree visualization arising from the Random Forest model. This provided significant new light on the underlying processes driving whether PE files are characterized as "malicious" or "legitimate."

In addition, the test.py script allowed unknown PE files to be easily judged using the trained model. It also facilitated and simplified the batch analysis and classification process. The integration of a GUI within the script ensures ease-of-use and expedites the process of timely and accurate classification of unknown files.

The findings of this research provide a practical, reliable manner of detecting malware and reinforce further efforts toward increasing cybersecurity protection. The developed model, using the test.py tool, is a fundamental tool needed to rapidly identify and classify potentially malicious PE files, thereby enabling cybersecurity experts and organizations to make well-informed decisions to efficiently reduce security threats.

In the future, it will be important to enhance the cybersecurity framework, and also deal with new risks aggressively in the digital landscape, by undertaking more research, enhancing the classification models, and developing feature extraction techniques.

References

1. Senanayake, J., Kalutarage, H., Al-Kadri, M.O.: Android mobile malware detection using machine learning: a systematic review. *Electronics* **10**(13), 1606 (2021)
2. Saxe, J., Berlin, K.: Deep neural network based malware detection using two dimensional binary program features. In 2015 10th International Conference on Malicious and Unwanted Software (MALWARE), pp. 11–20. IEEE (2015)
3. Chauhan, D., Shah, M., Joshi, H.: A novel intrusion detection system based on machine learning for internet of things (IoT) devices. In 2023 3rd International Conference on Smart Data Intelligence (ICSMDI), pp. 427–434. IEEE (2023)
4. Roseline, S.A., Geetha, S., Kadry, S., Nam, Y.: Intelligent vision-based malware detection and classification using deep random forest paradigm. *IEEE Access* **8**, 206303–206324 (2020)
5. Roseline, S.A., Geetha, S.: Intelligent malware detection using oblique random forest paradigm. In: 2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI), pp. 330–336. IEEE (2018)
6. Joshi, S., Upadhyay, H., Lagos, L., Akkipeddi, N.S., Guerra, V.: Machine learning approach for malware detection using random forest classifier on process list data structure. In: Proceedings of the 2nd International Conference on Information System and Data Mining, pp. 98–102 (2018)
7. Gao, X., Hu, C., Shan, C., Liu, B., Niu, Z., Xie, H.: Malware classification for the cloud via semi-supervised transfer learning. *J. Inf. Secur. Appl.* **55**, 102661 (2020)
8. Rathore, H., Agarwal, S., Sahay, S.K., Sewak, M.: Malware detection using machine learning and deep learning. In: Big Data Analytics: 6th International Conference, BDA 2018, Warangal, India, 18–21 December 2018, Proceedings 6, pp. 402–411. Springer, Heidelberg (2018)
9. Gavriluț, D., Cimpoeșu, M., Anton, D., Ciortuz, L.: Malware detection using machine learning. In 2009 International Multiconference on Computer Science and Information Technology, pp. 735–741. IEEE (2009)
10. Santos, I., Sanz, B., Laorden, C., Brezo, F., Bringas, P.G.: Opcode-sequence-based semi-supervised unknown malware detection. In: Computational Intelligence in Security for Information Systems: 4th International Conference, CISIS 2011, Held at IWANN 2011, Torremolinos-Málaga, Spain, 8–10 June 2011. Proceedings, pp. 50–57. Springer, Heidelberg (2011)
11. Rathore, H., Samavedhi, A., Sahay, S.K., Sewak, M.: Robust malware detection models: learning from adversarial attacks and defenses. *Forensic Sci. Int. Dig. Invest.* **37**, 301183 (2021)
12. Labaca-Castro, R., Franz, S., Rodosek, G.D.: AIMED-RL: exploring adversarial malware examples with reinforcement learning. In: Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track: European Conference, ECML PKDD 2021, Bilbao, Spain, 13–17, September 2021, Proceedings, Part IV 21, pp. 37–52. Springer, Heidelberg (2021)

13. Koza, J., Krcál, M., & Holena, M. (2020). Two Semi-supervised Approaches to Malware Detection with Neural Networks. In ITAT (pp. 176–185)
14. Khammas, B.M.: Ransomware detection using random forest technique. *ICT Express* **6**(4), 325–331 (2020)
15. Shijo, P.V., Salim, A.J.P.C.S.: Integrated static and dynamic analysis for malware detection. *Procedia Comput. Sci.* **46**, 804–811 (2015)



Development of an Artificial Intelligence-Driven Live Patient Scheduling for Modern Healthcare

Abdul Khalique Shaikh¹ (✉), Murtala Bello Abubakar², and Naresh Adhikari³

¹ Department of Information Systems, Sultan Qaboos University, Muscat, Sultanate of Oman
shaikh@squ.edu.om

² Department of Physiology, Sultan Qaboos University, Muscat, Sultanate of Oman
m.abubakar@squ.edu.om

³ Department of Computer Science, Slippery Rock University of Pennsylvania, Pennsylvania,
PA 16057, USA
naresh.adhikari@sru.edu

Abstract. In today's era, Artificial Intelligence (AI) has emerged in addressing a range of challenges within the healthcare sector, particularly by improving diagnostic precision, personalizing treatment plans, and enhancing operational efficiency. The growing demand for healthcare services has brought to light significant issues in patient appointment scheduling, particularly in Oman, where hospitals face difficulties related to extended waiting times and inefficient management of patient flow. The increasing demand of healthcare offerings has highlighted substantial challenges in patient appointment scheduling, particularly in Oman, wherein hospitals struggle with prolonged wait and inefficient patient flow management. The proposed AI-driven live appointment management system provides the solution to the challenges. The proposed system targets a diverse range of customer segments, primarily focusing on healthcare providers and patients. By addressing the specific needs of the targeted groups, the proposed system aims to enhance overall healthcare experience, reduce waiting times, and improve operational efficiency. The system is based on the development of an AI-driven appointment management system designed to address critical challenges in patient scheduling within modern healthcare facilities including AI-Powered Queue Management, Predictive Wait Time Analysis, Intelligent Rescheduling, Integration with Existing Systems and User-Friendly Interfaces. The successful integration and implementation of all layers of the proposed MEDLINK system will significantly benefit the project, positioning it to serve a broad spectrum of stakeholders, including patients, healthcare providers, institutions, administrators, and technology developers. Through the utilization of AI to overcome the challenges associated with appointment scheduling, this initiative seeks to improve the overall healthcare experience, increase operational efficiency, and establish a new industry benchmark.

Keywords: Artificial Intelligence · Patient Scheduling · Queue Management · Wait Time Prediction · Healthcare

1 Introduction

Artificial Intelligence (AI) has a transformative impact on healthcare [1], providing a wide range of advantages that improve patient care, optimize operational processes [2], and foster innovation. Its significance is evident in several critical areas, including personalized treatment plans and operational efficiency. Enhanced Drug Discovery and Development, Patient Management, Predictive Analytics and Preventive Care Telemedicine and Virtual Health Assistants, Decision Support for Clinicians [3]. All these continued integrations into healthcare systems hold immense potential for advancing medical research, treatment, and patient care.

Effective patient appointment scheduling has become essential because of the high demands placed upon healthcare providers to maintain both quality of care and productivity [4, 5]. The conventional ways of managing appointments cannot cope with the changing trends in the healthcare system [6] hence reason they often lead to congestion and patient discontentment due to excessive waiting or over giving appointment slots to people. As it is observed by authors, public and private hospitals in Oman, are dealing with significant challenges in coping with patient wait, specifically for those laid low with chronic diseases. Patients often enjoy extended ready instances to see a physician that may exacerbate their conditions, result in useless health facility visits, and result in a standard decline within the high-quality of care.

This research presents an easier way to tackle the problem of scheduling appointments by proposing an appointment management system that is intelligent and can learn over time using artificial intelligence [7]. It is the intention of the authors is to mitigate examples of such as mentioned above by perfecting both the patients' experience and the physicians' management system through the usage of AI technologies. The system allows for outlining present scheduling that encompasses prediction of wait times, intelligent queue management system and integration with the existing processes and technology of the hospital. Such strategies are been put in place to handle the challenges that are associated with appointment systems in modern health care institutions.

The dependence on technology continue to increase with the rapid growth of Artificial Intelligence, therefore the need for this project cannot be ignored. Such reduction of operations is being realized through a shift towards artificial intelligence by some health care providers. The beneficiaries of this research will include the government hospital including patients, healthcare providers, institutions, administrators, and technology developers. The rest of the paper is organized as below:

Section 2 presents the earlier research related to patient scheduling. Section 3 discusses the research methodology including the layered architecture of the system and Sect. 4 describe the System Evaluation and Optimization. Section 5 concludes the research and presents the future research directions.

2 Literature Review

This section discusses the current systems of patient scheduling and appointment along with their challenges and opportunities.

Most of the outpatient clinics in Oman have scheduling policies for patient visits that usually leave the patients, operations coordinators and physicians irked because they are

kept waiting. Patients, especially family consider inaccurate allocation of service time as a costly waste [8]. Furthermore, prolonged waiting for both patients and physicians may adversely affect patients' overall satisfaction with service delivery and can also lead to frustration of the physicians [9]. There have been several attempts to minimize waiting duration and improve service delivery [10]. However, no satisfactory progress has been made due to capital intensive nature of the initiatives [11]. This poses a lot of challenges since a number of hospitals have limited budgets [12]. Consequently, steps taken to minimize waiting time have negatively affected patients' satisfaction, thus, necessitating the need for additional alternatives [13] such as subsidizing health facilities that have low patients turnout or low service utilization so as to encourage patients to patronize them [14].

Earlier and recent researches on patient scheduling have focused on different goals and study designs [4, 10, 15, 16]. One of the popular approaches is to minimize patient or clinician wait time or decrease the frequency of absentism [17]. However, this is associated with long waiting period for the physician (which can make him idle) if patients come late or do not come at all [18]. Consequently, this can lead to rushing through patients schedule and at the end rendering poor services to the patients [19]. An unhappy or unsatisfied patient has a tendency to leave a negative review [20].

One of the strategies to further minimize patients waiting time is the use of data modeling such as machine learning [21]. The use of data analytics (artificial intelligence and machine learning) in patient scheduling for various healthcare services can significantly contribute in a timely risk detection [22] and an overall improvement in the scheduling outcome [23]. In fact, several studies suggest that the algorithm models have provided more precisions in patients scheduling [24–29] with a significantly improved patient flow [30] and a relatively minimal cost [31].

The ever increasing demand for healthcare services [32, 33] has also necessitated the need for a well coordinated plan for delivering the services, and this plus many other challenges such as the policy makers' will and collaboration by all stakeholders constitute a major obstacle towards integrating AI into healthcare delivery scheduling [34, 35]. One of the fundamental issues to be addressed by researchers is determining the actual clinical components required for application of AI in scheduling systems [36, 37]. The current development and interest in AI technology therefore presents and opportunity to conduct researches that are aimed at optimization and application of the different algorithms into patient scheduling with the ultimate goal of revolutionizing clinical practice.

3 Research Methodology

This research study is based on experiments and it utilises a quantitative research approach [38] to develop and evaluate the effectiveness of an AI-driven appointment management system within healthcare facilities, specifically focusing on reducing waiting times and optimizing patient flow in Oman. The main purpose of this research is to create an innovative solution that integrates artificial intelligence (AI) into patient scheduling systems. The core components of the research involves the development of the AI-based live appointment scheduling system (MEDLINK), which was built through iterative

design and testing. The system includes several key components such as AI-Powered Queue Management, Predictive Wait Time Analysis, Intelligent Rescheduling, Integration with Existing Systems and User-Friendly Interfaces. The development process will follow an iterative model, with multiple phases of prototyping, testing, and refinement based on feedback from stakeholders, including healthcare professionals and patients.

The research study intends to utilize patient records from Sultan Qaboos University hospital. These patient historical data will be utilised to implement various machine learning algorithms to enhance scheduling efficiency and address the challenges associated with overbooking and no-show occurrences [7, 39]. As suggested by this research study [38] algorithms can analyse patterns in historical data to predict patient attendance and optimal scheduling times, reducing the likelihood of overbookings and missed appointments. This approach would lead to a more efficient, patient-friendly, and cost-effective appointment scheduling system.

3.1 Layered Architecture

The Layered architecture of Artificial Intelligence based live patient scheduling system provides a high-level picture of how the system functions, including the connections and interactions among the various modules (Fig. 1).

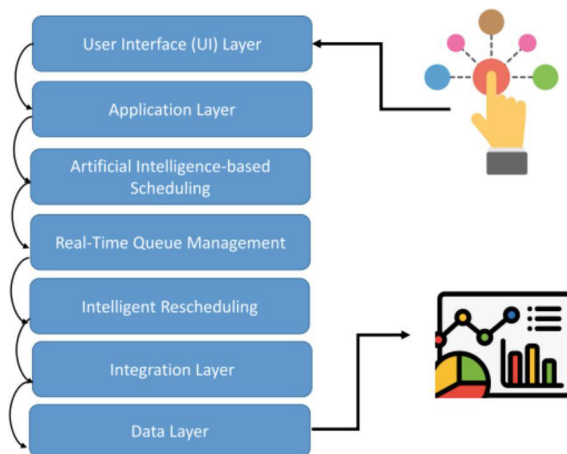


Fig. 1. Artificial Intelligence based live patient scheduling system.

User Interface (UI) Layer

This is the topmost layer where users interact with the system. It includes dashboards and interfaces for both patients and healthcare providers. The UI is designed to be user-friendly, allowing easy navigation for scheduling appointments, checking wait times, and receiving notifications. It serves as the point of interaction for input and output of information.

Application Layer

This layer acts as the intermediary between the UI and the underlying systems. It processes user inputs and manages the application logic. It handles requests from the UI, processes them, and communicates with the data and integration layers. This layer ensures that the application functions smoothly and efficiently.

Artificial Intelligence-Based Scheduling

This layer incorporates AI algorithms to optimize the scheduling process. It analyzes data to predict wait times, manage patient flow, and intelligently reschedule appointments based on real-time information. This layer is crucial for enhancing operational efficiency and improving patient experience.

Real-Time Queue Management

This layer focuses on managing patient queues in real-time. It monitors the status of patients in the queue, provides updates, and adjusts schedules dynamically to minimize wait times. This layer is essential for maintaining a smooth flow of patients through the healthcare facility.

Integration Layer

This layer facilitates communication between different systems and components. It ensures that the scheduling system can integrate with existing healthcare management systems, electronic health records (EHR), and other relevant technologies. This layer is vital for data consistency and interoperability.

Data Layer

This foundational layer stores and manages all the data used by the application. It includes databases that hold patient information, appointment records, and analytics data. The data layer is crucial for supporting the AI algorithms and ensuring that the application has access to accurate and up-to-date information.

4 System Evaluation and Optimization

The system's key performance indicators (KPIs) such as reduced patient wait times, enhanced resource utilization, and improved patient satisfaction are chosen to measure the system performance. As the proposed system has planned to use Real-world data that will collect from Sultan Qaboos Hospital during the pilot implementation, which include metrics such as appointment adherence, no-show rates, and resource allocation efficiency. This method support researchers to compare pre and post implementation performance to determine the system's practical benefits in reducing wait times and managing overbooking challenges.

The optimisation and improvement of the system will be handled by dynamic scheduling algorithms that adjust in real-time based on incoming data that can be utilized to enhance system efficiency. By analysing patient attendance patterns and real-time factors such as delays or cancellations, these algorithms optimize the allocation of time slots and resources dynamically, improving both system responsiveness and patient satisfaction.

Furthermore, the research study can consider to use predictive analytics techniques [40] to forecast patient behaviours, such as the likelihood of attending or missing appointments. By integrating these predictions into the scheduling process, systems can optimize the number of appointments scheduled per day and reduce inefficiencies caused by last-minute cancellations or no-shows.

After applying all evaluation and optimisation techniques, the proposed system measures the Operational Efficiency using various metrics such as resource utilization, appointment adherence, and the balance between supply (provider availability) and demand (patient appointments) are used to evaluate the operational impact of optimized scheduling systems. The evaluation and optimization of the AI-based scheduling system significantly improve the healthcare experience for both patients and providers, leading to better care, higher satisfaction, and more efficient healthcare delivery.

5 Conclusion and Future Directions

This research study proposes an AI-driven live patient scheduling system, MEDLINK, designed to address significant challenges in modern healthcare, such as extended waiting times and inefficient patient flow management, particularly in Oman. With the help of integration of various artificial intelligence techniques, along with predictive wait time evaluation, smart rescheduling, and actual-time queue management, the proposed model offers an advanced method to enhance operational performance and patient satisfaction. By leveraging patients' facts and machine learning algorithm, the model can optimize useful resource allocation, no-show, and streamline the scheduling process. This AI-based solution has the potential to revolutionize patient scheduling in healthcare settings, providing a foundation for future advancements and contributing to the development of more efficient, patient-centric healthcare services. The system could serve as a benchmark for the integration of AI in healthcare scheduling across various institutions. Future research could explore further improvements in system architecture and the integration of predictive analytics to address dynamic healthcare demands.

References

1. Secinaro, S., et al.: The role of artificial intelligence in healthcare: a structured literature review. *BMC Med. Inform. Decis. Mak.* **21**, 1–23 (2021)
2. Srinivas, S., Ravindran, A.R.: Optimizing outpatient appointment system using machine learning algorithms and scheduling rules: a prescriptive analytics framework. *Expert Syst. Appl.* **102**, 245–261 (2018)
3. Jiang, F., et al.: Artificial intelligence in healthcare: past, present and future. *Stroke Vascular Neurol.* **2**(4) (2017)
4. Gupta, D., Denton, B.: Appointment scheduling in health care: challenges and opportunities. *IIIE Trans.* **40**(9), 800–819 (2008)
5. Wang, D., Muthuraman, K., Morrice, D.: Coordinated patient appointment scheduling for a multistation healthcare network. *Oper. Res.* **67**(3), 599–618 (2019)
6. Yu, S., Kulkarni, V.G., Deshpande, V.: Appointment scheduling for a health care facility with series patients. *Prod. Oper. Manag.* **29**(2), 388–409 (2020)

7. Knight, D., et al.: Artificial Intelligence for Patient Scheduling in the Real-World Health Care Setting: A Metanarrative Review, p. 100824. *Health Policy and Technology* (2023)
8. Liu, N., et al.: When waiting to see a doctor is less irritating: Understanding patient preferences and choice behavior in appointment scheduling. *Manag. Sci.* **64**(5), 1975–1996 (2018)
9. McGinnis, J.M., et al.: Best care at lower cost: the path to continuously learning health care in America (2013)
10. Cayirli, T., Veral, E.: Outpatient scheduling in health care: a review of literature. *Prod. Oper. Manag.* **12**(4), 519–549 (2003)
11. Tohidi, M., Zanjani, M.K., Contreras, I.: A physician planning framework for polyclinics under uncertainty. *Omega* **101**, 102275 (2021)
12. Yang, L., Millstein, M.A., Campbell, J.F.: Unlocking cost savings hidden in hospital tier contracts. *Omega* **113**, 102713 (2022)
13. Senot, C., et al.: The impact of combining conformance and experiential quality on hospitals' readmissions and cost performance. *Manag. Sci.* **62**(3), 829–848 (2016)
14. Deng, Y., et al.: Optimal differential subsidy policy design for a workload-imbalanced outpatient care network. *Omega* **99**, 102194 (2021)
15. Ahmadi-Javid, A., Jalali, Z., Klassen, K.J.: Outpatient appointment systems in healthcare: a review of optimization studies. *Eur. J. Oper. Res.* **258**(1), 3–34 (2017)
16. Marynissen, J., Demeulemeester, E.: Literature review on multi-appointment scheduling problems in hospitals. *Eur. J. Oper. Res.* **272**(2), 407–419 (2019)
17. Samorani, M.: *Data Mining for Enhanced Operations Management Decision Making: Applications in Health Care*. University of Colorado at Boulder (2012)
18. Young, C., Chen, X.: Patients as consumers in the market for medicine: the halo effect of hospitality. *Soc. Forces* **99**(2), 504–531 (2020)
19. Drossman, D.A., Ruddy, J.: Improving patient-provider relationships to improve health care. *Clin. Gastroenterol. Hepatol.* **18**(7), 1417–1426 (2020)
20. Arendt, F., Forrai, M., Findl, O.: Dealing with negative reviews on physician-rating websites: an experimental test of how physicians can prevent reputational damage via effective response strategies. *Soc Sci Med* **266**, 113422 (2020)
21. Golmohammadi, D., Zhao, L., Dreyfus, D.: Using machine learning techniques to reduce uncertainty for outpatient appointment scheduling practices in outpatient clinics. *Omega* **120**, 102907 (2023)
22. Bellini, V., et al.: Machine learning in perioperative medicine: a systematic review. *J. Anesth. Analg. Crit. Care* **2**(1), 2 (2022)
23. Calegari, R., et al.: Surgery scheduling heuristic considering OR downstream and upstream facilities and resources. *BMC Health Serv. Res.* **20**, 1–11 (2020)
24. Abbas, A., et al.: Machine learning using preoperative patient factors can predict duration of surgery and length of stay for total knee arthroplasty. *Int. J. Med. Inform.* **158**, 104670 (2022)
25. Martinez, O., et al.: Machine learning for surgical time prediction. *Comput. Methods Prog. Biomed.* **208**, 106220 (2021)
26. Lai, J., et al.: Improving and Interpreting Surgical Case Duration Prediction with Machine Learning Methodology. *medRxiv*, 2020: 2020.06.10.20127910
27. Bartek, M.A., et al.: Improving operating room efficiency: machine learning approach to predict case-time duration. *J. Am. College Surg.* **229**(4), 346–354 (2019)
28. Tuwatananurak, J.P., et al.: Machine learning can improve estimation of surgical case duration: a pilot study. *J. Med. Syst.* **43**, 1–7 (2019)
29. Abedini, A., Li, W., Ye, H.: An optimization model for operating room scheduling to reduce blocking across the perioperative process. *Procedia Manuf.* **10**, 60–70 (2017)
30. Abdalkareem, Z.A., et al.: Healthcare scheduling in optimization context: a review. *Heal. Technol.* **11**, 445–469 (2021)

31. Otten, M., Braaksma, A., Boucherie, R.J.: Minimizing earliness/tardiness costs on multiple machines with an application to surgery scheduling. *Oper. Res. Health Care* **22**, 100194 (2019)
32. Information, C.I.F.H.: National Health Expenditure Trends. Canadian Institute for Health Information (1997)
33. Kadum, S.Y., et al.: Machine learning-based telemedicine framework to prioritize remote patients with multi-chronic diseases for emergency healthcare services. *Netw. Model. Anal. Health Inform. Bioinform.* **12**(1), 11 (2023)
34. Rajpurkar, P., et al.: AI in health and medicine. *Nat. Med.* **28**(1), 31–38 (2022)
35. Alami, H., et al.: Organizational readiness for artificial intelligence in health care: insights for decision-making and practice. *J. Health Organ Manag.* (2020). **ahead-of-print**(ahead-of-print)
36. Maadi, M., Akbarzadeh Khorshidi, H., Aickelin, U.: A review on human–AI interaction in machine learning and insights for medical applications. *Int. J. Environ. Res. Public Health* **18**(4), 2121 (2021)
37. Jansson, M., et al.: Artificial intelligence-enhanced care pathway planning and scheduling system: content validity assessment of required functionalities. *BMC Health Serv. Res.* **22**(1), 1513 (2022)
38. Ala, A., Chen, F.: Appointment scheduling problem in complexity systems of the healthcare services: a comprehensive review. *J. Healthcare Eng.* **2022**(1), 5819813 (2022)
39. Han, Y., et al.: A multi-appointment patient scheduling system with machine learning and optimization. *Decision Anal. J.* **10**, 100392 (2024)
40. Delen, D.: *Predictive Analytics: Data Mining, Machine Learning and Data Science for Practitioners*. FT Press (2020)



Blockchain Technology in Logistics Business

Nalyse Nakazato, Kashif Manzer^(✉), and Muhammad Abdul Basit Ur Rahim

Department of Computer Engineering and Computer Science,
California State University, Long Beach, CA 90840, USA

{Nalyse.Nakazato01,Kashif.Manzer01}@student.csulb.edu, m.basit@csulb.edu

Abstract. The logistics industry, valued at \$9.2 trillion annually, faces systemic inefficiencies due to fragmented information systems, manual documentation, and inconsistent data sharing, leading to over \$100 billion in losses. Blockchain technology offers a transformative solution with its decentralized architecture, cryptographic security, and smart contracts that automate processes while ensuring data integrity. This study presents a comprehensive examination of blockchain's role in enhancing logistics operations by exploring its technical foundations, real-world applications, and associated implementation challenges. Through a systematic review of existing literature and analysis of prominent case studies, the paper identifies key obstacles such as smart contract vulnerabilities, scalability constraints, and regulatory conflicts, particularly regarding GDPR compliance. Findings suggest that while blockchain can significantly improve supply chain traceability, transparency, and cost efficiency, widespread adoption is constrained by unresolved technical and legal barriers. This research contributes to the discourse by evaluating current solutions and proposing future directions to enable secure, scalable, and compliant blockchain integration in global logistics systems.

Keywords: Blockchain · Supply chain · Smart contract · Layer-2 · Compliance · IoT · Security

1 Introduction

A supply chain encompasses the entire network of entities, activities, information, and resources involved in the production and distribution of a product or service from the supplier to the end customer. The challenge that this traditional way of trade faces is a lack of transparency, inefficient data sharing, and inconsistencies in data as it moves between departments. The traditional systems face inefficiencies due to lengthy documentation processes and storage of this documentation, and delays caused by various stakeholders not having access to the same information due to different terms and policies at each stage of the supply chain lifecycle. Paper-based documentation and fragmented systems lead to increased chances of errors, fraud, and delays. These issues result in higher operational costs, reduced trust among stakeholders, and ultimately, an overall less efficient global supply chain.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
S. Hussain et al. (Eds.): SEET 2025, CCIS 2725, pp. 31–44, 2026.
https://doi.org/10.1007/978-3-032-08977-9_3

This research examines how blockchain can address longstanding challenges in the logistics industry, offering a transformative approach to modern supply chain management through the adoption of blockchain technology and smart contracts in logistics stems. This study investigates blockchain’s role in logistics by reviewing its technical foundations, alongside reviewing research paper of real-world implementations of blockchain in complex supply chain for its outcome and performance. The research also explores scalability solutions and regulatory challenges, drawing from academic literature and industry case studies.

Findings reveal that blockchain significantly enhances supply chain efficiency, with applications like MediLedger [17] ensuring drug authenticity and TradeLens [19] cutting administrative costs by 20%. Traceability is revolutionized, as seen in Walmart’s rapid response to the outbreak. Yet, challenges persist—scalability bottlenecks, interoperability, and GDPR conflicts demand ongoing innovation to fully realize blockchain’s potential in logistics.

2 Research Objectives and Questions

This study aims to critically examine blockchain technology’s potential, challenges, and practical implications within the logistics industry. The specific objectives guiding this research include understanding the technological barriers, evaluating current mechanisms for overcoming these barriers and analyzing real-world implementations to determine blockchain’s practical effectiveness.

The following research questions structure this investigation:

- RQ1: What are the primary technological, regulatory, and security challenges affecting blockchain adoption in logistics?
- RQ2: How effectively do existing smart contract security solutions mitigate vulnerabilities in logistics applications?
- RQ3: What lessons can be drawn from current blockchain deployments in logistics concerning scalability, transparency, and cost efficiency?

3 Research Methodology

This research adopts a systematic literature review method following the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) guidelines. Databases including IEEE Xplore, SpringerLink, ScienceDirect, and MDPI were systematically queried for relevant literature published between 2018 and 2024. Keywords such as “blockchain,” “smart contracts,” “logistics,” “supply chain,” “scalability,” and “GDPR compliance” were utilized to identify pertinent studies.

Initially, 324 studies were retrieved, and duplicates (56) were eliminated. Titles and abstracts were screened for relevance, resulting in the exclusion of 178 papers. Subsequently, a full-text review of 90 articles was conducted, applying specific inclusion criteria—such as relevance to logistics operations, recent

advancements in blockchain, and rigorous empirical or theoretical examination—leading to the selection of 24 studies for detailed analysis. Complementing this, the paper examines prominent industry case studies (e.g., TradeLens, MediLedger) to evaluate blockchain’s practical applications and limitations.

4 Related Work

Previous reviews have extensively analyzed blockchain’s theoretical foundations and its technological capabilities across diverse industries. For example, Kalra et al. [7] and Taherdoost [14] conducted comprehensive studies on blockchain and smart contract security, highlighting the complexity of mitigating vulnerabilities in various decentralized applications. Similarly, Vacca [1] systematically reviewed development tools and techniques for blockchain, underscoring both technological advancements and persistent challenges.

However, despite the wealth of general blockchain literature, specific investigations into blockchain’s unique implications for logistics remain limited. Notably, existing reviews often lack an in-depth analysis of industry-specific regulatory, ethical, and economic constraints, particularly relating to logistics’ operational contexts. This paper distinctly addresses these gaps by combining a detailed review of security mechanisms, scalability solutions, and regulatory concerns with an empirical evaluation of blockchain implementations specific to the logistics domain, thereby contributing uniquely to the existing academic discourse.

5 Challenges in Blockchain

5.1 Vulnerabilities in Smart Contracts

Blockchain systems in logistics, especially those using smart contracts to automate supply chain processes, face critical security challenges. Vulnerabilities such as re-entrancy attacks—where a malicious contract repeatedly calls a vulnerable one before execution completes—could disrupt automated payments or shipment verifications, leading to financial losses or delays. Tools like Oyente, Slither, and MAIAN help detect these flaws, with Oyente identifying over 8,000 vulnerable Ethereum contracts [1], a concern for logistics platforms built on similar frameworks.

Re-entrancy risks studied by [8] through the Reentrancy Analyzer (RA) could allow unauthorised fund withdrawals from a smart contract managing supplier payments, highlighting the need for robust static analysis tools. [4] notes RA’s use of symbolic execution to catch such exploits, yet tools like MAIAN struggle with scalability and nuanced flaws in complex supply chain systems [1].

Slither, with 99 vulnerability detectors, integrates into development pipelines to spot issues like gas inefficiencies that might slow down real-time tracking or IoT updates in logistics. [7] argues that securing smart contracts demands a layered approach—formal verification, runtime checks, and machine learning—to prevent errors that could erode trust in blockchain-based logistics networks.

These vulnerabilities underscore the need for ongoing research to protect supply chain operations. A single flaw in a smart contract managing customs clearance or provenance data could halt shipments, inflate costs, or compromise product integrity, threatening the reliability of global logistics systems.

5.2 Environmental Impact

Research by Taherdoost et al. [14] highlights inefficiencies in the relationship between gas costs and computational effort, especially on platforms like Ethereum. The challenges extend beyond individual contracts to the broader blockchain ecosystem. High energy consumption, particularly in proof-of-work (PoW) systems, raises environmental concerns and limits scalability. Alternative consensus mechanisms like proof-of-stake (PoS) aim to address these issues but require further refinement to balance efficiency with security. The shift toward Proof of Stake (PoS), exemplified by Ethereum’s 2022 upgrade, offers a greener alternative by eliminating energy-intensive mining. PoS reduces energy consumption by over 99%, relying on validators who “stake” coins rather than compete computationally. While challenges remain, the evolution toward energy-efficient protocols and sustainable practices highlights the dual role of blockchain as both a contributor to and a potential solution for environmental issues.

5.3 Solutions for Scalability

Scalability remains a pressing issue, with network congestion and high transaction costs limiting blockchain’s practical applications. To achieve scalability and security compatibility, lightweight privacy-preserving solutions in smart contracts have also been a focus. Innovations like the Sapphire smart-contract-based storage system demonstrate their utility in managing large datasets efficiently. Similarly, artificial intelligence (AI) can significantly enhance smart contracts, from vulnerability detection using machine learning to implement cognitive computing models for improved performance and scalability [14]. Layer-2 solutions, such as Optimistic and ZK-Rollups, process transactions off-chain while anchoring security to the main blockchain. Sharding, employed by Ethereum 2.0, divides the network into smaller chains (shards) that process transactions in parallel, boosting throughput from 15 to 100,000 TPS. Sidechains (e.g., Polygon) operate independently but connect to main chains, offering scalability for tasks like real-time IoT data logging.

5.4 Trade-Offs Between Security and Efficiency

Blockchain systems inherently face trade-offs between security and efficiency, requiring careful balance depending on use-case priorities. Formal verification tools like KEVM [5] and VERX [3] help to validate the functional properties of smart contracts by mathematically proving the correctness of code. These

integrated systems enhance the security of smart contracts while addressing performance bottlenecks. Layer-2 solutions like Rollups enhance scalability by processing transactions off-chain but rely on the security of the underlying Layer-1 blockchain, creating dependencies that may introduce vulnerabilities if not rigorously audited. Similarly, sharding (splitting the network into parallel chains) boosts efficiency but complicates consensus mechanisms, increasing attack surfaces. Cryptographic techniques like zero-knowledge proofs enhance privacy but add computational overhead, slowing transaction finality. Innovations like hybrid models (e.g., combining PoS with decentralized validators) and adaptive consensus algorithms aim to mitigate these trade-offs, but achieving optimal balance remains context-specific, demanding tailored solutions for diverse supply chain needs.

5.5 Performance Bottlenecks and Architectural Challenges

Blockchain technology faces a fundamental challenge in scalability known as the blockchain trilemma. This trilemma consists of three components: decentralization, security, and scalability. The trilemma explains that to increase two, one must be sacrificed. For example, increasing decentralization and security sacrifices scalability. If decentralization and scalability are prioritized, security is compromised. If security and scalability are enhanced, decentralization is reduced. Blockchain systems often encounter performance bottlenecks due to their architectural choices. For instance, Hyperledger Fabric demonstrates higher throughput and lower latency under increasing workloads, illustrating trade-offs between public and permissioned blockchains [13]. This means that such blockchain systems struggle to achieve a dynamic, fast, and efficient connection and safe data transfer significant in the supply chain.

6 Secure Smart Contracts

6.1 Static and Dynamic Code Analysis

Smart contracts require intricate mechanisms for testing, code analysis, and validation to ensure their reliability and security. To address the persistence of flaws, researchers have investigated empirical methods for identifying contract defects. [25] catalogued 20 common defects related to security and usability, many of which are not detectable with existing tools like Oyente and Zeus. This gap underscores the need for advanced detection techniques. Smart contract code analysis is an essential domain, where static and dynamic analysis tools like SmartCheck [6] and Slither [9] have proven invaluable. Static analysis tools like SmartCheck [6] systematically examine source code for vulnerabilities by translating it into intermediate representations. Similarly, dynamic tools such as Slither [9] offer real-time insights into code execution, allowing developers to detect flaws and optimize performance. These tools enhance code comprehension, identifying common pitfalls, such as reentrancy vulnerabilities and gas inefficiencies, and providing actionable feedback.

6.2 Advanced Detection Mechanisms

Moreover, smart contract analysis has significantly improved the ability to identify vulnerabilities and optimize performance. One notable development is structural comparison techniques, as exemplified by SmartEmbed [26]. By analyzing structural similarities in smart contract code, SmartEmbed achieves a 90% detection success rate for identifying clones and bugs, making it a critical tool in mitigating risks. Structural analysis allows for early detection of vulnerabilities, which can save significant resources during the development lifecycle.

In addition to structural comparison, fuzz testing methods are increasingly adopted to uncover hidden vulnerabilities. Tools like EVMFuzz and ContractFuzzer have identified vulnerabilities in real-world contracts, contributing to improved resilience against attacks. These tools generate randomized inputs to test the boundaries and robustness of contract execution. [27] noted that ContractFuzzer detected security issues in over 90% of real-world contracts it tested, emphasizing the necessity of stress testing in production environments. The maturity of tools and programming languages for smart contract development also continues to evolve, providing developers with more robust frameworks. Platforms such as Hyperledger Fabric offer alternative paradigms for smart contract implementation, emphasizing modularity and privacy. Unlike public blockchains like Ethereum, Hyperledger Fabric uses a permissioned blockchain model, which, according to [27], provides controlled environments where transaction privacy and confidentiality are prioritized. Comparative studies of smart contract platforms highlight Ethereum's dominance in flexibility and Hyperledger's superiority in corporate applications, concluding that choosing the right platform depends on the intended use case, with Ethereum excelling in public and decentralized settings, while Hyperledger is more suited for private and regulated environments.

To ensure seamless integration of detection mechanisms into the smart contract development process, new development environments are being designed with built-in analysis capabilities. Tools like Remix IDE incorporate live error detection, enabling developers to address potential vulnerabilities during the coding phase. This proactive approach minimizes the need for extensive post-deployment debugging and enhances the overall security of the blockchain ecosystem. By combining advanced techniques like structural analysis, fuzz testing, and integrated development environments, the integration of smart contracts restructures blockchain into more automated, scalable, and reliable solutions.

6.3 Consensus Mechanisms, Layer-2 and Layer-3 Solutions

Blockchain's scalability and energy efficiency depend on consensus mechanisms and Layer-2 solutions. These technologies balance security, decentralization, and performance to meet the demands of different blockchain use cases. Consensus mechanisms and Layer-2 solutions are pivotal for enhancing blockchain's efficiency in logistics. Proof of Work (PoW), used by Bitcoin, relies on energy-intensive mining to validate transactions, offering high security but impractical

scalability (7 TPS) and sustainability for supply chains [16]. Hyperledger Fabric uses the Raft consensus mechanism, which provides higher throughput than Kafka by removing Kafka’s overhead. In contrast, Proof of Stake (PoS), adopted by Ethereum 2.0, selects validators based on staked tokens, reducing energy use by 99% and enabling faster, cost-effective transactions ideal for real-time tracking and automated payments.

To further address scalability, Layer-2 solutions like Rollups process transactions off-chain while anchoring security to the main blockchain. Optimistic Rollups streamline bulk operations (e.g., shipment updates) by assuming validity unless disputed, while ZK-Rollups use cryptographic proofs for instant verification, enabling thousands of low-cost transactions per second. These innovations reduce bottlenecks in global logistics—such as tracking perishable goods or automating customs—but face challenges like integration with legacy systems and the complexity of implementing advanced cryptography. By combining PoS with Rollups, blockchain becomes a scalable, eco-friendly tool for supply chains [11].

Layer 3 in blockchain refers to application-specific scaling solutions built on Layer 2 networks. In supply chains, Layer 3 enhances interoperability, automation, and efficiency by enabling customized smart contracts, data privacy, and faster transactions. It can improve scalability by offloading heavy lifting from Layer 1, allowing seamless multi-chain interactions, reducing costs, and optimizing supply chain traceability and transparency [13].

7 Applications of Blockchain Technology in Supply Chain

Blockchain’s applications span sectors like healthcare, IoT, and payment systems. Innovations like MobiChain enable secure supply chain transactions. Blockchain-based database solutions, such as BigchainDB and Ethereum Query Language (EQL), improve data retrieval and storage in decentralized environments. Despite these advances, further research into smart contract integration and its regression testing are essential for improving blockchain technology in terms of addressing security, transparency, and trade-offs between gas consumption and code efficiency.

One of blockchain’s most significant applications in logistics is its ability to provide indisputable and immutable proof of the provenance of raw materials, products, and sales [11]. This feature is particularly valuable in sectors like pharmaceuticals and agriculture, where counterfeit products pose serious risks. By securing digital records with cryptographic techniques, blockchain prevents tampering, offering consumers guarantees of product quality and authenticity. As stated in [11], “digital records provide security and restore trust among the intermediaries and primary stakeholders involved in a logistical supply chain.”

High-profile incidents like the Salmonella outbreak in papayas and the E. coli spread linked to Chipotle demonstrate how blockchain could have mitigated financial damage and loss of trust by isolating contaminated products more effectively.

Blockchain transforms supply chain management by enhancing visibility, transparency, and security. Traditional platforms face high transaction costs and risks, while blockchain eliminates these challenges through trustless networks and immutable records. One key factor is the concept of smart contracts. Smart contracts, introduced by Szabo in 1996 [28], are self-executing contracts with predefined rules encoded directly into the blockchain. These automated agreements can streamline various aspects of logistics operations, from payment processing to customs clearance. Smart contracts could automatically trigger payments when predefined conditions are met, such as the successful delivery of goods or the completion of customs clearance. For example, smart wholesale price and revenue-sharing contracts improve coordination and economic feasibility in supply chains because they enable automated adjustments and ensure compliance with predefined terms, optimizing operations and mitigating risks [11]. The advantage of smart contracts is that they eliminate the need for intermediaries and ensure the immutability of terms once stored in the blockchain, enhancing trust and reducing the risk of fraud. This automation reduces the potential for human error, accelerates transaction times, and improves cash flow management.

7.1 Traceability and Provenance Through Blockchain

Blockchain technology revolutionizes traceability and provenance through its core principle of an immutable, decentralized ledger that records every transaction across all nodes. Traceability refers to tracking a product's journey from origin to end-user, while provenance verifies its authenticity and origin. The pharmaceutical industry faces significant risks from counterfeit drugs, which account for nearly 10% of global medical products in low- and middle-income countries [15]. Blockchain addresses this by assigning unique digital identifiers (e.g., serial numbers) to each product, recorded on the blockchain at every stage—from manufacturing to distribution. For example, MediLedger, a blockchain consortium involving Pfizer and Genentech, uses smart contracts to verify drug authenticity, ensuring compliance with the U.S. Drug Supply Chain Security Act (DSCSA). Stakeholders scan products at each checkpoint, updating the blockchain and triggering automated compliance checks [17].

Walmart reduced traceability time for mangoes from 7 days to 2.2 s by using IBM Food Trust's blockchain. Each fruit's origin, harvest date, and transit conditions are stored immutably, allowing instant recalls during outbreaks [12].

7.2 Smart Contracts in Logistics

In logistics, payments are often based on achieving milestones (e.g., delivery of goods). Smart contracts automate this by linking payments to verified data inputs. For instance, IoT sensors on shipments can relay real-time data (e.g., location, temperature) to the blockchain. Once goods arrive undamaged, the smart contract releases payment to the carrier. Maersk's TradeLens platform exemplifies this: it automates freight payments upon port arrival verification, reducing processing time from days to minutes. Similarly, De Beers' Tracr uses

smart contracts to distribute diamond sales revenue automatically between miners and distributors, ensuring transparency and trust. Despite benefits, smart contracts face hurdles. Legal frameworks lag behind technology; disputes over code-based agreements remain unresolved. Interoperability between blockchain platforms (e.g., Ethereum vs. Hyperledger) complicates cross-platform standardization.

7.3 Integration with IoT and RFID Technologies

Blockchain's integration with IoT and RFID technologies enables real-time data to feed into the blockchain, creating immutable records for tracking and decision-making, enhancing transparency and trust between different parties. QR codes allow consumers to scan products and access their history (e.g., origin, certifications). IBM Food Trust uses QR codes on meat packaging to trace products back to farms, displaying details like slaughter dates and transport conditions.

RFID tags embedded in pallets or containers transmit location, temperature, and handling data to the blockchain. For pharmaceuticals, MedsTracker uses RFID-enabled packaging to monitor storage conditions. If temperatures deviate, smart contracts alert stakeholders or reroute shipments, ensuring compliance with safety standards. Walmart's cold chain management system leverages RFID tags to track frozen goods, updating the blockchain every 30 s. During a salmonella outbreak, this system traced contaminated lettuce to its source in 2.2 s [12].

8 Real-World Implementations

Blockchain technology has demonstrated transformative potential in logistics through high-profile, real-world applications. Maersk's TradeLens, a blockchain platform co-developed with IBM, digitizes global shipping workflows, replacing error-prone paper-based processes with a shared, immutable ledger. By enabling real-time data exchange among 100+ stakeholders (carriers, ports, customs), TradeLens reduces transit delays by 40% and cuts administrative costs by 20%, streamlining operations across 600 ports worldwide [19]. Similarly, Walmart's collaboration with IBM Food Trust revolutionized food traceability. During a 2018 E. coli outbreak linked to romaine lettuce, Walmart traced the contamination source in 2.2 s—a task that previously took days—preventing widespread recalls and safeguarding its reputation [12]. The system uses blockchain to log every step of a product's journey, from farm to shelf, with IoT sensors ensuring temperature and handling compliance.

In the diamond industry, De Beers' Tracr blockchain tracks stones from mine to retail, assigning a digital fingerprint to each diamond. This system combats fraud and conflict diamonds by providing irrefutable provenance records, boosting consumer confidence [20].

Pharmaceutical logistics also benefit from blockchain. MedsTracker integrates RFID tags and blockchain to monitor drug shipments in real-time. If

a temperature-sensitive vaccine deviates from safe conditions, the system alerts stakeholders instantly, reducing spoilage and ensuring compliance with stringent regulatory standards [11]. Meanwhile, DHL’s blockchain pilot for high-value pharmaceuticals reduced shipment verification times by 90%, demonstrating blockchain’s ability to enhance both speed and security [21].

In customs clearance, the EU’s Blockchain for Customs initiative automates document verification using smart contracts, slashing processing times by 65%. For instance, shipments eligible for trade agreements are auto-approved, minimizing human error and bureaucratic delays [24].

Despite these successes, challenges persist. UPS’s blockchain platform for cross-border logistics faced scalability issues with high-frequency IoT data, prompting a shift to hybrid solutions combining Ethereum and Hyperledger [22].

These case studies underscore blockchain’s capacity to enhance transparency, reduce costs, and mitigate risks in logistics. While adoption barriers like scalability and standardization remain, early adopters prove that blockchain is not just theoretical—it’s a practical tool reshaping global supply chains (Table 1).

Table 1. Real-World Blockchain Implementations in Logistics

Case Study	Sector	Key Outcomes	Reference
<i>TradeLens</i>	Shipping & Freight	20% reduction in administrative costs; 40% fewer delays in global shipping	[19]
<i>MediLedger</i>	Pharmaceuticals	Improved drug traceability and authenticity compliance with regulatory standards	[17]
IBM Food Trust (Walmart)	Retail/Food Safety	Traceability improved from 7 days to 2.2s; reduced outbreak response time	[12]
<i>Tracr</i> (De Beers)	Diamonds	Reduced diamond fraud; increased consumer trust via immutable provenance records	[20]
DHL Blockchain Pilot	Pharmaceutical Logistics	90% reduction in shipment verification times; enhanced compliance and safety	[21]
UPS Cross-border Logistics	Logistics	Identified scalability challenges; hybrid blockchain for IoT integration	[22]

9 Regulatory, Economic, and Ethical Considerations

9.1 GDPR Compliance and Data Immutability

Blockchain’s immutability—ensuring data cannot be altered—directly conflicts with the EU’s General Data Protection Regulation (GDPR), particularly the

“right to be forgotten.” This regulation mandates that individuals can request personal data deletion, which is incompatible with blockchain’s permanent ledger [24]. For instance, logistics platforms storing customer addresses or transaction histories on a public blockchain risk violating GDPR if they cannot erase such data. Solutions include hybrid blockchains (e.g., VeChain), where sensitive data is stored off-chain, linked via hashes to the immutable chain, allowing deletion without compromising integrity. Alternatively, zero-knowledge proofs enable data validation without exposing personal details.

Projects like LTO Network combine GDPR-compliant workflows with blockchain by using temporary anchors for non-critical data [2]. Despite these innovations, challenges persist: permissioned blockchains may centralize control, undermining decentralization ideals, while public chains face legal ambiguities [10]. As regulations evolve, interoperability between legal frameworks and blockchain design will be critical for global logistics operations.

9.2 ROI and Cost-Benefit Analysis

Blockchain adoption in logistics offers significant return on investment (ROI) through automation, reduced fraud, and streamlined operations. DHL’s blockchain pilot reduced supply chain costs by 20% by automating documentation and minimizing delays in customs clearance [21]. Cost savings stem from eliminating intermediaries (e.g., banks for letters of credit), reducing manual errors, and accelerating dispute resolution via immutable records. However, upfront costs—such as IoT integration, staff training, and blockchain development—can be substantial. A 2022 McKinsey study found that companies recoup these investments within 2–3 years through operational efficiencies. Smaller firms face higher barriers due to limited resources, but cloud-based blockchain services (e.g., AWS Blockchain Templates) lower entry costs [23].

Beyond direct savings, blockchain enhances brand trust by ensuring product authenticity, as seen in Walmart’s food traceability system, which reduced recall costs by 30% [12]. While ROI varies by scale and use case, the long-term economic benefits of transparency, speed, and risk mitigation position blockchain as a strategic investment for modern supply chains.

9.3 Ethical Concerns

Blockchain’s promise of decentralization often clashes with corporate realities. While public blockchains like Ethereum emphasize democratized control, enterprise adoption frequently leads to permissioned systems (e.g., IBM’s Hyperledger), where corporations govern nodes and data access [19]. This centralization risks creating “walled gardens,” where a few entities dominate decision-making, contradicting blockchain’s egalitarian ethos [18]. Additionally, energy-intensive PoW blockchains conflict with sustainability goals, though shifts to PoS (e.g., Ethereum 2.0) mitigate this. Ethical dilemmas also arise in data ownership: who controls IoT-generated supply chain data—corporations, suppliers, or consumers?

Projects like Fairchain aim to democratize data access, ensuring farmers or small suppliers retain ownership. However, power imbalances persist, as seen in De Beers' Tracr, where miners have limited influence over the diamond-tracking platform [20]. Balancing decentralization with practical governance remains a key challenge, necessitating frameworks that prioritize transparency, equitable participation, and environmental responsibility to uphold blockchain's ethical potential in logistics.

10 Conclusion

This study provides a comprehensive evaluation of blockchain technology's transformative potential and associated implementation challenges within the logistics sector. It emphasizes that blockchain significantly enhances supply chain efficiency, transparency, and security by addressing longstanding inefficiencies related to data fragmentation, manual processes, and lack of traceability. Real-world case studies, such as TradeLens, MediLedger, and IBM Food Trust, underscore blockchain's substantial benefits, including reduced operational costs and improved traceability response times.

Nevertheless, this research also identifies critical barriers—including scalability constraints, smart contract vulnerabilities, and regulatory challenges like GDPR compliance—that currently limit blockchain's widespread adoption. Therefore, continued technological innovation, rigorous development of secure smart contracts, and evolution of regulatory frameworks are imperative. Future studies should explore advanced interoperability solutions across blockchain platforms, hybrid models balancing decentralization with practical governance, and comprehensive economic analyses to quantify blockchain's long-term impacts on logistics. Addressing these areas will facilitate blockchain's transition from experimental adoption to industry-wide standardization, thereby realizing its full potential for a more resilient, transparent, and efficient global supply chain.

Acknowledgments. This study was supported by the Department of Computer Engineering and Computer Science at California State University, Long Beach.

Competing Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Vacca, A.: A systematic literature review of blockchain and smart contract development: techniques, tools, and open challenges. *J. Syst. Softw.* **174**, 110891 (2021). <https://doi.org/10.1016/j.jss.2020.110891>
2. Černý, M.: Potential use of RFID and QR code in the supply chain based on blockchain and smart contract. *Procedia Comput. Sci.* **220**, 497–502 (2023). <https://doi.org/10.1016/j.procs.2023.03.063>

3. Permenev, A., Dimitrov, D., Tsankov, P., Drachsler-Cohen, D., Vechev, M.: VerX: safety verification of smart contracts. In: Proceedings of IEEE Symposium Security Privacy (SP), pp. 1661–1677. IEEE, San Francisco (2020). <https://doi.org/10.1109/SP40000.2020.00066>
4. Chinen, Y., Yanai, N., Cruz, J.P., Okamura, S.: RA: Hunting for re-entrancy attacks in Ethereum smart contracts via static analysis. In: Proceedings of IEEE International Conference Blockchain, pp. 327–336. IEEE, Rhodes (2020). <https://doi.org/10.1109/Blockchain50366.2020.00048>
5. Hildenbrandt, E., et al.: KEVM: a complete formal semantics of the Ethereum virtual machine. In: Proceedings of IEEE 31st Computer Security Foundations Symposium (CSF), pp. 204–217. IEEE, Oxford (2018). <https://doi.org/10.1109/CSF.2018.00022>
6. Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y.: SmartCheck: static analysis of ethereum smart contracts. In: Proceedings of IEEE/ACM 1st International Workshop Emerging Trends Software Engineering Blockchain (WETSEB), pp. 9–16. IEEE, Montreal (2018). <https://doi.org/10.1145/3194113.3194115>
7. Kalra, S., Goel, S., Dhawan, M., Sharma, S.: ZEUS: analyzing safety of smart contracts. In: Proceedings of Network Distribution on Systems and Security Symposium (NDSS). Internet Society, San Diego (2018). <https://doi.org/10.14722/ndss.2018.23092>
8. Prasad, B.: Prevention and detection mechanisms for re-entrancy attack and king of Ether throne attack for Ethereum smart contracts. *Int. J. Adv. Comput. Sci. Appl.* **13**(5), 1–8 (2022). <https://doi.org/10.14569/IJACSA.2022.0130501>
9. Feist, J., Grieco, G., Groce, A.: Slither: a static analysis framework for smart contracts. In: Proceedings of IEEE/ACM 2nd International Workshop Emerging Trends Software Engineering Blockchain (WETSEB), pp. 8–15. IEEE, Seoul (2019). <https://doi.org/10.1109/WETSEB.2019.00008>
10. De Giovanni, P.: Blockchain and smart contracts in supply chain management: a game-theoretic model. *Eur. J. Oper. Res.* **284**(1), 366–376 (2020). <https://doi.org/10.1016/j.ejor.2019.12.027>
11. Alqarni, M.A.: Use of blockchain-based smart contracts in logistics and supply chains. *Appl. Sci.* **13**(6), 3542 (2023). <https://doi.org/10.3390/app13063542>
12. Casado-Vara, R., Prieto, J., De la Prieta, F., Corchado, J.M.: How blockchain improves the supply chain: case study alimentary supply chain. *Procedia Comput. Sci.* **134**, 393–398 (2018). <https://doi.org/10.1016/j.procs.2018.07.193>
13. Gao, J.: Blockchain-enabled internet of vehicles applications. *Electronics* **12**(5), 1122 (2023). <https://doi.org/10.3390/electronics12051122>
14. Taherdoost, H.: Smart contracts in blockchain technology: a critical review. *Information* **14**(2), 117 (2023). <https://doi.org/10.3390/info14020117>
15. World Health Organization: Substandard and falsified medical products. WHO, Geneva, Switzerland (2020). <https://www.who.int/news-room/fact-sheets/detail/substandard-and-falsified-medical-products>
16. Nakamoto, S.: Bitcoin: a peer-to-peer electronic cash system (2008). <https://bitcoin.org/bitcoin.pdf>
17. Chronicled Inc.: MediLedger Project: Blockchain solutions for the pharmaceutical supply chain. Chronicled, San Francisco, CA, USA (2023). <https://www.chronicled.com/mediledger>
18. Buterin, V.: Rollups: The future of Ethereum scaling. Ethereum Foundation Blog (2021). <https://vitalik.eth.limo/general/2021/01/05/rollup.html>

19. IBM: TradeLens: Revolutionizing global trade. IBM, Armonk, NY, USA (2022). <https://www.ibm.com/case-studies/tradelens>
20. De Beers Group: Tracr: Ethical diamond sourcing. De Beers, London, UK (2021). <https://www.debeersgroup.com/tracr>
21. DHL: Blockchain in logistics: Cost reduction and efficiency. DHL, Bonn, Germany (2021). <https://www.dhl.com/global-en/home/insights-and-innovation/innovation/blockchain.html>
22. UPS: Blockchain in cross-border logistics: Challenges and solutions. UPS, Atlanta, GA, USA (2022). <https://www.ups.com/us/en/supply-chain-solutions/blockchain.page>
23. Amazon Web Services: AWS blockchain templates: Lowering entry barriers. AWS, Seattle, WA, USA (2023). <https://aws.amazon.com/blockchain/templates/>
24. European Commission: GDPR and blockchain compliance guidelines, Brussels, Belgium (2023). <https://ec.europa.eu/info/law/law-topic/data-protection>
25. Chen, T., et al.: An empirical study of smart contract vulnerabilities. In: Proceedings of IEEE International Conference on Software Analysis, Evolution Reengineering (SANER), pp. 270–281. IEEE, Honolulu (2020). <https://doi.org/10.1109/SANER48275.2020.9054840>
26. Gao, J., et al.: SmartEmbed: a tool for clone and bug detection in smart contracts through structural code embedding. In: Proceedings of IEEE International Conference on Software Maintenance Evolution (ICSME), pp. 394–397. IEEE, Raleigh (2019). <https://doi.org/10.1109/ICSME.2019.00067>
27. Vacca, A., Di Sorbo, A., Visaggio, C.A., Canfora, G.: A systematic literature review of blockchain and smart contract development: techniques, tools, and open challenges. *J. Syst. Softw.* **174**, 110891 (2021). <https://doi.org/10.1016/j.jss.2020.110891>
28. Szabo, N.: Smart contracts: Building blocks for digital markets. *Extropy* **8**(2) (1996). http://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart_contracts_2.html



Predicting User Affective States from Mobile Notification Interactions Using LLM-Based Machine Learning Models

Muhammad Faizan Khan^{1(✉)}, Israr Ahmad², Muhammad Asif Khan², Ghulam Mudassir³, and Gohar Hayat Khan⁴

¹ Aberdeen Institute of Data Science and Artificial Intelligence, South China Normal University, Guangzhou, China
20220246@m.scnu.edu.cn

² South China University of Technology, Guangzhou, China

³ University of Buckingham, Buckingham, UK

⁴ University of Kassel, Kassel, Germany

Abstract. The rapid adoption of smartphones has created opportunities for understanding the emotional state of the user's through smartphone interactions. This work presents a novel approach using Large Language Models (LLMs) and smartphone notification interaction data for the estimation of the affective state of the user, an area that has not been extensively studied. Recent studies face challenges in generating personalized predictions; instead, they deliver generalized outcomes that are not suitable for an individual's behavior. Most models failed to incorporate real-time variables, which limits the accuracy and relevance of user's emotional state predictions. This work applies the use of LLMs to analyze user's complex, dynamic interactions with smartphone notifications, resulting in a greater understanding of emotions such as happiness, irritation, and frustration. Our work uses real-time smartphone notification data, "NotifyMiner," and LLMs to deliver context-aware, personalized emotional state predictions. In our experiments, the LLM is trained on interaction data from user's, which includes response latency, level of engagement, and type of notification, and its predictive capacity is assessed for recognizing various emotional states. We evaluate zero-shot and few-shot embedding techniques regarding their use in LLM to measure and predict user well-being status. The model proved to be successful in identifying various affective states, highlighting its potential for customized alerts, mental health tracking, and enhanced usability features. Through context-aware and personalized predictions, LLMs demonstrate their ability to overcome standard modelling restrictions and analyze complex user behaviors.

Keywords: Large Language Models · Context-aware predictions · Emotional state · Profile of Mood States (POMS) · Behavior analysis · Global Positioning System (GPS)

1 Introduction

The rapid spread of smart devices and the increasing usage of smartphones have generated a lot of information about user behavior and interactions. Mobile notifications are of particular interest since they contain rich information about user engagement, response patterns, and emotional state. Predictions of the user's emotional state from the interactions have also attracted a lot of interest due to their potential to enhance user experience, improve personalized delivery of information, and support the tracking of mental well-being [1]. In recent research works, various strategies have been explored to predict the user's emotional state with smartphone information, but they are prone to operate with either static models or simplistic models that do not support the dynamic interactions of user behavior with mobile alerts. Dynamic interaction refers to the continuously changing and context-dependent ways in which a user engages with smartphone notifications. It captures variations in user behavior over time, influenced by factors such as emotional state, notification type, urgency, time of day, and surrounding environment. Unlike static interaction patterns, dynamic interactions are flexible and adaptive, reflecting the user's shifting attention, priorities, and emotional responses in real time. Conventional models have also had the propensity to overlook the personalization aspect, failing to learn about the distinctive user behavior of individuals to generate generalized predictions that are less personalized [2]. Given the need to provide personalized, context-sensitive feedback, the need for complex models that are able to look at dynamic interactions is of the highest priority. LLMs, with their ability to work with complex, multi-dimensional information, are a possible solution to those problems.

Recent advancements in deep learning and machine learning have introduced various frameworks to support predictive applications, including predicting affective states. For example, XGBoost [3] is commonly applied to the task of prediction but is not well-suited to deal with dynamic, high-dimensional data like real-time mobile alerts. In [4], the authors used a decision support system for making natural disaster predictions, but they ignored the fact that real-time emotional data is needed in post-disaster recovery. LeCun et al.'s [5] deep learning models have made complex learning possible with big datasets, but they are short of providing personalized and context-aware predictions within dynamic real-time settings. The Transformer [6] architecture is a significant improvement to natural language processing, while its potential in the domain of predicting affective state with dynamic mobile interactions is less understood, with no effort to provide personalization and real-time context. Recently, work that uses LLMs to support the prediction of the emotional state with smartphone sensors is mostly concerned with the sensors [7] while ignoring the rich real-time information that smartphone notifications provide regarding user emotions. This work is also an extension of the following work that includes some similar experiments on the real-time notifications data with different evaluation matrices, which were not incorporated before.

Recently, LLMs have largely been used for all sorts of predictive tasks because they can handle complex data. LLMs have been widely explored in various fields, including sentiment analysis, emotion detection, and affective state prediction. Brown et al. [8] highlighted GPT-3's effectiveness in generating human-like text. However, its ability to personalize responses and adapt to individual user behaviors remains limited, often leading to generalized predictions. Similarly, in [9], the authors applied GPT-2 to a range

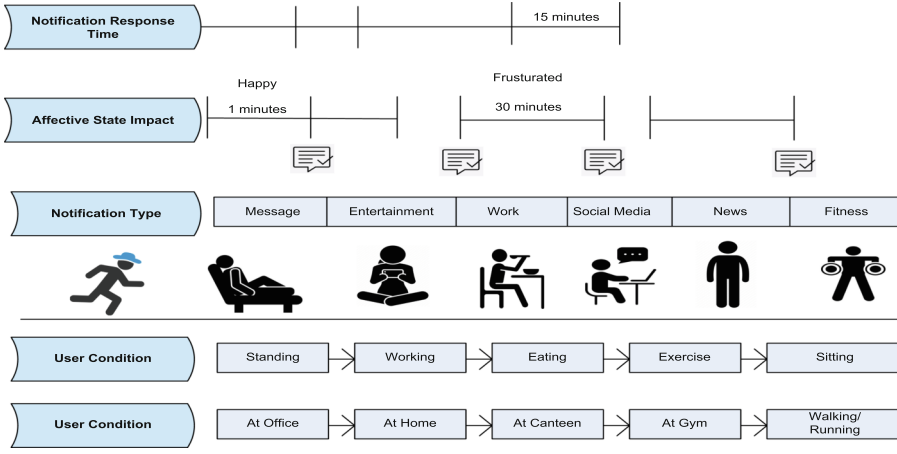


Fig. 1. Factors influencing user's smartphone interaction, including notification type, user condition, and response time.

of NLP tasks, but it struggled with handling dynamic, real-time data, such as ongoing responses to mobile notifications. Zhang et al. [10] looked at using LLMs for figuring out emotions, but they mainly worked with static data, which means the models didn't do as well in the real world, where things are always changing. Later, Li et al. [11] showed that transformer models could help predict mental health conditions, but because they only use past data and don't get real-time updates, they miss the context they need for accurate predictions. To overcome these limitations, our approach leverages dynamic user interaction data from mobile notifications, allowing us to make personalized and context-aware predictions of emotional states, advancing the field toward more accurate and adaptive models for affective state prediction. Unlike static datasets, dynamic notification interaction data is continuously generated as users interact with various types of notifications in real time, reflecting natural changes in user behavior, attention, and preferences. Each notification event, such as viewing, dismissing, replying, or ignoring, is captured and recorded dynamically on our server. This method ensures that the collected dataset evolves with the users' real-world interactions, providing a more authentic and temporally rich source of information compared to pre-collected or manually labeled static datasets. Traditional machine learning applications that address sensor inputs and pre-defined features lack the capabilities of the LLM which allows it to analyze complex user interaction patterns.

The LLMs are able to reveal hidden user behavior patterns by analyzing many smartphone notifications, which helps detect precise mental or emotional states for highly individualized affective analysis. This study explores the LLM's ability to determine affective states using human behavioral interaction data from smartphone notifications used as its key input source. In our work, we will examine the associations between the behavioral activities collected through smartphone notifications and their effect on user's. We will also show the eligibility of few-shots and zero-shots planted LLMs to conclude emotional cases based upon notification's gained from human properties. Proposed study results reveal a clear bond among smartphone notifications-extracted

activities of different users and their emotional levels, which LLMs illustrate with their reasoning processes. This work is considered as a totally new study to ground LLMs with mobile notifications features for affective state prediction.

Problem Statement: By referring to existing issues, the main problem being conveyed in that paper is defined as. *Understanding users' emotional states through passive smartphone interaction has become increasingly important for developing intelligent and context-aware applications. Traditional approaches rely heavily on structured surveys or intrusive monitoring, which limits scalability and real-time applicability. With the emergence of Large Language Models (LLMs), there is an opportunity to explore their ability to interpret subtle behavioral patterns in natural language data. However, it remains unclear whether LLMs can accurately infer emotional states from dynamic, real-world interactions such as smartphone notifications. This research addresses the problem of evaluating LLMs' capability to detect and reason about users' emotional states based solely on their interactions with notifications, using zero-shot and few-shot learning methods. The goal is to establish a minimally invasive, scalable, and intelligent system for affective computing using modern NLP techniques.*

In the later section, we will discuss the process of our dataset collection. After that, the data preprocessing will be discussed, which is the most crucial part of our research. Moving forward, the analysis process will be reviewed, which includes the explanation of zero-shot and few-shot methods that are being incorporated into our research work. Lastly, the logical interference and reason will be introduced, and we will see how our designed tasks help us understand more about how the LLM generates its predictions.

2 Materials and Methods

2.1 Dataset Collection

Notifications are considered the best source for gathering user's information. Different user's states have different impacts on smartphone usage. Similarly, the affective states can impact the user interaction with notifications. In Fig. 1, the notification arrival process is discussed, and it shows different factors that influence the user's emotional state, including notification type, user condition, and response time. In our previous work, while predicting future interactions with smartphones, we had reached the dead point where there was no exact ideal real-time notification data was present at the moment. So, we had collected the smartphone notifications dataset from the app developed named NotifyMiner [12] for the span of eight months from different users. There were two kinds of users that we have targeted, one was the university students, and the second was the employees of different companies. Notify Miner is a data collection app that uses a timestamp-based experience sampling method (TESP) to capture user reactions and thoughts about different notifications. Using Notify Miner, we gathered 14,480 notifications and 300 responses to the TESP questionnaire. The app relies on built-in Android features and APIs, such as the ESSensor Manager [13] and Google's Activity Recognition API, to gather contextual data, along with the Android Notification Listener Service.

The LLM models are trained to analyze and predict emotions or moods based on interaction trends with notification's, which include various features like User ID, Device

ID, App Name, Arrival Time, Seen Time, User Action, Title, GPS Location, Alert Type, Network Usage, and User Condition. The app collects data from multiple sources, such as notifications, apps, content, battery status, Bluetooth, Wi-Fi, mobile data, calls, messages, location, screen status, light, and a simple questionnaire form. The questionnaire is triggered when the user opens the app, changes location, or clicks on a notification. A notification is considered ‘clicked’ if the corresponding app is opened after the notification is interacted.

Post-hoc Integration of POMS Questionnaire: To effectively assess users’ emotional responses to notifications, a structured method for measuring mood was essential, and the POMS questionnaire provided the necessary framework [14]. Although the initial dataset of notifications was collected without the POMS assessment, it was incorporated into the analysis through a systematic post-hoc process. Specifically, we mapped the POMS responses, which assess mood states such as happy, irritated, Excited, and Frustrated, to the relevant emotional dimensions in our dataset. This integration allowed a more precise alignment between the POMS dimensions and user interactions, thereby enhancing our understanding of the emotional impact of notifications. The questionnaire is composed of 10 Likert scales. Five of the values reflect the positive impact (Happy, Content, Excited, Curious, Interested), and five of the values reflect the negative impact of notifications (Indifferent, Sad, Anxious, Frustrated, Irritated).

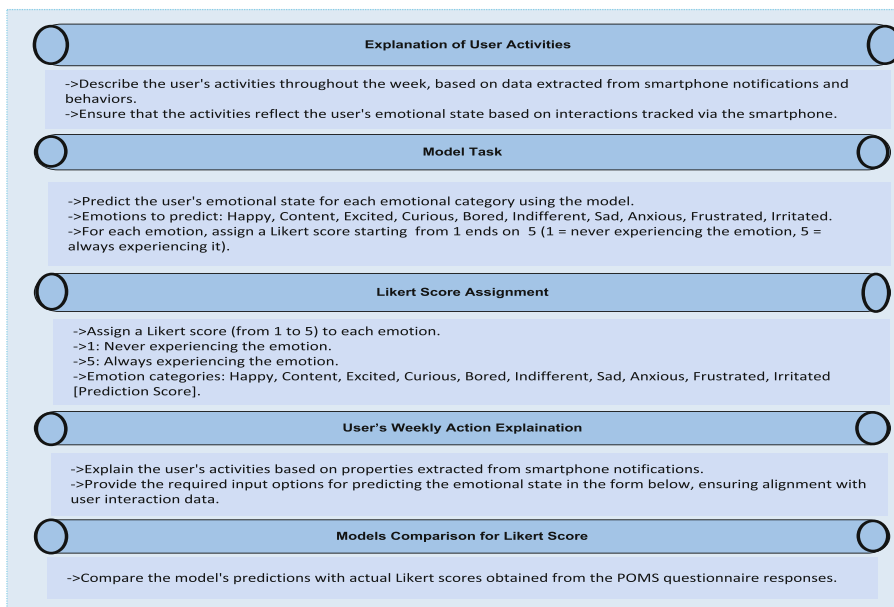


Fig. 2. Framework for Zero-Shot Emotional State Prediction

2.2 Data Preprocessing

The data collected through the Notify Miner app undergoes a series of preprocessing steps tailored to work with LLMs. The first step in preprocessing text data involves tokenization, which splits input data into smaller, manageable units so that LLM can analyze notifications and their contextual responses [15]. Extracting meaningful data from the unstructured information is a very crucial step in data processing. BERT (Bidirectional Encoder Representations from Transformers) [16] has been used to produce embeddings that enable the model to detect semantic ties within different user interactions. Later, stop-word removal filters [17] have been used to eliminate common words that produce unnecessary noise, which makes the model performance unaffected by non-informative words. Following preprocessing methods, make sure the LLM receives data that is both clean and relevant so it can produce precise emotional state predictions.

Providing descriptions set outlining a user's weekly activities extracted through smartphone notifications (e.g., messages, social media updates, app interactions), along with their corresponding emotional states, model job is to recognize patterns between the user's interaction behaviors and emotional responses. Based on these patterns, predict the user's emotional state for their upcoming interactions.

Example Format for Prediction:

Based on the following interactions:

- Happy Feeling: *score* - Determined Feeling: *score* - Attentive feeling: *score* - Their feeling of alertness: *score*
 - frustrated feeling: *score* - The feeling of anxious: *score* - Interested Feeling: *score* - Their feeling of irritation: *score*

Based on the above, select the appropriate Likert score (from 1 to 5) representing the user's emotional state throughout the weeks, based on their interaction data with notifications.

Task Details:- List of emotions: Happy, Content, Excited, Curious, Interested, Indifferent, Sad, Anxious, Frustrated, Irritated.- For each emotion, choose a score from 1 (Not once) to 5 (Everytime) representing the student's emotional state.

Upcoming Weeks Prediction:- Based on the patterns learned from previous data, predict how the student will feel during the upcoming week.- Provide your answers in the following format:

<Emotion>: [Predicted Number]

Fig. 3. Framework for the Few-Shots Emotional State Prediction

Given the processing capabilities of LLMs with natural language data, we created a system for the estimation of the present emotional state of the user from their actual interaction with the notifications from their smartphones. Each feature associated with the behavior of the user in terms of the latency of the responses, the level of engagement, as well as the type of notification, was transcribed as short text-based representations reflecting the present emotional state. Each interaction's timestamp (e.g., 2023-08-02 09:15:00 through 2023-08-02 09:16:00) was also added as a point of temporal context, enabling the LLM to understand the timing of the behavior of the user. Each notification was also classified (e.g., social media, reminders, alerts etc.) with associated emotional responses (e.g., "user came across as frustrated," "user engaged positively"). Concurrent with these, the interaction was associated with the responses from the POMS questionnaire responses to ensure the emotional states found through the LLM's processing corresponded with the more traditional mood dimensions of happy, curious, anxious, as well as irritated. This conversion enabled the system to estimate the emotional state

of the user from the present interaction patterns with the provision of context-based emotional estimates corresponding with the more traditional POMS dimensions.

2.3 Affective Analysis Process

To investigate if Large Language Models (LLMs) can detect the relationship between smartphone notification behavior and emotional states, few shots and zero shots tasks were conducted using the POMS questionnaire. Each week's data contained approximately 4,000 tokens from users' notification interactions. Due to resource constraints, we opted to use the OpenAI GPT-4 model [18] In our experiments, because it excels in natural language understanding and generating structured outputs from textual inputs, which is exactly what we need when mapping notifications to emotional states. The model's task was to generate Likert scores, not text descriptions, to predict emotional states from the notification data. To ensure consistent predictions, the model's temperature was set to zero, though we did find some minimal variation in results, especially with extreme emotional scores (e.g., low scores such as 1, and high scores such as 5).

This novel study has involved 12 user's, and the 20-week dataset was split into a 12-week training set and an 8-week test set. Stratified sampling was used to balance the distribution of the data across user's. For the few-shot influencing approach, we started with one training set instance and increased the number of examples incrementally (from one to ten shots) to test the ability of the LLM to predict emotional states based on notification data. The POMS questionnaire was essential to provide standardized measurements of mood, especially for mood dimensions like Happy, Interested, and Frustrated, thus enabling consistent and reliable emotional analysis throughout the study.

Zero Shot: As part of the baseline comparison for our study, we conducted a zero-shot task. The task was designed to assess how well the model could predict a user's emotional state based on their interaction with smartphone notifications [19]. Following Fig. 2, the task was created. Notifications were received by users through various means in the course of the week: friend messages, reminder alerts by learning applications, updates on social networking sites, and notifications on ongoing events. The notifications were diverse in content, volume, and priority, and thus produced different affective states. The interactions between the user and notifications were recorded, including Notification rate, Response, Notification type, Degree of engagement, and Relevance in context.

Few Shots: For every few-shot assignment, multiple weeks of smartphones notifications interaction data were selected from the training dataset, designated as labelled dataset. Few-shot tasks framework was created based on Fig. 3, which defines the process of predicting user emotional states based on their interactions with notifications. These frameworks used interaction features such as response time, engagement level, and notification type to guide the model in generating Likert scores for emotions.

Logical Interference and Reasoning: We created tasks to gain a deeper understanding of how LLM generates its predictions by assessing its abilities to make rational inferences based on user behavior. To achieve this, the prompt is adjusted via modifying the sentence "Provide your choices in the following form with no other reasoning" to "Provide your choices in the following form along with a rationale for each prediction". The following rationale depends upon comparing the user's weekly interaction behaviors with smartphone notifications: item: [predicted number and rationale]. " This adjustment

encouraged the model to explain its predictions, using the user's behavior data, such as response times, level of engagement, and notification types, ensuring that each emotional state prediction was not only based on the data but also logically justified.

2.4 Experimental Setup

In this study, the experimental setup was designed to investigate the relationship between user interaction with smartphone notifications and their emotional states using Large Language Models (LLMs). The process began with dynamic data collection from real-world smartphone notifications, followed by careful preprocessing to prepare the data for model input. Affective analysis was conducted through both zero-shot and few-shot learning strategies to evaluate how effectively LLMs could infer emotional patterns from user behavior. Logical inference tasks were also developed to assess the models' reasoning abilities. Finally, model performance was measured using established evaluation metrics, providing a comprehensive view of the effectiveness of our approach.

3 Results and Discussions

In the following section, the matrices that are employed to check the LLM's predictions are outlined. Later, we review the model's performance in zero-shot and few-shot scenarios. Finally, we present an analytical evaluation of the LLM's reasoning ability using a zero-shot case study, where the model's reasoning for emotional state predictions based on smartphone interaction data is analyzed.

3.1 Evaluation Metric

To measure our predictions accuracy, we implemented a macro-level evaluation strategy using both Root Mean Squared Error (RMSE) and relative error (ϵ). This process consisted of two main stages: initially, we computed the error metrics for each user's independently, and subsequently, we aggregated these metrics to calculate the average error across all users, providing a comprehensive view of the model's performance.

The RMSE for predicting emotional states based on smartphone notifications is calculated as follows:

For each user i , calculate the squared error for each observation k , where E_{ik} represents the predicted emotional state for the k^{th} notification interaction, and A_{ik} represents the true emotional state for the same notification interaction.

$$RMSE_i = \sqrt{\frac{1}{n_i} \sum_{k=1}^{n_i} (E_{ik} - A_{ik})^2}$$

where E_{ik} is the predicted emotional state of the k^{th} observation of the i^{th} user (predicted from notifications data), A_{ik} is the true emotional state for the k^{th} observation of the i^{th} user (based on actual emotions recorded, e.g., from POMS or another emotional rating system), n^i Number of notifications or interactions that the user i has during the week.

Calculate the Overall RMSE: After calculating the individual RMSE for each user, we then average these RMSEs across all user's to get the overall RMSE.

$$RMSE_{overall} = \frac{1}{N} \sum_{i=k}^N RMSE_i$$

where N is the total number of user's, $RMSE_i$ is the RMSE for user i calculated in the previous step.

Relative Error Calculation: After calculating the overall RMSE, we can compute the overall relative error ($\epsilon_{overall}$) is as follows:

$$\epsilon_{overall} = \frac{1}{N} \sum_{i=1}^N \left(\frac{RMSE_i}{\bar{A}_i} * 100 \right)$$

where $\bar{A}_i$ is the mean of the true emotional states of each user i , calculated as

$$\bar{A}_i = \frac{1}{n_i} \sum_{k=1}^{n_i} A_{ik}$$

The overall relative error measures the percentage error across all user's based on their emotional state predictions.

3.2 Performance Evaluation of Zero-Shot

In Table 1, the means and standard deviations were calculated for each of the twelve POMS items. Positive and negative emotional states were derived as the average RMSE's for the associated notification-related items. Highlighted results represent the best-executing shot for different affective states. It is explained that the zero-shot RMSE results are relatively underperforming in providing predictions, with an average RMSE of 1.25 out of 5 ($\epsilon = 35.2\%$). The RMSE values across all twelve user's show minimal variability ($std_all = 0.02$, $std_positive = 0.08$, $std_negative = 0.03$), reflecting steadily predictive performance of LLMs over different user's. The model exhibits similar performance for both favourable emotions (e.g., Happy, Content, Excited) with an average $RMSE = 1.20$ ($\epsilon = 30.5\%$) and negative emotions (e.g., Sad, Anxious, Frustrated) with $RMSE = 1.30$ ($\epsilon = 22.33\%$), but it demonstrates varying performance across individual emotional states. Based on smartphone notification interaction data, the model's performance remains quite stable. Specifically, the model excels with the "Happy" emotional state ($RMSE = 0.29$, $\epsilon = 51.1\%$) in zero-shot scenarios while showing its lowest performance with "Irritated" ($RMSE = 0.63$, $\epsilon = 58.8\%$) and "Frustrated" ($RMSE = 67.6$, $\epsilon = 51.3\%$). The following performance differences are most likely to be generated due to the changing amounts of prior knowledge embedded in the LLM or the unique behavioral indicators captured by smartphone notifications that correspond to each emotional state.

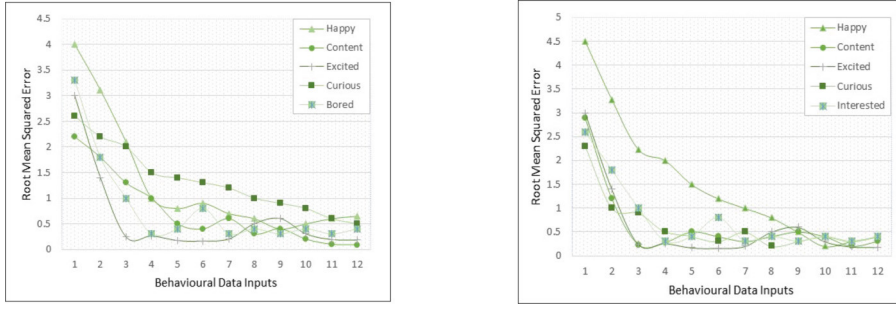


Fig. 4. Positive Affective States Learning Curves from two different user's.

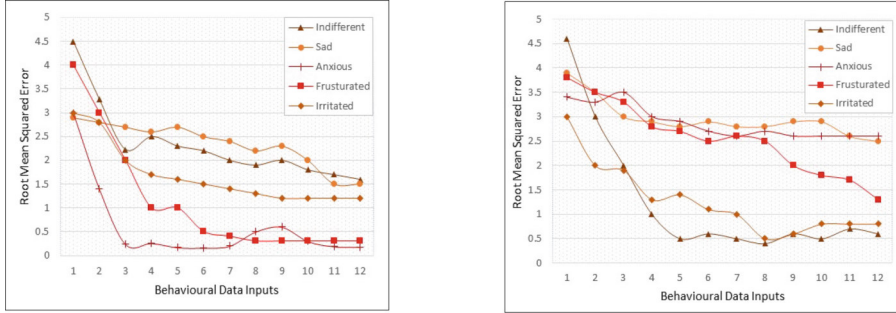


Fig. 5. Negative Affective States Learning Curves from two different user's.

Concluding our analysis demonstrates that the LLM shows acceptable performance at determining emotional states from smartphone notice data during zero-shot conditions but struggles with recognizing advanced emotional conditions such as “Irritated” and “Frustrated”. This is probably due to the model’s limited ability to interpret contextual nuances within notifications related to these more intense emotions. However, the model still provides valuable insights for general emotional state prediction, leveraging user interaction data for effective emotion recognition.

3.3 Few-Shot Performance Evaluation

To evaluate the effectiveness of LLMs in few-shot learning for predicting emotional states from smartphone notification data, we created few-shot tasks to assess the model’s capability to predict user emotions based on notification interactions. The results, displayed in Table 1, reveal that as more training examples were provided, the Root Mean Squared Error (RMSE) consistently decreased, showing a clear improvement in prediction accuracy. The one-shot learning showed satisfactory outcomes, yet adding the extra examples led to a performance decline, which is because of the increase in the number of examples. Particularly, RMSE value decreased by an average of 8.33% for positive emotions and 41.2% for negative emotions after one shot, with a decline as if

more examples were provided. When we averaged the RMSE for positive and negative emotional states, the model performed better for negative emotions in one-shot learning, with $RMSE_{positive} = 1.20$ and $RMSE_{negative} = 1.12$, while the performance between the two categories levelled out in ten-shot learning ($RMSE_{positive} = 1.05$, $RMSE_{negative} = 1.03$). This suggests that the model adapts itself quickly towards negative emotions in terms of smartphone notifications. if we break it down further, we have.

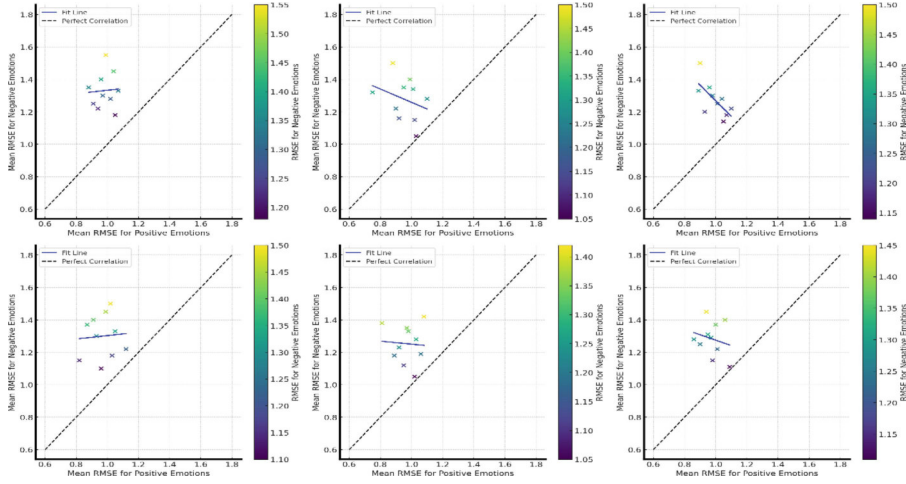


Fig. 6. Linear Correlations between positive and negative emotions for six user's.

Table 1. The Root Mean Square Error (RMSE), Mean, Std, Positive and Negative affects across twelve user's.

Shot	Happy	Content	Excited	Curious	Interested	Indifferent	Sad	Anxious	Frustrated	Irritated	Mean	Std	Positive affects	Negative affects
1.0	0.29	0.33	0.35	0.35	0.39	0.36	0.43	0.42	0.48	0.63	0.4	0.1	0.34	0.49
2.0	0.33	0.31	0.34	0.36	0.38	0.35	0.44	0.43	0.49	0.62	0.4	0.09	0.34	0.5
3.0	0.3	0.32	0.33	0.34	0.37	0.36	0.42	0.42	0.47	0.61	0.39	0.09	0.33	0.48
4.0	0.31	0.34	0.36	0.35	0.39	0.36	0.43	0.42	0.48	0.63	0.41	0.09	0.35	0.49
5.0	0.32	0.33	0.35	0.36	0.38	0.36	0.42	0.41	0.47	0.62	0.4	0.09	0.35	0.48
6.0	0.33	0.32	0.36	0.35	0.39	0.35	0.43	0.42	0.48	0.61	0.4	0.09	0.35	0.48
7.0	0.3	0.33	0.34	0.36	0.38	0.36	0.44	0.42	0.49	0.62	0.4	0.09	0.34	0.49
8.0	0.31	0.34	0.33	0.34	0.39	0.36	0.42	0.41	0.47	0.61	0.4	0.09	0.34	0.48
9.0	0.29	0.33	0.34	0.35	0.38	0.36	0.43	0.42	0.48	0.62	0.4	0.09	0.34	0.49
10.0	0.3	0.31	0.35	0.34	0.39	0.35	0.43	0.42	0.47	0.61	0.4	0.09	0.34	0.48
11.0	0.29	0.33	0.34	0.35	0.37	0.36	0.42	0.42	0.48	0.62	0.4	0.09	0.34	0.48
12.0	0.32	0.34	0.35	0.35	0.39	0.36	0.43	0.42	0.48	0.63	0.41	0.09	0.35	0.49

noticed that the LLM showed its best performance in predicting the emotional state “Excited” (RMSE = 1.15), while struggling the most with “Irritated” (RMSE = 0.63) and “Sad” (RMSE = 1.44). The model demonstrated its best performance through an 8.57% point decrease in RMSE when identifying “Anxious” emotions as it received ten-shot conditions. The accuracy of emotional state identification increased as the number of examples increased, leading to decreased standard deviation in RMSE from 0.27 in zero-shot learning to 0.12 in ten-shot learning. It is advisable, as the model receives more examples, that the variability in performance across individual emotional states decreases, likely due to an increase in the model’s understanding of the user’s emotional context. This improvement in prediction accuracy was observed for all user’s, although some emotional states were predicted more effectively for certain individuals. For instance, Fig. 4 illustrates the learning curves of two user’s, showing distinct variations in how the model adapted to the data in terms of positive affective states and Fig. 5 illustrates the learning curves of two user’s, showing distinct variations in how the model adapted to the data in terms of negative affective states. Additionally, Fig. 6 highlights the correlation between positive and negative emotions for six user’s, showing that the LLM’s predictive performance improves at similar rates for both emotional categories, suggesting a linear relationship in prediction accuracy. The blue fit line represents a clear relationship between positive and negative emotions in terms of RMSE.

Table 2. Zero-shot Predicted Score with Logical interference and Reasoning for One User.

Emotion	Predicted Score	Logical Interference and Reason
Happy (5)	4	The person engaged frequently with notifications, receiving high levels of interaction with positive content.
Content (4)	3	Notifications were moderate, with few distractions, suggesting the person felt content during the week.
Excited (5)	4	Frequent notifications, especially with social media updates, indicate a heightened sense of excitement.
Curious (4)	4	The person interacted regularly with notifications, often checking for updates and new content, indicating curiosity.
Interested (4)	3	Notifications indicated a mild interest in various apps and activities, but with average engagement overall.
Indifferent (2)	1	There were fewer interactions with less relevant notifications, suggesting indifference.
Sad (3)	2	A lack of frequent social interactions and neutral notifications suggest mild sadness but no strong emotional indicators.
Anxious (5)	4	The user had a higher volume of notifications but with occasional delays in responses, indicating anxiety.
Frustrated (4)	3	A high frequency of missed or ignored notifications suggests mild frustration with technology or notifications.
Irritated (2)	1	No significant fluctuations in notification engagement suggest no irritation during the week.

This makes the prediction easy to identify. In all the plots, the dashed line is used as a benchmark to predict the values against the ideal case. The clustering of data points closer to the blue fit line shows a greater correlation between the RMSE values for positive and negative emotional states. This tighter clustering means that the model’s

performance is consistent and more accurate across different emotional states. As more points move away from the fit line, it reveals the higher RMSE values, incorporating the fact that the model's prediction becomes less accurate.

It is revealed from the learning patterns that the zero-shot predictions tend to yield larger errors (located in the upper left corner of the graph); if more examples are provided, the data points shift toward regions of lower RMSE (lower left corner). It shows that when we provide more examples to LLM, it performs increasingly better for emotional state prediction. Overall, for our findings, our idea is supported that smartphone-inferred behavioral data holds noteworthy promise for predicting affective states with LLMs, reinforcing the strong relationships that are present between user behavior and emotional predictions.

3.4 Logical Interference and Reasoning

By equipping the LLM with an understanding of predicting emotional states based on smartphone notifications, the model offers a deeper look into the correlation between user activities with their emotional responses. Though these validation insights are still under review, they still provide valuable observations. A positive emotional state emerges when people actively use their smartphones and unlock their screens many times or use apps more than usual coupled together with long typing sessions. Rapid typing rates indicate a user's determination and ability to type quickly during short intervals, which contributes to their focus and attention on the screen. The system detects alertness through numerous screens and unlocks and recognizes missed calls together with quick typing adaptations; when users feel upset, the calls are brief, indicating hostility. Users who respond more slowly to messages or use their phones less tend to experience uninterruptedness, but the cessation of screen unlocking with fast location modifications indicates fear. The LLM uses different smartphone interactions, including typing behavior, screen unlocks instances, phone calls, and application usage, to determine user emotional states. Figure 3 graphs shows that the model generates meaningful and clear reasoning for its forecast process, which matches its predicted outcomes.

Table 3. Ten-shot Predicted Score with Logical interference and Reasoning for One User.

Emotion	Predicted Score	Logical Interference and Reason
Happy (5)	5	The person engaged frequently with social media, received a high volume of positive notifications, and had consistent responses, indicating high happiness.
Content (4)	4	A consistent engagement with notifications related to positive content, with average response times, indicating contentment and a balanced emotional state.
Excited (5)	5	The person interacted intensively with notifications, especially with social media, suggesting a high level of excitement.
Curious (4)	5	The person consistently checked notifications across different apps, including news and educational updates, suggesting strong curiosity.
Interested (4)	5	High engagement with notifications from multiple apps, particularly related to interests like hobbies and news, indicates high interest and engagement.
Indifferent (2)	1	Minimal interaction with notifications, particularly with less relevant content, indicating complete indifference.
Sad (3)	2	Fewer notifications related to social interactions and neutral content suggest mild sadness, but no major emotional impact.
Anxious (5)	5	The user frequently interacted with urgent notifications, delays in responses, and high engagement with important updates indicate significant anxiety.
Frustrated (4)	4	Moderate engagement with notifications but occasional delays, indicating frustration, though not overwhelming.
Irritated (2)	1	No noticeable patterns of delayed or ignored notifications suggest no irritation during the week.

The features, including battery consumption along with application usage, real-time location along with smartphone notifications interactions and typing activity, are emphasized repeatedly throughout the model's logical evaluation process. Those suggested LLM analyzes behavioral traces to generate emotional predictions that future researchers can enhance by including additional sensor details in their analysis. The LLM showed a higher degree of uncertainty during the task when identifying negative emotions, which was different from when recognizing positive emotions. The model chose scores at level 1 or 2 when identifying negative emotional states because this prediction category proved especially complex. The LLM frequently explained the assignment of one score by considering insufficient results instead of using the score as a default middle value. The results discovery process caused the score to evolve from the initial level 1 up to a range spanning from 2 through 3 and beyond. Testing a "Not able to decide (-1)" option did not yield results because the LLM showed confidence whenever it discovered enough evidence for its predictions. Tables 2 and 3 display RMSE improvement data, which occurs when the LLM receives additional examples. The model applied reasoning improvement to behavioral data from different weeks, which led to increased accuracy in predictions. The ability of LLM to learn new examples indicates how it can modify its predictive output.

The following study results proved that the smartphone utilization data aligns meaningfully with emotional state patterns, which demonstrates why such research results are important in emotional prediction analysis. The LLM points out that critical phone activities include reading generated content with excessive application usage coupled with numerous device unlocking sessions. Our analysis indicates that future predictions

will become more accurate when researchers also incorporate sleep patterns alongside particular app use times and the content of typing or viewing data into the analysis. Future studies should evaluate how these factors boost the accuracy of the prediction of emotional states.

4 Conclusion

In the proposed work, we have introduced the novel Language Models (LLMs) to detect user's emotional states through their smartphone notification activities. When the real-time user interaction data like engagement levels, latency, and different notification types are incorporated, then the proposed model has achieved personalized context-aware predictions for different emotional states like happiness, frustration, and irritation. Our results have indicated the valuable insights upon zero-shot models; they fell short in prediction accuracy if it is compared with few-shot learning approaches, which show better results when labelled data is incorporated.

Thus, our study suggests that LLMs are a great tool if used for personalized mental state tracking, especially in the field of smartphone notifications. By embracing Logical Inference and Reasoning, we can enhance prediction models in scenarios where data is sparse or ambiguous, making the predictions more adaptive to individual users' emotional dynamics. This approach would make the predictions more adaptive to the emotional dynamics of individual users.

In future work, if the LLMs are fine-tuned to comprise more real-time individualized user's data integrated with Logical Inference and Reasoning, it will set the way for a more automatic, emotionally intelligent notification system. Those systems would be able to dynamically adjust notifications based on user's real-time emotional states, which can improve user experiences and mental health. Our research study results imply that, with further refining, proposed emotion-aware systems can get control of traditional limitations in understanding user behaviors and offer a deeper understanding of user actions, making them indispensable for enhancing smartphone interactions and supporting emotional conditions.

Threats to Validity: Several potential threats to validity were considered in this study. To address internal validity, we controlled for external factors by using the POMS questionnaire, which provided standardized mood measurements to account for baseline emotional states and reduce the influence of unaccounted variables. Regarding external validity, although the sample size was limited to 12 users, we used stratified sampling to ensure a balanced distribution of data across users, aiming for a representative subset of behaviors. To mitigate concerns about construct validity, we selected the POMS questionnaire for its well-established reliability in measuring emotional states, acknowledging its limitations in capturing all context-specific emotions but recognizing its suitability for the scope of this study. Additionally, we ensured the real-time data collected from smartphone notifications was rich enough to provide meaningful insights. These efforts helped reduce the impact of potential validity threats, ensuring more reliable and contextually relevant findings.

Funding. The authors have not disclosed any funding.

Data Availability. The datasets generated and analyzed during the current study are not publicly available due to privacy Reasons and Ethical Concerns (Data includes personal information's, notifications and contacts) but are available from the corresponding author on reasonable request.

Conflict of Interest. On behalf of all authors, the corresponding author states that there is no conflict of interest.

References

1. Kanjo, E., Kuss, D.J., Ang, C.S.: NotiMind: utilizing responses to smart phone notifications as affective sensors. *IEEE Access* **5**, 22023–22035 (2017)
2. Wampfler, R., Klingler, S., Solenthaler, B., Schinazi, V.R., Gross, M., Holz, C.: Affective state prediction from smartphone touch and sensor data in the wild (2022)
3. Nalluri, M., Pentela, M., Eluri, N.R.: A scalable tree boosting system: XG boost. *Int. J. Res. Stud. Sci. Eng. Technol.* **7**(12), 36–51 (2020)
4. Mudassir, G., Di Marco, A.: Social-based city reconstruction planning in case of natural disasters: a reinforcement learning approach. *IEEE* (2021)
5. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* **521**(7553), 436–444 (2015)
6. Vaswani, A., et al.: Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30** (2017)
7. Zhang, T., Teng, S., Jia, H., D'Alfonso, S.: Leveraging LLMs to predict affective states via smartphone sensor features (2024)
8. Brown, T., et al.: Language models are few-shot learners. *Adv. Neural. Inf. Process. Syst.* **33**, 1877–1901 (2020)
9. Radford, A., Narasimhan, K., Salimans, T., Sutskever, I.: Improving language understanding by generative pre-training (2018)
10. Acheampong, F.A., Nunoo-Mensah, H., Chen, W.: Transformer models for text-based emotion detection: a review of BERT-based approaches. *Artif. Intell. Rev.* **54**(8), 5789–5829 (2021)
11. Kokane, V., Abhyankar, A., Shrirao, N., Khadkikar, P.: Predicting mental illness (depression) with the help of nlp transformers. *IEEE* (2024)
12. Khan, M.F., Lu, L., Toseef, M., Musyafa, A., Amin, A.: NotifyMiner: rule based user behavioral machine learning approach for context wise personalized notification services. *J. Ambient. Intell. Humaniz. Comput.* **14**(10), 13301–13317 (2023)
13. Fischer, J.E., Yee, N., Bellotti, V., Good, N., Benford, S., Greenhalgh, C.: Effects of content and time of delivery on receptivity to mobile interruptions (2010)
14. Selmi, O., Ouergui, I., Muscella, A., Levitt, D.E., Suzuki, K., Bouassida, A.: Monitoring mood state to improve performance in soccer players: a brief review. *Front. Psychol.* **14**, 1095238 (2023)
15. Ashish, V.: Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30**, I (2017)
16. Liu, Z., Lin, W., Shi, Y., Zhao, J.: A Robustly Optimized BERT Pre-training Approach with Post-Training. Springer, Heidelberg (2021)
17. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: Bert: pre-training of deep bidirectional transformers for language understanding (2019)
18. Sorin, V., et al.: Large language models and empathy: systematic review. *J. Med. Internet Res.* **26**, e52597 (2024)
19. Dorneles, S.O., Francisco, R., Barbosa, D.N.F., Barbosa, J.L.V.: Context awareness in recognition of affective states: a systematic mapping of the literature. *Int. J. Hum.-Comput. Interact.* **39**(8), 1563–1581 (2023)



Responsible AI Transforms Insurance Claims via Prompt Engineering

Shravya Kalva¹ and Atif Farid Mohammad²(✉)

¹ Cognizant, Bentonville, AR 72712, USA

² Purdue Global University, West Lafayette, IN 47906, USA
atif.mohammad@purdueglobal.edu

Abstract. The life insurance industry is poised to significantly from integrating Generative/Responsible AI, particularly through Prompt Engineering within Large Language Models (LLMs). This paper explores how advancements in Generative/Responsible AI can automate and enhance claim processing. Traditional claim processes, involving steps like document verification and beneficiary validation, require meticulous attention. Generative/Responsible AI can analyze complex documents, extract key information, and generate summaries for agents, expediting the process and ensuring accuracy. Prompt Engineering is essential for guiding AI models to produce accurate and ethically aligned outputs, adhering to industry standards. The research highlights potential benefits such as reduced processing times, improved customer satisfaction, and enhanced fraud detection, while addressing challenges and ethical considerations. By leveraging these technologies, the industry can achieve more efficient, accurate, and responsible claim processing, benefiting both providers and claimants. This paper provides a comprehensive overview of this transformative journey.

Keywords: Artificial Intelligence · LLMs · Prompt Engineering · Generative AI · Responsible AI

1 Introduction

The arena known as Life Insurance has a vast potential to embed Generative/Responsible AI by incorporating Prompt Engineering within a possible LLM. We will be exploring the automation of claim processing using the research, conducted within the realm of AI, in specific Generative/Responsible and Responsible AI. This paper will embed the traditional detail of processes involved in providing claims and research associated within the organization and the claimant. Generative/Responsible AI can analyze such a document to create summaries for the agents/providers before they create a claim.

The arena of life insurance is on the cusp of a transformative era, where the vast potential of Generative/Responsible AI, coupled with Prompt Engineering in a given LLM [1] suited for the required indexes creation, promises to revolutionize traditional processes. This paper delves into the intricate world of claim processing, exploring how advancements in AI, particularly Generative/Responsible and Responsible AI, can streamline

and enhance efficiency. By embedding traditional claim processes with cutting-edge AI technologies, we aim to bridge the gap between conventional methods and innovative solutions. Claim processing in life insurance involves a multitude of intricate steps, from document verification to beneficiary validation, each requiring meticulous attention to detail. Generative/Responsible AI, with its ability to create coherent and contextually relevant content, can significantly expedite these processes. For instance, Generative/Responsible AI can analyze complex insurance documents, extracting pertinent information and generating concise summaries for agents and providers. This not only saves time but also ensures that critical details are not overlooked, thereby enhancing the accuracy and reliability of claim processing.

Prompt Engineering plays a pivotal role in this transformation. By crafting precise and context-specific prompts, we can guide Generative/Responsible AI models to produce outputs that are not only accurate but also aligned with the ethical and regulatory standards of the insurance industry. This ensures that the AI-generated content is both useful and responsible, adhering to the principles of fairness, transparency, and accountability. This paper will explore the research conducted within the realm of AI, focusing on how Generative/Responsible and Responsible AI can be integrated into the existing frameworks of life insurance organizations. We will examine the potential benefits, such as reduced processing times, improved customer satisfaction, and enhanced fraud detection. We will also address the challenges and ethical considerations associated with implementing AI in claim processing, ensuring that the transition is smooth and beneficial for all stakeholders involved.

The integration of Generative/Responsible AI and Prompt Engineering in life insurance claim processing holds immense promise. By leveraging these technologies, we can create a more efficient, accurate, and responsible claim processing system, ultimately benefiting both the insurance providers and the claimants. This paper aims to provide a comprehensive overview of this transformative journey, highlighting the potential and the path forward.

2 Using Gen AI in Insurance

Gen AI can automate processes by creating claims management that would radically improve the operations in the insurance sector powered by a suited LLM [2].

Gen AI can be used at the following stages of the claim process (Fig. 1):

1. Intake stage: where claim is created via digital portals or thru phone or mail.
2. Adjudication stage: where the claim is in adjudication
3. Claim creation process: coverage eligibility, policy holder information
4. Claim processing stage: payment and post payment activity

2.1 Intake Stage

The intake stage in the insurance domain involves the initial creation of a claim, which can be submitted through various channels such as digital portals, phone calls, or mail [1]. AI can significantly enhance this stage by automating the data entry process and

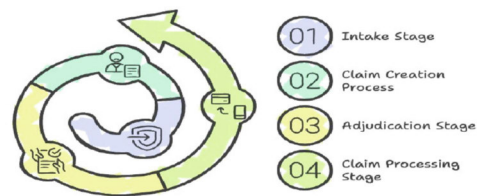


Fig. 1. Intake Stage: Where Claim is Created via Digital Portals or Through Phone or Mail

ensuring accuracy. For instance, natural language processing (NLP) can be used to extract relevant information from emails or transcribed phone calls, automatically populating claim forms. This not only speeds up the intake process but also reduces the likelihood of human error, ensuring that all necessary details are captured correctly from the outset.

AI-powered chatbots can provide immediate assistance to policyholders, guiding them through the claim submission process and answering any queries they may have. These chatbots can operate 24/7, offering convenience and accessibility to customers regardless of the time or day. By leveraging machine learning algorithms, the chatbots can continuously improve their responses based on previous interactions, providing a more personalized and efficient service over time.

2.2 Adjudication Stage: Where the Claim is in Adjudication

The adjudication stage is critical as it involves the evaluation and decision-making process for insurance claims. AI can play a crucial role in streamlining this stage by analyzing vast amounts of data to determine the validity and extent of a claim [2]. Machine learning models can be trained to identify patterns and anomalies in claim data, helping to detect fraudulent activities more accurately. This not only speeds up the adjudication process but also ensures that legitimate claims are processed efficiently while fraudulent ones are flagged for further investigation.

AI can assist in predicting the likelihood of a claim being approved based on historical data and current trends. Predictive analytics can provide insights into potential outcomes, allowing adjudicators to make more informed decisions. This can lead to faster resolution times and improved customer satisfaction, as policyholders receive timely and fair assessments of their claims.

2.3 Claim Creation Process: Coverage Eligibility, Policyholder Information

The claim creation process involves verifying coverage eligibility and gathering policyholder information, which are essential steps in determining the validity of a claim. AI can automate the verification of coverage eligibility by cross-referencing claim data with policy details stored in databases. This ensures that only eligible claims are processed, reducing the burden on human adjudicators and minimizing the risk of errors. AI can enhance the accuracy and completeness of policyholder information by using data enrichment techniques [3]. For example, AI algorithms can integrate data from various sources, such as public records and social media, to update and verify policyholder

information. This ensures that the most current and accurate data is used in the claim creation process, leading to more reliable and efficient claim handling.

2.4 Claim Processing Stage: Payment and Post-Payment Activity

The claim processing stage encompasses the payment of approved claims and post-payment activities, such as follow-up and auditing. AI can automate the payment process by integrating with financial systems to initiate and track payments. This ensures that payments are made promptly and accurately, reducing the need for manual intervention and minimizing the risk of payment errors.

In addition, AI can assist in post-payment activities by monitoring and analyzing payment data to detect any irregularities or potential issues [4]. Machine learning algorithms can identify patterns that may indicate fraud or errors, triggering automated alerts for further investigation [5]. This proactive approach helps to safeguard against financial losses and ensures the integrity of the claim processing system. By leveraging AI, insurance companies can enhance the efficiency and accuracy of their claim processing, ultimately leading to better outcomes for both the insurer and the policyholder. Use of Generative AI improves the following:

- A. Profitability and growth – by cutting down human intervention and processing claims quickly
- B. Cost savings and efficiency – By investing in Generative/Responsible AI-driven solutions insurers can reduce spending on cost savings and operational efficiency gains to process claims.
- C. Operational intelligence and effectiveness –Using Generative/Responsible AI for autonomous coding is accelerating the software development life cycle resulting in productivity gains and reduction in training time, which may enhance workforce productivity.

The integration of Generative/Responsible AI (Gen AI) in the insurance industry offers several transformative benefits, significantly enhancing profitability, operational efficiency, and overall effectiveness. One of the most notable advantages is the improvement in profitability and growth. By leveraging Gen AI [6], insurers can significantly reduce human intervention in various processes, particularly in claims processing. This reduction not only speeds up the claims process but also ensures greater accuracy and consistency, leading to higher customer satisfaction and operational efficiency. The ability of Gen AI to handle complex tasks autonomously [7] allows insurance companies to process claims more quickly, thereby enhancing customer service and reducing the time and costs associated with manual interventions (Fig. 2).

Gen AI contributes to substantial cost savings and efficiency. Investing in Generative/Responsible AI-driven solutions enables insurers to streamline their operations, reducing expenditure on manual processes and enhancing overall efficiency. By automating routine tasks and leveraging AI for more complex decision-making, insurance companies can achieve significant cost savings. This efficiency gain allows insurers to allocate resources more effectively, focusing on strategic initiatives that drive growth and innovation. The implementation of Gen AI also helps in identifying and mitigating fraudulent claims, further reducing operational costs and enhancing the bottom line. Operational

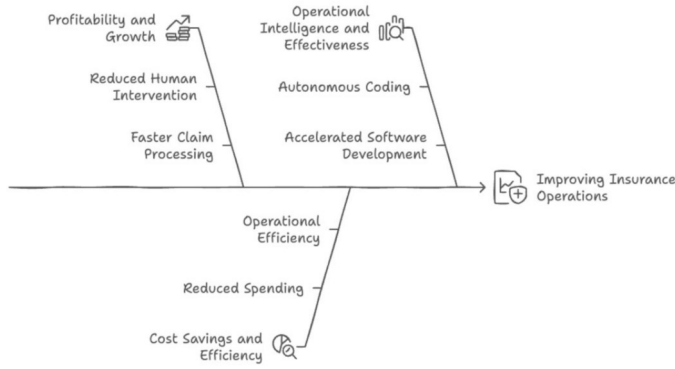


Fig. 2. Enhancing Insurance Operations with GenAI

intelligence and effectiveness are also significantly enhanced through the use of Gen AI. In the software development life cycle, Gen AI can accelerate processes by enabling autonomous coding and other automated tasks.

This acceleration leads to productivity gains and a reduction in training time for developers, ultimately enhancing workforce productivity. By integrating Gen AI into their operational frameworks, insurance companies can achieve faster time-to-market for new products and services, staying ahead of the competition and meeting evolving customer demands more effectively. Gen AI's ability to generate novel and creative outputs [9], accessible even to those with little technical knowledge, further amplifies its impact on operational intelligence. This capability allows insurance companies to innovate and adapt quickly to market changes, ensuring that they remain competitive and responsive to customer needs. The transformative power of Gen AI in the insurance industry is undeniable, making it a critical component for future growth and success.

3 Research Proposition: Algorithm

Here are the necessary elements of the proposed algorithm. The following insertion of States, Claims and Transitions are associated with the mathematical algorithm we are proposing in this paper (Table 1, 2 and 3).

Table 1. States and Claims

S_0 : Initial state (no claim)	C_{01} : Claim is submitted
S_1 : Intake stage	C_{12} : Claim is validated for adjudication
S_2 : Adjudication stage	C_{23} : Coverage eligibility and policyholder information are verified
S_3 : Claim creation process	C_{34} : Claim is approved for payment
S_4 : Claim processing stage	C_{45} : Payment and post-payment activities are completed
S_5 : Final state (claim completed)	

Table 2. Transitions

1. Initialization:
 - Set initial state $S = S_0$
2. Intake Stage:
 - If C_{01} is true:
 - $S = S_1$
 - Record claim details (digital, phone, mail)
3. Adjudication Stage:
 - If $S = S_1$ and C_{12} is true:
 - $S = S_2$
 - Initiate adjudication process
4. Claim Creation Process:
 - If $S = S_2$ and C_{23} is true:
 - $S = S_3$
 - Verify coverage eligibility and policyholder information
5. Claim Processing Stage:
 - If $S = S_3$ and C_{34} is true:
 - $S = S_4$
 - Initiate payment and post-payment activities
6. Final State:
 - If $S = S_4$ and C_{45} is true:
 - $S = S_5$
 - Mark claim as completed

T_{01} : Transition from initial state to intake stage
 T_{12} : Transition from intake stage to adjudication stage
 T_{23} : Transition from adjudication stage to claim creation process
 T_{34} : Transition from claim creation process to claim processing stage
 T_{45} : Transition from claim processing stage to final state

Table 3. Matrix with Conditions

Matrix	Conditions
$ \begin{pmatrix} 0 & C_{01} & 0 & 0 & 0 \\ 0 & 0 & C_{12} & 0 & 0 \\ 0 & 0 & 0 & C_{23} & 0 \\ 0 & 0 & 0 & 0 & C_{34} \\ 0 & 0 & 0 & 0 & C_{45} \end{pmatrix} $	$C_{01} = f(\text{claim_submission})$ $C_{12} = f(\text{claim_validation})$ $C_{23} = f(\text{coverage_eligibility}, \text{policyholder_info})$ $C_{34} = f(\text{claim_approval})$ $C_{45} = f(\text{payment_completion}, \text{post_payment_activities})$

4 Conclusion

The integration of Generative/Responsible AI (Gen AI) in the insurance industry represents a transformative shift, offering substantial benefits across various stages of the claims process. In the adjudication stage, AI streamlines decision-making by analyzing data to determine claim validity and detect fraud, ensuring efficient and accurate claim processing. During the claim creation process, AI automates coverage eligibility verification and enhances policyholder information accuracy through data enrichment techniques. In the claim processing stage, AI automates payments and monitors post-payment activities to detect irregularities, safeguarding against financial losses. The adoption of Gen AI improves profitability and growth by reducing human intervention and accelerating claims processing. It also drives cost savings and operational efficiency, allowing insurers to invest in more effective solutions. Furthermore, Gen AI enhances operational intelligence and effectiveness by accelerating software development and reducing training time, leading to productivity gains. By leveraging these capabilities, insurance companies can achieve better outcomes, ensuring higher customer satisfaction and operational excellence.

References

1. Eckert, C., Osterrieder, K.: How digitalization affects insurance companies: overview and use cases of digital technologies. *Zeitschrift für die gesamte Versicherungswissenschaft* **109**(5), 333–360 (2020)
2. Van Anh, N., Duc, T.M.: Big data-driven predictive modeling for pricing, claims processing and fraud reduction in the insurance industry globally. *Int. J. Respon. Artif. Intell.* **14**(2), 12–23 (2024)
3. Nimmagadda, V.S.P.: Artificial intelligence for customer behavior analysis in insurance: advanced models, techniques, and real-world applications. *J. AI Healthc. Med.* **2**(1), 227–263 (2022)
4. Arpteg, A., Brinne, B., Crnkovic-Friis, L., Bosch, J.: Software engineering challenges of deep learning. In: 2018 44th Euromicro Conference on Software Engineering and Advanced applications (SEAA), pp. 50–59. IEEE (2018)
5. Mohammad, A.F., Agarwal, R., Columbo, T., Vigorito, J.: Generative/Responsible & responsible AI-llms use in differential governance. In: 2023 International Conference on Computational Science and Computational Intelligence (CSCI), pp. 291–295 (2023)
6. Mohammad, A.F., Clark, B., Agarwal, R., Summers, S.: LLM GPT generative/responsible ai and artificial general intelligence (AGI): the next frontier. In: 2023 Congress in Computer Science, Computer Engineering, and Applied Computing (CSCE), pp. 413–417 (2023)
7. Mohammad, A.F., Clark, B., Hegde, R.: Large language model (LLM) & GPT, a monolithic study in generative/responsible AI. In: Proceedings of the 2023 Congress in Computer Science, Computer Engineering, and Applied Computing, CSCE 2023, Las Vegas, NV, USA, 24–27 July 2023, pp. 383–388 (2023)
8. Mohammad, A.F., Clark, B., Agarwal, R., Summers, S.: LLM/GPT generative/responsible AI and artificial general intelligence (AGI): the next frontier. In: 2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE), Las Vegas, NV, USA, pp. 413–417 (2023)
9. Koivisto, M., Grassini, S.: Best humans still outperform artificial intelligence in a creative divergent thinking task. *Sci. Rep.* **13**(1), 13601 (2023)



Responsible AI Incorporation in Advanced Persistent Threat Landscape

Sarthak Bhatt¹, Atif Farid Mohammad²(✉), and Urmita Banerjee³

¹ Healable Inc., Rockville, MD, USA
sarthak.bhatt@healable.com

² Purdue Global University, West Lafayette, IN, USA
atif.mohammad@purdueglobal.edu

³ University of Maryland, Baltimore, MD, USA
uban1@umbc.edu

Abstract. The multifaceted nature and the quantity of digital assaults has been expanding constantly which induces that the security frameworks that are being used today have restricted accomplishment in keeping these advanced assaults. By incorporating Responsible AI, we can predict the possible threat patterns to get guardrails applied. Advanced Persistent Threats like Stuxnet, hacking group RCS, Red October, Wild Neutron and all the more as of late Carbanak have further lessened the trust in present security frameworks. In this paper, we have performed the specialized investigation of these Advanced Persistent Threats which highlights their individual salient features and identify common patterns among them. This analysis will assist us to think of a skeletal structure for all the APT's which in turn assists in concocting a cautious defensive mechanism to identify similar threats in future. This paper sheds light on methodologies as well as research covered on the mentioned in depth.

Keywords: APT · Cyber Attacks · Digital · Responsible AI

1 Introduction

Advanced Persistent Threat (APT) is a term began over the recent years for another type of guileful dangers that utilize different assault procedures and vectors and that are led by stealth to stay away from identification so hackers can hold control over target frameworks unnoticed for drawn out stretches of time. Cyber-attacks are expanding in intricacy, persistence and numbers. It speaks to another era of system dangers of high unpredictability. APT also targets a specific system or entity with a very precise goal. In addition, APT executes supported and viable attack for a long stretch. The aim of an APT assault is to take information that it can and then to harm the system or association. APT attack target associations in sectors with high esteem data, for example, National Guard and the money related industry, and governments. Because of the intricacy and stealthiness of APT's, there is no single arrangement that offers powerful insurance. The attention ought to be on building a protection top to bottom methodology that means to

continually screen systems and security controls for their viability. In order to build a system, we need to be quite mindful of APT's features along with their technicalities. In this paper we have laid our emphasis to study the features of different APT's that can be blended with the new technical methodologies to prevent the APT attacks. For this we have considered some potential parameters which helps in identifying the APT's and take some defensive measures as such.

In this paper, we will focus on the problem that has given us the motivation to conduct this survey. Along with that we would like to give some insights into the preliminary findings of our survey. We also discuss the related work if any that has been done in the field of APT attacks. Next, we will present the methodology that we have implemented in successfully conducting this survey in a block diagram. We further describe the actual survey that we have conducted which includes the attacks considered & importance of parameters considered and classification of attacks accordingly. This paper also deals with the work that is related to after classification of APT attacks. We also share the results of our survey along with the insights that we have gained through this survey. We propose the defensive mechanism to prevent or identify the APT attacks. This defense mechanism is modelled based on the survey results found.

2 Motivation

The APT attacks have been increasing enormously with the increasing use of computers. Below graph (Fig. 1) highlight the growth of APT's from 2008 to 2016. With the increase in the use of computers even more, there is every chance of increase in the APT attacks as well. APT attacks are very harmful as their main motto is to get access of confidential data of an organization. Once the data is accessed it is easy to get hold of the operations and get benefitted from that. Carbanak is one of the APT attacks in which the attackers targeted a bank, compromised the security and then started mimicking like the organization employees using their credentials. Once they got hold of the operations, they were able to steal 1 billion from the bank which is a very huge amount. There is every chance that the attackers target the most valuable data related to any fields like health care, Army, etc.

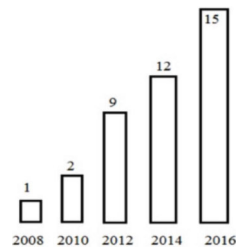


Fig. 1. APT Stacked Chart

Identifying these APT attacks is very crucial for the organizations to survive. In order to identify these APT attacks and classify them we have considered the parameters like

Targeted operating system and architecture, Initial attack vectors, Ease of spread, Attack purpose, Exploitation of digital signatures/ confidentiality and Network Evaluation.

3 Related Work

A lot of work is being done on APT attacks as it has been the major cyber threat in the coming future for the organizations of different industries. Most of the work related to APT's has concentrated on how the attack happens in general. In [1], the author has mentioned in detail about the salient features of the APT. The author has mostly concentrated on giving in depth knowledge of APT's in which he answered questions like what is the methodology of APT attacks and the tools or elements that are targeted by the APT attackers. He also mentioned the precautionary measures for the APT's but the author did not talk about how to identify the cyber-attacks as an APT attack taking all these factors mentioned into consideration. In [2] as well, the author concentrated on giving the details about the APT attacks, their life cycle and some precautionary measures. Some of the other papers have talked about a particular APT attack and the tools that can be helpful in preventing that particular attack. There are several other papers as well which talked about the significance of APT attacks in general or in particular of an APT attack. So we found the need to address the APT's individually i.e. different APT attacks and possibly find any relation among them. A skeleton can be formed out of the commonalities we found and accordingly develop a defensive mechanism which helps in identifying any APT attack and prevent them rather than curing them.

3.1 State of the Art

In order to come up with a skeletal structure capturing the potential differences among the APT attacks we have considered some of the APT attacks for our survey.

3.2 APT Attacks

- *Stuxnet*: A worm which discovered in the year 2010 which attacks is targeted towards Reprogram Industrial Control Systems (ICS), On Programmable Logic Controllers (PLC) which are mostly related to the nuclear industry. Stuxnet is one of the early APT attacks known. Vast array of components are used for the attack like Zeroday exploits, Windows rootkit, PLC rootkit (first ever), Antivirus evasion, Peer-to-Peer updates, Signed driver with a valid certificate, Command and control interface. In 2007, Siemens industrial company has been attacked compromising Siemens PCS 7, WinCC and STEP7 industrial software applications along with one or more Siemens S7 PLCs. The robust systems behind Stuxnet made it auto-executable, which required no user interaction, thus resulting in a zero-day vulnerability. Some of the vulnerabilities that were observed were MS 10-046: vulnerability which used modified autorun.inf technique MS08-067: Exploiting any network accessible windows systems using the windows server service. MS10-061: Vulnerabilities using privilege escalation using MS10-073 and MS10-092.
 - a. Target organization is Siemens industries.

- b. Select a Siemens system for attack launch (OS).
 - c. Identify the network propagation (Network).
 - d. Launch the attack via a USB stick which infects all machines (Attack vector).
 - e. Compromise the target system's logic controllers exploiting vulnerabilities (Signature Exploitation).
 - f. Download a more recent version and update the attack programming logic (Ease of spread).
 - g. Control the systems by spying and gather information (Attack purpose).
- **Hacking team RCS:** RCS is a spyware program discovered in 2011 which is targeted towards individuals like politicians, journalists and activists. The attack initially uses social engineering and then updates itself using the USB drives and cables. Rather than stealing data it surveillances the attacked systems. In 2012, all antivirus companies received malicious code in an email which they were able to identify from the keyword 'RCS' in that code.
 - a. Target individual will be selected.
 - b. Personal computer of the target individual is the target system (OS).
 - c. Launch the attack via CD-ROM or USB (Attack vector).
 - d. Compromise the internal security of the personal computer by exploiting the resources (Signature Exploitation).
 - e. Self-update based on the information in the personal computer (Ease of spread).
 - f. Monitor and log and actions performed in the personal computer (Attack purpose).
 - **Carbanak:** An APT attack which was discovered in the year 2014 and still active. It mainly targets the financial organizations to steal the money. It has been the largest cyber heist ever in terms of stealing money. It uses the backdoor and exploits the target organization systems with the help of spam emails to the employees. Once the employee clicks on the spam email the attack will be launched and spreads over all the systems in that organization compromising the security.
 - a. Target financial organization (mostly banks) is selected.
 - b. Get the domain name of organization and send email to an employee (Attack vector).
 - c. If the system satisfies the OS version, the attack will be launched (OS).
 - d. Infects all the systems in the organization through the intranet used by the organization (Ease of spread).
 - e. Compromise the security by monitoring the employee system and change credentials (Signature Exploitation).
 - f. Once the attackers compromise the admin system, they steal the money (Attack purpose).
 - **Wild Neutron:** It is a cyber espionage that was discovered in 2013 which is mainly targeted towards IT, healthcare, real estate, investment companies and law firms. It uses a backdoor module which initiates communication with C&C servers and then followed by exploitation tools, SSH-based exfiltration tools. Compromised the site 'www.iphonedevs.dk [.com]' and redirected the users to other websites from where they hosted a zero-day exploit.

- a. A multinational organization will be selected.
 - b. The website that belongs to the targeted organization will be compromised (Attack vector).
 - c. The users will be directed to the vulnerable sites from where they host a zero day exploit.
 - d. The attack will be launched in the system (OS).
 - e. Several exploitation tools are deployed which helps in spreading the attack (Ease of spread).
 - f. The information is passed onto the attackers (Attack purpose).
- **Red October:** A complex cyberattack platform discovered in the year 2013 which is targeted towards the reputed academia/research organizations, Energy, oil and gas companies and military as well. Malicious code was sent through emails using the Microsoft office, PDF, and java vulnerabilities. In 2012, the target companies have received an email with the updates for Microsoft office component, which compromises the security once they got clicked. The Domains were pointed to IP addresses that ended up just being proxies. As a strong persistent component, Red October introduced a module by inserting a malware visual basic code in the script written for Office and Adobe reader applications. This parsed each opened Office or PDF document and attempted to distinguish inserted commands (infused by the attackers) to execute. Utilizing this procedure, regardless of the fact that the C&C servers were brought down, the attackers could email exceptionally created files to their victims and can easily be able to reconnect with their systems. The known vulnerabilities such as CVE-2009-3129, CVE-2010-3333 and CVE-2012-0158 that were found in the exploited documents was the first step to infect the targets. In some cases, a JAVA exploit CVE-2011-3544 was also used to infect the systems with malware payloads.
 - a. Target organization will be identified.
 - b. A spear phishing mail is sent to the target organization (Attack vector).
 - c. When the user opens the spear phishing email the attack will be launched using the known exploits of MS office (Ease of spread).
 - d. The attacker sends the malicious code to work on windows knowing that most of the organizations use windows operating system (OS).
 - e. Once the attack is launched the malicious code operates and sends the confidential data to the attackers (Attack Purpose).

3.3 Attack Classification

Observing the attack launch processes of different APT attacks that we have considered, we found a skeletal structure that captures the commonalities in different APT's. The structure can be defined as a list of parameters which will be helpful for us in coming up with a defensive mechanism (Table 1).

Table 1. Classification of Attacks

	Attack vector	network	Ease of spread	OS	purpose	Signature exploitation
Stuxnet	PLC's, USB drives, LAN	LAN	Programming code and logics	Windows(only)	Design Theft	yes
Wild Neutron	Watering hole	NA	Command prompt	Windows, OS X	Data theft	no
Red October	Social engineering	HTTP Proxies	Microsoft office files, malware infected emails.	Windows, Android	Data theft	no
Carbanak	Social engineering	TCP	intranet	Windows	Money theft	yes
Hacking team RCS	USB drives & cables, Direct hard disk, Bootable CD-ROM	NA	Self-Updates	Windows, Mac, BlackBerry, Android, IOS	Surveillance	yes

3.4 Parameters

- *Targeted operating system and architecture:* Operating system and its architecture are crucial enough during the initial stage of attack. The attackers may not be able to get through it if the configuration is different from the one that can be compromised. (E.g. the malware might work only for 32-bit version of windows).
- *Initial attack vectors:* It is very crucial since this is the initial mode of propagation that helps the attackers to launch the attack. Identifying the different types of propagation used helps in detecting the APT's at the initial stage itself.
- *Attack purpose:* The number APT attacks are constantly increasing over the last few years. These attacks are launched for achieving different purposes. Based on the purpose of attack the approach of the attack might differ from one another.
- *Ease of spread:* APT's are known for their persistence. They remain undetected for years and keep on spreading until they were able to access the confidential data. In the process of accessing data different APT attacks will have to go through several backdoors which are opened once the security is breached.
- *Exploitation of digital signatures/ confidentiality:* APT's are digitally signed using compromised digital certificates. Thus, these samples would manage to infect hardened systems. Based on this, allowing execution of binaries based on the existence of valid digital signatures cannot be considered an effective defense on its own. This is particularly important for Anti-Virus, which tend to avoid real-time analysis of signed binaries for performance reasons.
- *Network Evaluation:* The egress traffic which is destined to the commonly used ports is frequently allowed to pass through network access control mechanisms. The malwares use these common ports for establishing a connection and communicates back n forth - a worrying finding, taking into account the sensitive nature of the targeted organizations.

3.5 Research

It can be found from the above classification that in maximum cases most of the systems with Windows as the operating system and their version being 32 bits. Has compromised with their privacies and were found more vulnerable to Advanced persistent threats. In a related study presented by “Nikos, Virvilis” in his paper “Trusted computing vs APT” it can be concluded that none of the malware in all of the attacks were able to infect the 64 bits version of windows. This was because the extra security feature that is present in the 64-bit version of windows which makes the exploitation highly difficult for the attackers, especially preventing the kernel components of the system from getting accessed by malware targets. According to the study, it can be said that the main reason of exploitation of 32 bit systems was, that the maximum number of the victims were using that architecture. But does this make Apple ios more secured from APT's? The answer is completely “NO”.

Another study proposed by “Mathew J Schwartz” reveals that “ios” is relatively easier to exploit as there are “Pervasive authentication issues” in OS X. and that it makes OS easy to exploit. Two of the mainly used protocols that are used in managing the functionalities of “IOS”, which consists of AFP (Apple Filling Protocol) and Bonjour, a service delivery protocol that is used in establishing a connection between Macs and servers. Particularly, in comparison with windows it's easy to replace a malicious server for a fake one and then forcing users to connect to it. These vulnerabilities provide a base for infecting Macs by APT's. The above two related work in the field of APT's contributes in concluding that because Apple's market share is only about 6%, which means that if attacker want to compromise many computers at once, they will typically go for “Windows”.

Another important point of entrance for APT's found was malware infected Word and Excel documents. Analyzing the different versions of Microsoft office suite, it can be observed that from the release of MS Office 2010, a new feature of “Protected View” was added, which ensures the opening of files from untrusted source in a sandbox view which enabled the prevention of exploitation of system through these files. This default opening of any new file in protected mode, played a role of additional barrier which made malware unable to pass through it. By digging into the analysis of the Victim's system which were affected by such files, it can be concluded that either they were running outdated versions of MS Office or they had disabled the security feature of the software.

3.6 Results

The attack vectors that have been used for different APT attacks are the basic ways we use to transfer files over the internet or in person. It is our general tendency to not scan for any viruses if we use USB's or CD-ROM's when we do file sharing and when it comes to email, we just go through the body of mail and do not pay any attention to the subject or from whom we have got the mail. The attackers take advantage of the user's negligence in exploiting and attacking the systems. Analyzing the network parameter tells us that most of the victims have a very relaxed internet access. The attackers identify the ports through which much of the traffic goes and launch the attack through those ports. As the network user is relaxed it paves way for the attackers to compromise the security

and communicate back to the C&C infrastructure. During the initial stage, the APT attacks are targeted towards stealing the confidential data from different organizations. But in later stages, APT's are not only confined to steal data they have targeted several financial organizations in order to steal money and also to steal confidential designs of new ventures or on which the current system works.

For the APT to spread through the other systems it needs a medium. The medium turns out to be the vulnerable applications in the target systems. The attackers launch zero-day exploits to cash on these vulnerabilities and the APT's spread using these vulnerable applications. If we observe these applications, they are not the core applications of an operating system but the installed components for the user purpose. As the user uses them extensively for his purposes it feeds the APT to spread through the other systems. The attackers also look after the operating system which they target as the malicious code is intended to work on it. From the attack classification table, we might come to the conclusion of windows as the most vulnerable operating system but as mentioned in the Sect. 3.4, the OS X market share has been considerable very less compared to windows. As windows is the most used operating system, the attackers develop malicious code to run on windows so that they attack a wide range of systems at once. Signature exploitation of significant applications and accessing the target systems using these signatures to compromise the security has been a major thing in many APT attacks. This tells us that the applications that user installs should also be taken care of. The users have the tendency to just install the application to satisfy their need without thinking much about its security concerns. This gives the attacker the chance to exploit and launch the attack.

3.7 Proposal of Defense Mechanism

Defensive Measure 1: Multi layered networks with robust mechanisms should be used to prevent the APT attacks or at least detect the anomalies. The employees or individuals should be knowledgeable enough about how the APT attacks through social engineering so that they do not fall prey to spam emails and websites. There should be a security layer within each system so that the APT cannot access the resources in the personal computer of an individual or organization (Fig. 2).

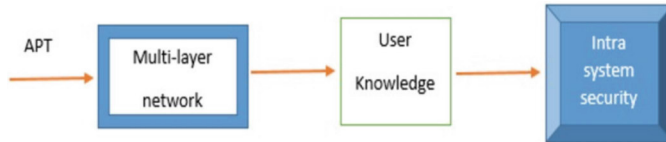


Fig. 2. APT Life Cycle

Defensive Measure 2: Inspired from batch processing in “Hadoop” an open-source Real time computational system has been designed which functions on the method of multilayer “Gene Pool” which has a semantic-rich network behavior pattern contains inside it, which facilitates the detection of APT. In Phase one a network packet is fetched

from a network stream and is preprocessed and transformed to extract the appropriate message behavior information. This information is sent to second and third phase which regulates the matching of message extracted in phase one with a semantic rich network behavior pattern and conclude the presence of anomaly if any. Phase four is in charge of response towards anomaly either by anomaly alarm or recording the anomaly pattern directly.

4 Conclusion

APTs have turned into a significant concern for organizations from different fields. The use of Responsible AI to conduct research on APTs allow us to dig deep in the weeds to get the insights, which are not usually visible on a clear canvas of the threat landscape. This pattern is not hinting any signs of slowdown. Because of their stealthiness and spotlight on information, APTs present a developing peril that is difficult to avert, distinguish and shield against. This is where the incorporation of LLM permits us to establish patterns, by applying Cosine-Similarity for extrapolating the possible underlined threats. Lately, we have seen various situations where APTs have brought on noteworthy harm to different associations, in type of information robbery, as well as quick physical harm, this is also an area of interest that we will later be exploring using Responsible AI as our future work. APTs have an advanced way to deal with attacking organizations, they have the assets and learning, they don't stop surrender when they are unsuccessful, they cover their nearness, and they focus on association's important assets. Thus, new security methodologies are required. Associations can't depend on customary firewalls and malware discovery. They have to utilize different advanced intrusion detection tools and, in view of their exercises, develop new techniques for identifying abnormalities in a particular system, and correspond them keeping in mind the end goal to find security ruptures. Despite the fact that there is no insurance that any system is totally secured, organizations can implement tools and security strategies that can extraordinarily decrease the danger of an attack. Notwithstanding, doing that requires learning, assets and time.

References

- Sood, A.K., Enbody, R.J.: Targeted cyberattacks: a superset of advanced persistent threats. *IEEE Secur. Priv.* **11**(1), 54–61 (2013)
- Vukalović, J., Delija, D.: Advanced persistent threats - detection and defense. In: 2015 38th Int. Conv. Inf. Commun. Technol. Electron. Microelectron. MIPRO 2015 - Proc., no. May, pp. 1324–1330 (2015)
- Kaspersky: "APT Trends Report," Securelist, [Online]. Available: <https://apt.securelist.com/#secondPage>. [Accessed: Nov. 10, 2024]
- Collberg, C.: Advanced Persistent Threats. University of Arizona, [Online]. Available: <http://www.cs.arizona.edu/~collberg/Teaching/466-566/2012/Resources/presentations/2012/topic9final/report.pdf>. [Accessed: Nov. 15, 2024]
- Greenberg, A.: Iran's nuclear computer attack. *Forbes*, [Online]. Available: <http://www.forbes.com/2010/10/06/iran-nuclear-computer-technology-security-stuxnet-worm.html>. [Accessed: Nov. 20, 2024]

- Langner, R.: “Stuxnet: Dissecting a cyberweapon,” Langner Group, [Online]. Available: http://embeddeds.w.net/doc/Stuxnet_white_paper.html. [Accessed: Dec. 5, 2024]
- Antoniades, D., Vasiliadis, G., Ioannidis, S.: Advanced Persistent Threats: A Survey. Computing and Information Systems, [Online]. Available: https://www.cis.aueb.gr/Publications/ARE_S2013%20APT%20Short.pdf. [Accessed: Dec. 10, 2024]
- Kaspersky, “Wild Neutron: Economic Espionage Threat Actor Returns with New Tricks,” Securelist, [Online]. Available: <https://securelist.com/blog/research/71275/wild-neutron-economic-espionage-threat-actor-returns-with-new-tricks/>. [Accessed: Dec. 15, 2024]
- Kaspersky, “APT Trends Report,” Securelist, [Online]. Available: <https://apt.securelist.com/#secondPage>. [Accessed: Jan. 3, 2025]
- Collberg, C.: Advanced Persistent Threats. University of Arizona, [Online]. Available: <http://www.cs.arizona.edu/~collberg/Teaching/466-566/2012/Resources/presentations/2012/topic9final/report.pdf>. [Accessed: Jan. 8, 2025]
- Greenberg, A.: Iran’s nuclear computer attack. Forbes, [Online]. Available: <http://www.forbes.com/2010/10/06/iran-nuclear-computer-technology-security-stuxnet-worm.html>. [Accessed: Jan. 12, 2025]
- Langner, R.: Stuxnet: Dissecting a cyberweapon. Langner Group, [Online]. Available: http://embeddeds.w.net/doc/Stuxnet_white_paper.html. [Accessed: Jan. 18, 2025]
- Antoniades, D., Vasiliadis, G., Ioannidis, S.: Advanced Persistent Threats: A Survey,” Computing and Information Systems, [Online]. Available: https://www.cis.aueb.gr/Publications/ARE_S2013%20APT%20Short.pdf. [Accessed: Jan. 22, 2025]
- Kaspersky: “Wild Neutron: Economic Espionage Threat Actor Returns with New Tricks,” Securelist, [Online]. Available: <https://securelist.com/blog/research/71275/wild-neutron-economic-espionage-threat-actor-returns-with-new-tricks/>. [Accessed: Jan. 27, 2025]



Enhancing Anomaly Detection in Software Logs with Bi-LSTM and Attention Mechanisms

Zulfiqar Ali¹(✉) , Israr Ur Rehman² ,
Muhammad Abdul Basit Ur Rahim³ , and Jonathan Witkowski³

¹ NFC Institute of Engineering and Fertilizer Research, Faisalabad, Pakistan
zulfiqar.ali@ieftr.edu.pk

² Pak-Austria Fachhochschule Institute of Applied Sciences and Technology,
Haripur, Pakistan
israr.rehman@paf-iast.edu.pk

³ California State University, Long Beach, USA
m.basit@csulb.edu, Jonathan.Witkowski01@student.csulb.edu

Abstract. The volume and complexity of log data have increased exponentially due to the rapid proliferation of modern software systems, making anomaly detection a crucial task for maintaining system security and dependability. By combining attention processes with Bidirectional Long Short-Term Memory (Bi-LSTM) networks, this research introduces a novel method for software log anomaly identification. The proposed model leverages the sequential dependencies in log data captured by Bi-LSTM, while the attention mechanism enhances the interpretability and focuses on key features indicative of anomalies. We compare the model's performance against state-of-the-art techniques in terms of detection accuracy, precision, recall, and F1-score using publically accessible log datasets. The results highlight the robustness of the proposed approach in identifying subtle and complex anomalies, offering a scalable solution for real-world applications. This study not only advances anomaly detection capabilities in software systems but also provides insights into the synergistic effects of deep learning architectures and attention mechanisms for log analysis.

Keywords: Anomaly detection · Log analysis · Bi-LSTM · Attention mechanism · Feature fusion

1 Introduction

The unprecedented growth of software systems in modern industries has led to massive amounts of log data, which serve as vital records for monitoring, diagnosing, and maintaining system health. These logs are essential for detecting abnormalities that point to possible problems like software defects, security breaches, or system failures [11]. They provide comprehensive traces of system

operations. However, typical anomaly detection techniques which frequently rely on manual feature extraction or heuristic-based approaches, face considerable problems due to the amount and complexity of log data. These methods struggle to adapt to the dynamic nature of logs, often resulting in limited scalability, reduced accuracy, and high false-positive rates [10]. In recent years, deep learning has emerged as a transformative approach to anomaly detection [15], offering the ability to learn complex patterns directly from raw data [19]. These techniques, such as Bidirectional Long Short-Term Memory (Bi-LSTM) [21] networks, are ideal for log analysis since they have demonstrated exceptional success in capturing temporal dependencies in sequential data. However, while Bi-LSTM models excel at learning sequential relationships, they can struggle with interpretability and fail to adequately emphasize the most critical features of the input data adequately [1]. To address these challenges, this study proposes a novel framework integrating Bi-LSTM networks with attention mechanisms to enhance anomaly detection in software logs. The attention mechanism enables the model to dynamically focus on the log sequences' most relevant parts, thereby improving detection accuracy and model interpretability. By combining the strengths of Bi-LSTM and attention, the proposed approach not only captures long-range dependencies in the data but also highlights key features indicative of anomalies. We perform extensive trials on publicly available log datasets, such as HDFS, BGL, and OpenStack logs, to verify the efficacy of our methodology. The outcomes show that our model outperforms the most advanced techniques, greatly enhancing detection accuracy, precision, recall, and F1-score. Furthermore, the incorporation of attention mechanisms enhances the model's practical usefulness in real-world circumstances by offering insightful information about the model's decision-making process. This paper makes three key contributions:

1. Proposing a novel Bi-LSTM with an attention mechanism framework for anomaly detection in software logs.
2. Demonstrating the model's effectiveness through comprehensive evaluations with benchmark datasets.
3. Highlighting the interpretability benefits provided by attention mechanisms, which enhance the understanding of detected anomalies.

The remainder of the document is structured as follows: Sect. 2, relevant work in deep learning and anomaly detection for log analysis is reviewed. The problem is defined in Sect. 3. The suggested methodology, including the architecture and training procedure, is explained in Sect. 4. The experimental setup and findings are presented in Sect. 5, which is followed by an ablation analysis and discussion. Section 6 wraps up the work and suggests areas for further research.

2 Related Work

Due to its significance in guaranteeing the dependability, security, and effectiveness of software systems, anomaly detection in software logs [19] has been

a focus of current study. Both conventional and machine learning-based methods have been thoroughly investigated for this purpose [3, 9], yet the emergence of deep learning has brought transformative advancements in the field. This section provides an overview of existing techniques categorized into traditional methods, machine learning approaches, and deep learning methods, followed by a discussion of attention mechanisms in anomaly detection. Traditional anomaly detection techniques, such as rule-based systems, statistical models, and clustering algorithms, have long been used for log analysis. Rule-based methods rely on predefined rules to identify deviations, but they require extensive domain expertise and are not adaptable to dynamic systems. Statistical approaches, including PCA (Principal Component Analysis) and autoregressive models, aim to detect deviations based on probabilistic thresholds. While these methods are computationally efficient, they often fall short when dealing with high-dimensional, unstructured log data [8].

Automated feature extraction and more precise anomaly detection were made possible by machine learning techniques [13]. Log data has been categorised into normal and anomalous groups using supervised models like Random Forests and Support Vector Machines (SVM). Nevertheless, these models necessitate labelled data, which is either costly or unavailable in large-scale systems. [20]. Unsupervised learning techniques, including Isolation Forests and Autoencoders, have gained popularity for anomaly detection in scenarios with limited or no labeled data. Autoencoders, in particular, have been widely adopted due to their ability to reconstruct input data and identify deviations based on reconstruction errors. Despite these advances, machine learning models often fail to capture temporal dependencies and sequential patterns in log data, which are crucial for detecting anomalies in complex systems.

Similarly, the introduction of deep learning has transformed software log anomaly detection by allowing models to directly learn hierarchical representations from unprocessed data. Sequential dependencies in log data have been widely captured by recurrent neural networks (RNNs) and their variations, including Long Short-Term Memory (LSTM) networks [4] [16]. Additionally, by processing input sequences in both forward and backward directions, bidirectional LSTM (Bi-LSTM) networks improve the model's capacity to grasp long-range dependencies [14]. Bi-LSTMs are excellent at sequence modelling, however, they frequently lack interpretability and might not pay attention to the most important portions of the input sequences. [12]. Recent studies have explored hybrid approaches that integrate LSTMs with attention mechanisms to address these challenges. An Attention-Augmented LSTM model for log-based anomaly identification is proposed in [6], showing the capacity to more accurately detect minor anomalies. Similarly, to improve anomaly detection in industrial time-series data, Haque et al. [7] integrated GRUs with attention mechanisms. These experiments demonstrate how attention mechanisms might enhance deep learning models' interpretability and accuracy. Furthermore, attention mechanisms have been used more and more in anomaly detection tasks since they were initially made popular in natural language processing (NLP) by models like as Transformers [5, 17]. For log anomaly identification, Su et al. [17]

used a Transformer-based model that used self-attention to identify both local and global relationships in the data. Although Transformer models have shown promising results, their computational complexity can be a limiting factor in resource-constrained environments [2]. Hybrid approaches, such as integrating attention with Bi-LSTM, offer a balance between computational efficiency and model performance, making them well-suited for log anomaly detection tasks [10]. Despite significant advancements, several challenges remain in anomaly detection in software logs [18, 19]. Traditional and machine learning methods often struggle with unstructured and high-dimensional log data. While deep learning models such as Bi-LSTM have demonstrated superior performance, they lack interpretability and fail to emphasize critical features indicative of anomalies. Attention mechanisms have emerged as a promising solution [17], but their integration with sequence models for log anomaly detection is still an active area of research [14]. This study suggests a Bi-LSTM model that has been improved with attention mechanisms to better detect anomalies in software logs in order to fill in these gaps. The suggested method seeks to provide improved interpretability and scalability for practical applications, all while achieving state-of-the-art performance.

3 Problem Definition

Anomaly detection in software logs is a fundamental task in ensuring system reliability and security. Modern software systems generate vast amounts of log data, which consist of sequential events representing system activities. These logs provide valuable insights into system behavior, facilitating fault diagnosis, intrusion detection, and performance monitoring. However, identifying anomalies in log data presents several challenges due to the unstructured nature of logs, the complexity of event sequences, and the rarity of anomalies in real-world datasets. Let the software log data be represented as a sequence of events:

$$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$$

where $\mathbf{x}_t \in \mathbb{R}^d$ denotes the feature vector of the log event at time t , d is the dimensionality of the feature vector, and T is the total number of events in the sequence.

The objective is to classify each log event $\mathbf{x}_t$ as either **normal** or **anomalous**. This can be formalized as:

$$\mathbf{Y} = \{y_1, y_2, \dots, y_T\}, \quad y_t \in \{0, 1\}$$

where $y_t = 1$ indicates an anomaly at time t , and $y_t = 0$ indicates normal behavior.

The challenges associated with this problem include:

- **High dimensionality and unstructured nature:** Log data is often unstructured and heterogeneous, requiring effective feature extraction methods to represent meaningful patterns.

- **Temporal dependencies:** Anomalies are context-dependent and often require capturing sequential relationships between events in the log data.
- **Imbalanced data:** Real-world datasets are often heavily imbalanced, with normal events significantly outnumbering anomalous events, making the detection of rare anomalies more difficult.

The goal is to design a model that learns a mapping:

$$f : \mathbf{X} \rightarrow \mathbf{Y}$$

such that the discrepancy between the predicted labels $\hat{\mathbf{Y}}$ and the ground truth labels $\mathbf{Y}$ is minimized. This involves accurately detecting anomalies in log data while addressing the inherent challenges of temporal dependencies, data imbalance, and scalability.

4 Proposed Model

The suggested model uses an attention mechanism and Bidirectional Long Short-Term Memory (Bi-LSTM) networks to identify anomalies in software log data. The following is the architecture: The software log data is preprocessed into a sequence of feature vectors:

$$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}, \quad \mathbf{x}_t \in \mathbb{R}^d \quad (1)$$

where d is the feature vector's dimensionality, T is the total number of events, and $\mathbf{x}_t$ is the feature vector of the log event at time step t .

The sequence $\mathbf{X}$ is run through a Bidirectional LSTM (Bi-LSTM) layer to record temporal dependencies. Each time step's hidden states are calculated as follows:

$$\vec{\mathbf{h}}_t = \text{LSTM}_{\text{forward}}(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1}), \quad \overleftarrow{\mathbf{h}}_t = \text{LSTM}_{\text{backward}}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1}) \quad (2)$$

The final hidden state is given by:

$$\mathbf{h}_t = \begin{bmatrix} \vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t \end{bmatrix}, \quad \mathbf{h}_t \in \mathbb{R}^{2h} \quad (3)$$

where h is the dimensionality of the hidden state for each direction.

An attention mechanism is applied over the hidden states to focus on the most critical events. The attention score for each time step is computed as:

$$\alpha_t = \frac{\exp(\mathbf{u}_t^\top \mathbf{w})}{\sum_{k=1}^T \exp(\mathbf{u}_k^\top \mathbf{w})}, \quad \alpha_t \in [0, 1] \quad (4)$$

where $\mathbf{u}_t = \tanh(\mathbf{W}_h \mathbf{h}_t + \mathbf{b}_h)$, and $\mathbf{W}_h, \mathbf{b}_h, \mathbf{w}$ are learnable parameters. The context vector is computed as:

$$\mathbf{c}_t = \sum_{k=1}^T \alpha_k \mathbf{h}_k \quad (5)$$

The context vector $\mathbf{c}_t$ is passed through a fully connected layer with a softmax activation to predict the probability of the event being normal or anomalous:

$$\hat{y}_t = \text{Softmax}(\mathbf{W}_c \mathbf{c}_t + \mathbf{b}_c), \quad \hat{y}_t \in [0, 1] \quad (6)$$

where $\mathbf{W}_c$ and $\mathbf{b}_c$ are learnable parameters.

The model is trained by minimizing the cross-entropy loss:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T [y_t \log(\hat{y}_t) + (1 - y_t) \log(1 - \hat{y}_t)] \quad (7)$$

where y_t is the ground truth label and $\hat{y}_t$ is the predicted probability. The suggested model's general process is described in Algorithm 1.

Algorithm 1. Anomaly Detection in Software Logs using Bi-LSTM and Attention

Require: Log sequence $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_T\}$, model parameters θ

Ensure: Predicted anomaly labels $\hat{\mathbf{Y}}$

Preprocessing: Convert logs into feature vectors and normalize.

Bi-LSTM Encoding:

for each $t \in \{1, \dots, T\}$ **do**

 Compute forward and backward LSTM hidden states.

 Concatenate hidden states: $\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$

end for

Attention Mechanism:

 Compute attention scores α_t and context vectors $\mathbf{c}_t$.

Classification:

 Compute predicted anomaly probability:

$$\hat{y}_t = \text{Softmax}(\mathbf{W}_c \mathbf{c}_t + \mathbf{b}_c)$$

Loss Computation: Compute cross-entropy loss and update parameters.

return $\hat{\mathbf{Y}}$

5 Experimental Evaluation

5.1 Dataset

We employ three publicly accessible benchmark datasets—HDFS Logs, BGL Logs, and Thunderbird Logs—to assess our model's performance. Table 1 provides a summary of the statistics and overview of a few chosen datasets. **HDFS Logs Dataset** This dataset contains event logs generated by the Hadoop Distributed File System, which is widely used for anomaly detection research. It consists of millions of log entries from distributed systems and includes both normal and anomalous events, labeled for supervised learning. The dataset captures a variety of real-world scenarios, such as missing blocks, checksum errors,

and replication failures, making it highly suitable for evaluating anomaly detection models. **BGL Logs Dataset** This dataset contains logs collected from the Blue Gene/L supercomputer, one of the fastest supercomputers of its time. It features labeled log entries that identify normal and anomalous events, such as hardware failures, software bugs, and system misconfigurations. With millions of entries and diverse anomaly types, the dataset provides a challenging benchmark for anomaly detection methods. Researchers frequently use it to test machine learning models in large-scale distributed systems. **Thunderbird Logs Dataset** This dataset comprises system log entries collected from the Thunderbird supercomputer, containing a mixture of normal and anomalous events. This dataset highlights issues like memory errors, disk failures, and node outages, reflecting real-world challenges in high-performance computing environments. The detailed labeling and varied log patterns make it an excellent resource for evaluating anomaly detection techniques.

Table 1. Statistics of Selected Datasets for Anomaly Detection in Software Logs.

Dataset	Log Entries	Anomalies	Labels Available
HDFS Logs	11,175,629	16,838	Yes
BGL Logs	4,747,963	348,460	Yes
Thunderbird Logs	2,037,197	42,354	Yes

5.2 Evaluation Measures

The proposed model is evaluated using rating and ranking performance measures to assess classification accuracy and ranking effectiveness. Rating performance measures evaluate the model’s ability to classify instances correctly. Metrics such as Accuracy, Precision, Recall, and F1-score quantify prediction quality. Accuracy determines the proportion of correctly classified instances, while Precision and Recall measure the correctness and completeness of positive predictions, respectively. The F1-Score offers a balanced evaluation by taking the harmonic mean of Precision and Recall; it is especially helpful for datasets that are unbalanced. In log-based anomaly classification, these indicators are essential for guaranteeing the model’s dependability. Ranking performance metrics evaluate how well predictions are arranged according to relevance, which is essential for setting anomaly priorities. Measures like Normalised Discounted Cumulative Gain (NDCG) and Area Under the Curve (AUC) are employed. AUC assesses how well the model can differentiate between anomalous and normal logs at various levels. To ensure ideal prioritisation, NDCG takes into account both the instances’ ranking positions and their significance.

5.3 Baseline Models

We assess and contrast our suggested model’s performance in our experiments with a number of cutting-edge models. These chosen models improve the performance of anomaly detection by utilizing deep learning techniques.

- **Long Short-Term Memory (LSTM)** LSTM is a recurrent neural network capable of learning long-term dependencies in sequential data. Its gating mechanism allows it to effectively retain and forget information, making it a strong candidate for anomaly detection in time-series data. However, LSTM may not fully exploit bidirectional context, which can limit its performance in capturing dependencies across a sequence.
- **Convolutional Neural Network (CNN)** CNNs, traditionally used for image processing, are adapted here to detect anomalies in software logs by treating sequences as one-dimensional inputs. They excel in feature extraction, identifying local patterns effectively. Nevertheless, CNNs cannot model sequential dependencies, which can reduce their effectiveness in temporal data.
- **Transformer Model** The Transformer architecture employs self-attention mechanisms to model global dependencies within sequential data. It has proven highly effective in capturing complex patterns and relationships in log data. However, its computational complexity can pose challenges for training, particularly in resource-constrained environments.
- **GRU with Attention Mechanism** Gated Recurrent Units (GRUs) are computationally efficient variants of LSTM, and when combined with an attention mechanism, they focus on the most relevant parts of input sequences. This baseline captures important temporal dependencies and highlights key patterns, but it may still underperform in scenarios requiring bidirectional sequence processing.
- **Deep Log Analyzer (DLA)** DLA is a specialized model for anomaly detection in software logs. It combines sequential modeling and statistical analysis to identify irregular patterns in log data. While it is effective for structured logs, it may struggle to generalize to more complex or semi-structured datasets.

5.4 Results and Discussions

Four primary rating performance metrics—Accuracy, Precision, Recall, and F1-Score are used to compare the suggested model’s performance against baseline models across three datasets: HDFS Logs, BGL Logs, and Thunderbird Logs. Here, Table 2 displays the findings for Accuracy and F1-Score, whereas Fig. 1 displays the results for Precision and Recall. The findings show that the Proposed Model achieves the greatest scores across all datasets and evaluation parameters, routinely outperforming all baseline models. The proposed model achieves the highest accuracy, recording 91.2% on HDFS Logs and 90.5% on BGL Logs. The Transformer-based model follows closely but remains inferior to the proposed model, while CNN-based architectures exhibit the lowest accuracy across all datasets. Similarly, for precision, the proposed model achieves the highest precision with 89.8% on HDFS Logs and 89.0% on BGL Logs, demonstrating its ability to minimize false positives. The Transformer-based model performs slightly worse, while CNN and LSTM models show relatively lower precision

values, indicating their susceptibility to misclassification. In terms of recall, the proposed model again surpasses the baseline models, achieving 90.1% on HDFS Logs and 89.3% on BGL Logs. The GRU with Attention model follows in performance, while CNN-based models exhibit the lowest recall values, signifying their tendency to miss a considerable number of actual anomalies. Finally, the F1-Score further validates the effectiveness of the proposed model. The highest F1-scores are observed for the proposed model, with 90.0% on HDFS Logs and 89.5% on BGL Logs. The Transformer-based model follows with slightly lower values, while CNN and LSTM models remain the weakest performers. Thus, the proposed model significantly outperforms all baseline models across all key performance metrics. The Transformer-based model emerges as a competitive alternative but remains inferior to the proposed approach. Conversely, CNN-based architectures exhibit the weakest performance, indicating their limitations in learning sequential dependencies in log data.

Table 2. Comparison of Accuracy and F1-Score Measures Between Proposed Model and Baselines across HDFS Logs, BGL Logs, and Thunderbird Logs datasets, where the best values are highlighted in bold.

Model	HDFS Logs		BGL Logs		Thunderbird Logs	
	Accuracy (%)	F1-Score (%)	Accuracy (%)	F1-Score (%)	Accuracy (%)	F1-Score (%)
LSTM	85.2	83.9	84.5	82.9	83.8	81.8
CNN	82.4	80.6	81.8	79.8	81.2	79.1
Transformer	88.9	87.5	88.3	87.0	87.5	86.2
GRU with Attention	87.6	86.7	87.1	86.0	86.4	85.1
Deep Log Analyzer (DLA)	84.1	82.9	83.5	82.0	82.8	81.3
Proposed Model	91.2	90.0	90.5	89.5	89.8	88.7

Similarly, Table 3 compares ranking performance across HDFS Logs, BGL Logs, and Thunderbird Logs, using AUC, and NDCG@10. The proposed model consistently outperforms baseline methods across all datasets and metrics. For HDFS Logs, the proposed model achieves the highest AUC 93.2%, surpassing the Transformer 90.8% and GRU with Attention 89.4%. Similarly, it attains the best NDCG@10 91.5%, demonstrating superior ranking effectiveness. In BGL Logs, the proposed model maintains its lead with an AUC of 92.8%, and NDCG@10 of 91.1%, outperforming the baselines. This highlights the advantage of its enhancements over attention-based baselines. For Thunderbird logs, the proposed model again achieves the best results i.e. AUC: 92.3%, and NDCG@10: 90.4%, followed by the Transformer. While CNN and Deep Log Analyzer remain competitive, sequential and attention-based methods prove more effective. Overall, the proposed model demonstrates robustness and efficiency in ranking log-based anomalies, consistently outperforming baselines across all datasets.

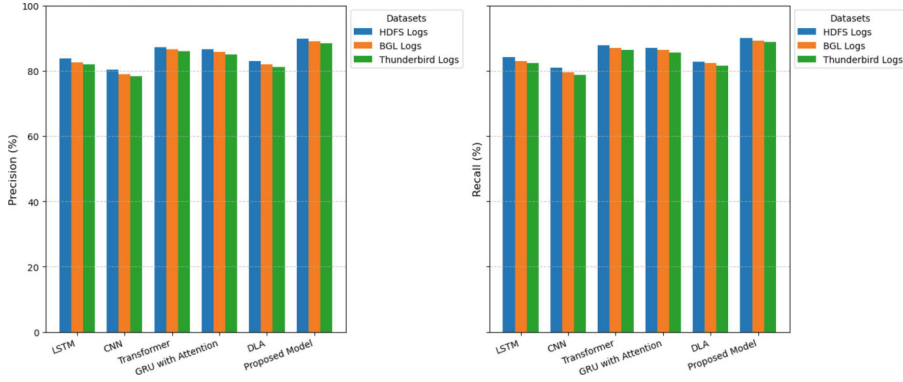


Fig. 1. Precision and Recall Comparison of the Proposed Model with Baseline Models.

Table 3. Comparison of AUC and NDCG@10 Measures Between Proposed Model and Baselines across HDFS Logs, BGL Logs, and Thunderbird Logs datasets, where the best values are highlighted in bold

Model	HDFS Logs		BGL Logs		Thunderbird Logs	
	AUC (%)	NDCG@10 (%)	AUC (%)	NDCG@10 (%)	AUC (%)	NDCG@10 (%)
LSTM	87.5	85.4	86.8	84.7	85.9	83.2
CNN	84.2	81.8	83.6	80.9	82.7	79.5
Transformer	90.8	89.3	90.2	88.7	89.6	87.9
GRU with Attention	89.4	88.1	88.8	87.6	88.1	86.5
Deep Log Analyzer (DLA)	85.6	83.2	84.9	82.5	84.1	81.7
Proposed Model	93.2	91.5	92.8	91.1	92.3	90.4

5.5 Ablation Analysis

To evaluate the impact of key components, we conducted an ablation study, on HDFS Logs, BGL Logs, and Thunderbird Logs. compares Accuracy, Precision, Recall, and F1-Score across model variants. The full model achieves the best performance, with F1-Scores of 91.3% for HDFS Logs, while 90.8% for BGL Logs, and 90.0% for Thunderbird Logs, confirming its effectiveness. Removing the attention mechanism reduces accuracy by 3.3%, showing its role in feature enhancement. Excluding Bi-LSTM significantly lowers recall by 4.5%, indicating its importance in capturing sequence dependencies. The feature fusion layer removal mainly impacts precision and F1-Score, while context embedding removal causes the highest drop, with a 5.2% accuracy decline. Thus, attention mechanisms, Bi-LSTM, feature fusion, and contextual embeddings are essential, with attention and Bi-LSTM having the most impact on performance (Fig. 2).

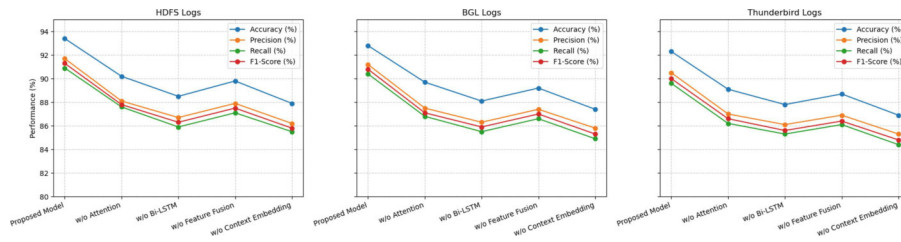


Fig. 2. Ablation analysis of the proposed model.

6 Conclusion and Future Work

In this paper, we proposed a Bi-LSTM-based anomaly detection model with an attention mechanism to identify anomalies in software log data. Our approach effectively captures temporal dependencies and emphasizes critical log events through attention weighting, leading to improved anomaly detection performance. The experimental results demonstrate that our model outperforms existing baselines in terms of accuracy and robustness, particularly in handling imbalanced log data. The ablation study further validates the impact of each component of the model, highlighting the importance of Bi-LSTM and attention in achieving superior results. Despite the promising performance, there are several avenues for future research. First, incorporating external knowledge sources, such as domain-specific log patterns, could enhance the model's interpretability and precision. Second, exploring self-supervised learning techniques may reduce dependency on labeled datasets and improve generalization to unseen log data. Finally, optimizing the computational efficiency of our approach could facilitate real-time anomaly detection in large-scale systems.

References

1. Shaker AbdulAziz Ali: Anomaly detection in telecommunication networks: leveraging novel big data and machine learning techniques for proactive fault management. *Educ. Adm. Theory Pract.* **30**(5), 5751–5770 (2024)
2. Cao, Y., Tang, X., Deng, X., Wang, P.: Fault detection of complicated processes based on an enhanced transformer network with graph attention mechanism. *Process Saf. Environ. Prot.* **186**, 783–797 (2024)
3. Edozie, E., Shuaibu, A.N., Sadiq, B.O., John, U.K.: Artificial intelligence advances in anomaly detection for telecom networks. *Artif. Intell. Rev.* **58**(4), 100 (2025)
4. Fayyadh, S.A.F.: Automatic detection of intrusion attacks in iot networks using bi-lstm-cnn neural network. Master's thesis, Kirsehir Ahi Evran University (Turkey) (2023)
5. Galassi, A., Lippi, M., Torrioni, P.: Attention in natural language processing. *IEEE Trans. Neural Netw. Learn. Syst.* **32**(10), 4291–4308 (2020)
6. Gong, K., Luo, S., Pan, L., Zhang, L., Zhang, Y., Haomiao, Yu.: Logeta: time-aware cross-system log-based anomaly detection with inter-class boundary optimization. *Futur. Gener. Comput. Syst.* **157**, 16–28 (2024)

7. Haque, M.N., et al.: Gru-based attention mechanism for human activity recognition. In: 2019 1st International Conference on Advances in Science, Engineering and Robotics Technology (ICASERT), pp. 1–6. IEEE (2019)
8. He, S., He, P., Chen, Z., Yang, T., Su, Y., Lyu, M.R.: A survey on automated log analysis for reliability engineering. *ACM Comput. Surv. (CSUR)* **54**(6), 1–37 (2021)
9. Karunaratne, T.: Machine learning and big data approaches to enhancing e-commerce anomaly detection and proactive defense strategies in cybersecurity. *J. Adv. Cybersecur. Sci. Threat Intell. Countermeas.* **7**(12), 1–16 (2023)
10. Liu, Y., Ren, S., Wang, X., Zhou, M.: Temporal logical attention network for log-based anomaly detection in distributed systems. *Sensors* **24**(24), 7949 (2024)
11. Lu, R., Zhu, X., Li, X., Long, N., Zhang, G.: An anomaly monitoring and early warning method for power grid microservice network based on log visualisation and analysis. In: 2024 5th International Conference on Machine Learning and Computer Application (ICMLCA), pp. 584–590. IEEE (2024)
12. Maxima, A.: Integration and analysis of unstructured data towards database optimization and decision making using deep learning techniques. PhD thesis, Kampala International University (2024)
13. Mienye, I.D., Swart, T.G., Obaido, G.: Recurrent neural networks: a comprehensive review of architectures, variants, and applications. *Information* **15**(9), 517 (2024)
14. Natha, S., Ahmed, F., Siraj, M., Lagari, M., Altamimi, M., Chandio, A.A.: Deep BiLSTM attention model for spatial and temporal anomaly detection in video surveillance. *Sensors* **25**(1), 251 (2025)
15. Pang, G., Shen, C., Cao, L., Van Den Hengel, A.: Deep learning for anomaly detection: a review. *ACM Comput. Surv. (CSUR)* **54**(2), 1–38 (2021)
16. Ur Rehman, I., Ali, Z., Jan, Z.: A collaborative cnn-lstm framework with attention mechanism for aspect-based sentiment analysis. In: 2024 International Conference on Frontiers of Information Technology (FIT), pp. 1–6. IEEE (2024)
17. Su, J., et al.: Large language models for forecasting and anomaly detection: a systematic literature review. arXiv preprint [arXiv:2402.10350](https://arxiv.org/abs/2402.10350) (2024)
18. Sun, Y., Keung, J., Yang, Z., Liu, S., Zhang, J.: Advancing semi-supervised anomaly detection in software logs with deep grouping and auto-optimization. SSRN 4918203 (2024)
19. Sun, Y., Keung, J., Yang, Z., Liu, S., Zhang, J., Liao, Y.: Improving anomaly detection in software logs through hybrid language modeling and reduced reliance on parser. SSRN 4978351
20. ur Rehman, I., Ali, W., Jan, Z., Ali, Z., Xu, H., Shao, J.: CAML: contextual augmented meta-learning for cold-start recommendation. *Neurocomputing* **533**, 178–190 (2023)
21. Yang, Y., et al.: Intrusion detection based on bidirectional long short-term memory with attention mechanism (2023)



FIRMMOD: Generating API Taint Models for Firmware Analysis

Ken Yihang Bai^(✉)  and Tuba Yavuz 

University of Florida, Gainesville, FL, USA
{baiyihang, tyavuz}@ufl.edu

Abstract. Firmware is an important part of the attack surface for the Internet of Things. Attackers typically inject malicious data through the peripherals. Understanding the impact of attacker-controlled data requires an understanding of where such data flows within software, which is challenging due to the complex peripheral behavior. We present a symbolic execution-based firmware rehosting approach that utilizes MMIO access guided API modeling to avoid the complexity in handling low-level details of peripheral behaviors and to precisely track the flows of the peripheral data registers into the upper layers in the software stack. We have developed FIRMMOD on top of S2E for ARM Cortex M3 and applied it to Amazon FreeRTOS firmware samples. Our results indicate that API modeling is an effective approach for goal-based reachability analysis for firmware. It enables FIRMMOD to achieve up to 2X more code coverage compared to Fuzzware and to detect two known buffer overflow vulnerabilities at the ES WIFI layer.

Keywords: firmware analysis · symbolic execution · memory errors

1 Introduction

Firmware is an important part of the Internet of Things (IoT) attack surface. It is typically formed of multiple layers including the application layer, third-party library layer, the Hardware Abstraction Layer (HAL), and the device driver layer. Among the various security risks include firmware developers' reliance on the correctness of third-party libraries without proper vetting for vulnerabilities and failure to properly sanitize the data received from the peripherals. So, attackers can exploit the vulnerabilities in various part of the firmware by carefully crafting malicious data that flows from the peripherals into various layers of the firmware. Therefore, it is important to precisely track the data flows from the peripherals and analyze how the data gets used by the firmware.

Firmware rehosting has become an effective way to analyze firmware without an actual device. Various approaches that utilize firmware rehosting either use fuzzing [7, 8, 11], symbolic execution [5, 10], or a combination of both [13, 14]. A challenge in firmware analysis is the complexity of low-level interactions with the peripherals. Approaches that use symbolic execution typically use symbolic

values for the data received from the peripherals. However, the complex peripheral interactions in the device driver layer lead to a path explosion and prevents achieving high coverage in the upper layers. This also poses as a challenge to fuzzing, which needs to generate input that will try to cover a variety of cases in the peripheral interaction logic along with the code in the upper layers. So, previous work deals with the problem by learning some peripheral model to achieve high coverage, which is typically measured in terms of basic block coverage. State-of-the-art firmware rehosting approaches assume that the firmware exists in binary form only. Although this is a valid assumption for vetting firmware in the wild, there are other scenarios that invalidates this assumption such as an IoT vendor using firmware rehosting to test IoT device firmware, which allows the detection of vulnerabilities before deployment. As we show in this paper, assuming the existence of source code and the availability of debug information allows for more effective model generation and tracking the flow of peripheral data to upper layers in the firmware.

In this paper, we present, FIRMMOD, an MMIO access guided API modeling approach to goal-based firmware analysis. We assume that the analyst or the developer is interested in assessing the security risks associated with data flows from a specific set of target peripherals within the specific target layers of the firmware. We also assume the availability of debugging information as an IoT vendor may want to vet their firmware before deployment. Our approach deals with the complexity of peripheral interactions by selectively modeling the lower level APIs that process such data. FIRMMOD applies precise tracking of the MMIO accesses to the data registers of the target peripherals and generates taint models for API that process data from these peripherals. FIRMMOD decides the scope of API modeling based on the amount of coverage achieved. So, it may decide to apply modeling to the callers of the modeled API as long as they are not part of the target layers. For the peripherals that are not in the target set, FIRMMOD applies a less precise and more aggressive modeling.

Our approach leverages the insight that although imprecise modeling of firmware peripherals may not be reliable in reasoning about the reachability of certain bugs or vulnerabilities in firmware, it may still identify code locations that may have a higher security risk due to accessing data tainted with potentially unconstrained or improperly constrained peripheral data. Although the imprecision in the peripheral modeling may still incur imprecision to the taint propagation or the constraints on the state of the peripheral data, the memory locations that get tainted with peripheral data are less likely to differ depending on the peripheral state.

We formulate and evaluate the following research questions (RQs) in this paper:

- **RQ1:** How does the scope of API modeling impact firmware analysis?
- **RQ2:** Does API modeling enable bug finding in upper layers?
- **RQ3:** How does FIRMMOD perform compared to Fuzzware, a state-of-the-art firmware rehosting approach, and to FIRMSTAT, a static API modeling approach?

This paper has been organized as follows. We place our work in the context of related work in Sect. 2. We discuss the challenges of tracking MMIO accesses in firmware and explain our solutions at a high level in Sect. 3. We present the technical details of our approach in Sect. 4. We present the results of our evaluation in Sect. 5. We conclude and discuss future work in Sect. 6.

2 Related Work

HALucinator [7] performs high level emulation of firmware by replacing Hardware Abstraction Layer (HAL) API with manually prepared models using contextual binary matching techniques. P2IM [8] presents a pattern-based extraction of concrete peripheral and firmware I/O models to achieve firmware rehosting. Both HALucinator and P2IM demonstrate the effectiveness of their rehosting approaches through fuzzing, which succeeds in finding vulnerabilities in the analyzed firmware. Para-rehosting is presented as an alternative approach to rehosting in [11]. The idea is to hook HAL API with the approximated and ported versions that could be linked with the rest of the software stack and executed on commodity processors to leverage the existing analysis support such as sanitizers and fuzzers. However, like HALucinator, this requires a manual effort while FIRMMOD generates API models automatically. Although FIRM-STAT [4] uses static analysis to automatically generate MMIO taint models of firmware APIs, its analysis overapproximates over all paths and cannot provide precise size information for the tainted regions whereas FIRMMOD uses symbolic execution to dynamically generate API taint models that include the size information.

PRETENDER [9] learns a peripheral model by combining firmware emulation with real-hardware and recording the MMIO responses and interrupt events from the device. The learnt model is used in rehosting firmware.

Laelaps [5] uses symbolic execution to rehost the firmware. It switches between symbolic execution and concrete execution for scalability and uses context-bounded symbolic execution to generate consistent concrete models of peripheral accesses.

Jetset [10] performs targeted symbolic execution by progressively generating an updated control-flow graph to steer the execution towards the goal. It generates a mostly symbolic peripheral access model as the path explosion is controlled by filtering paths based on the target and concretizes a symbolic value only after ensuring non-crashing behavior for a configured number of times. However, the peripheral access constraints and values at the time of reaching the target are used to synthesize a concrete access model to be used in QEMU for dynamic analysis to ensure reachability of a target such as the end of the boot process and continue with the further analysis of the firmware.

DICE [12] detects DMA channels used by firmware and generates more paths compared to analysis that cannot leverage DMA channels. It uses the DMA address range to identify addresses used in DMA configuration assuming the addresses are written to the peripheral registers and not to the RAM and filters

the candidates and determine the buffer sizes based on their accesses within firmware. `pEmu` [14] uses a multi-tier peripheral access modeling that associates the access values with increasingly detailed context information.

Fuzzware [13] uses locally-scoped dynamic symbolic execution to identify the parts of the peripheral data that affects the firmware logic.

SEMU [15] uses NLP-based techniques to learn the peripheral command-action rules and extract a precise peripheral model from the data sheets of MCUs. It validates the learnt model using `pEmu`.

By learning and employing API taint models, FIRMMOD automatically bypasses the complexities of peripheral access logic and leverages debug information to improve precision of the generated models. As we demonstrate in Sect. 5, FIRMMOD can reach upper layers in the firmware much faster than state-of-the-art firmware rehosting approaches.

3 Overview

In this section, we provide an overview of our approach, FIRMMOD, which aims to precisely track dataflows from the peripherals to the upper layers of firmware. FIRMMOD utilizes symbolic execution for firmware rehosting due to its path-precise semantics. Specifically, similar to other firmware rehosting approaches that use symbolic execution, it models peripheral interactions by representing data loaded from Memory Mapped I/O (MMIO) addresses with symbolic values. However, FIRMMOD needs to deal with the path explosion problem and other challenges while achieving its goal of precisely tracking dataflows from the peripherals.

These challenges can be summarized as follows:

1. **Challenge 1:** Peripheral accesses often rely on the proper triggering of interrupts and timely execution of the interrupt handlers, which represent a critical part of the taint propagation chain regarding the peripherals.
2. **Challenge 2:** Determining the size of tainted regions is not always obvious as different APIs use different mechanisms. Some APIs use a parameter that represents the size of the tainted buffer whereas others have implicit assumptions about the maximum size for the tainted region.
3. **Challenge 3:** Propagation of data received from the peripherals to upper layers get blocked or slowed down due to path explosion triggered by unbounded loops or complex logic that handles failure cases.

FIRMMOD has been designed as shown in Fig. 1. It generates MMIO taint models of functions and applies them on-the-fly when it encounters a callsite of a modeled function. However, it deals with the above challenges as it generates the function models. Specifically, it initiates taint propagation monitoring upon encountering explicit tainting and ensures monitoring of implicit taint propagations due to operations within the interrupt handlers by triggering interrupts upon encountering wait event APIs and those due to function calls by starting limited scope analysis of functions that receive tainted data as a parameter

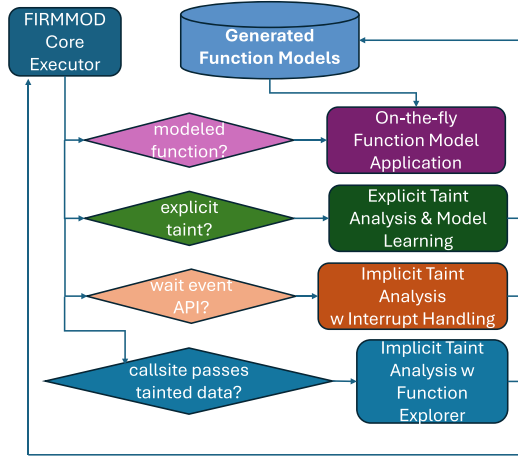


Fig. 1. An overview of the API model generation approach in FIRMMOD.

(Challenge 1). It leverages API conventions and debug information to infer the sizes of tainted buffers precisely (Challenge 2). Finally, FIRMMOD deals with the path explosion problem by leveraging API success case return value models to distinguish success paths from the failure paths (Challenge 3).

4 Approach

In this section, we motivate the problem in Sect. 4.1 and present the technical details of API taint model generation and dealing with the challenges in Sect. 4.2.

4.1 Problem Formulation

Symbolic execution of firmware creates several challenges when all reads from MMIO addresses are treated as unconstrained symbolic values. In an RTOS system, there are synchronization mechanisms such as semaphores that help tasks coordinate. When the firmware communicates with the peripherals using synchronization primitives, a precise modeling of the interrupt handling mechanism becomes imperative. An active task that processes peripheral requests typically waits for a specific event, e.g. the copying of the data received from a peripheral, by blocking on a semaphore. The active task then gets deactivated and other tasks will be executed.

If modeling of interrupts fails to lead symbolic execution to that specific event by not scheduling the code that notifies completion or occurrence of the event, e.g. performing an up on a semaphore, the task that is waiting for the event will never have a chance to be scheduled again, and the code that gets executed after the occurrence of the specific event will not be covered. In addition, branches that check conditions involving symbolic values of the control or status registers

enlarges the state space and contributes to the path explosion. However, as we discussed earlier, we care more about how the data flows from the peripherals to the upper layers rather than capturing the state of the peripherals in a precise way.

Our goal is to model the API automatically and to improve the coverage of code locations that process data received from a specific peripheral. An RTOS-based firmware typically involves the following types of APIs:

- Application level APIs, which are often defined as callbacks that are registered with some libraries.
- RTOS related APIs, e.g. those for task management.
- APIs of the third party libraries, e.g. communication protocols such as MQTT and cryptographic libraries such as mbedTLS.
- HAL APIs that provide an abstract interface of common hardware interactions.
- Board Support Package (BSP) functions, such as the drivers for specific peripherals.
- Standard C library and intrinsic functions, e.g. `strstr` or `_aeabi_f2uiz`.

An impact of the path explosion for firmware analysis is that covering success cases of the HAL APIs become challenging and such functions end up returning failure values, e.g., `HAL_ERROR`. This creates a cascading effect on the the upper layer functions, which check the return value of the HAL APIs, i.e., upper layer functions also fail, and limit code coverage. Although FIRMMOD’s overarching goal is not achieving high coverage across the whole firmware, not reaching success cases of HAL API reduces coverage of code that access peripherals of interest. Our goal is to generate a model for these HAL APIs and the related driver APIs to increase coverage of the upper layers, and, specifically the code coverage within the target function.

HAL APIs are typically designed for the following purposes:

- Initialize/enable and deinitialize/disable the peripheral.
- Configure the peripheral for specific operations.
- Transmit data via the peripheral by moving the data from a memory region to the MMIO data channel.
- Receive data either in a buffer or by a return value by moving the data from the MMIO data channel to a memory region or a register.

In this paper, the goal is to model the API that receive data from the peripheral and track its propagation to the upper layers. So, it is important to capture both the explicit and the implicit propagation of peripheral data. Explicit propagation happens when memory gets explicitly tainted by the MMIO data register in the context of a task. Implicit propagation happens in a context other than the one HAL API was running either through interrupt handling or DMA. Tracking explicit data flows is straightforward since symbolic execution of the HAL API can be monitored to detect symbolic writes to the memory, where these symbolic values originate from some MMIO data register. However, it is challenging

to track implicit data flows in a symbolic execution based firmware rehosting environment due to the path explosion induced by precise modeling of interrupt handling.

The modularity of the firmware stack enables us to model the low level APIs in a relatively modular way. The values from MMIO status or control register values are typically not propagated to the upper layers. Therefore, modeling of HAL APIs helps in controlling the path explosion problem and allowing monitoring of data flows from the peripherals. The implicit data transfer usually involves interrupt version of the HAL APIs as opposed to those that use polling.

```

1 // 1. Receive data from peripheral in normal task context
2 HAL_StatusTypeDef HAL_SPI_Receive(SPI_HandleTypeDef *hspi, uint8_t
   *pData, uint16_t Size, uint32_t Timeout) { ...
3     // receive from data register directly
4     (*(uint8_t *)pData) = (*(__IO uint8_t *)&hspi->Instance->DR; ...}
5 // 2.a.setting ISR, non blocking mode in task context
6 HAL_StatusTypeDef HAL_SPI_Receive_IT(SPI_HandleTypeDef *hspi,
   uint8_t *pData, uint16_t Size) {...
7     hspi->RxISR = SPI_RxISR_8BIT; /* setting ISR callback */ ...}
8 // 2.b interrupt line triggered, jump to IRQ handler
9 void HAL_SPI_IRQHandler(SPI_HandleTypeDef *hspi) {...
10     hspi->RxISR(hspi); /* call ISR function */ ...}
11 // 2.c Receive data in ISR callback
12 static void SPI_RxISR_8BIT(struct __SPI_HandleTypeDef *hspi)
13 {... *hspi->pRxBuffPtr++ = (*(__IO uint8_t *)&hspi->Instance->DR)
   ;...}

```

Listing 1.1. Two ways of accessing peripherals

Listing 1.1 shows two typical ways of receiving data from a peripheral using the SPI bus as the example peripheral. In the first one (1), the HAL API directly accesses the peripheral data register and copies the data to a buffer whose address is passed through some argument. In the second one (2), the interrupt mechanism is used to receive data in an asynchronous way. The important steps for this case include setting the transfer relevant callback function (2.a), execution of the callback function within the interrupt handler by passing a pointer to the handler data structure `hspi` (2.b), and the actual transfer of the data to the receive buffer whose address can be accessed through `hspi` (2.c). An important point is that, in (2) the actual data flow from the MMIO data channel to the memory happens inside an interrupt handler context.

4.2 MMIO Based Function Modeling

We hypothesize that the lower-level interactions with the firmware, which causes path explosion in symbolic execution, can be ignored to a large degree in understanding how the data from the peripherals flow into the upper layers. Therefore, the goal of our API modeling approach is to achieve a high coverage in the upper layers of the firmware while precisely keeping track of taint propagation from the peripheral data registers.

As in state-of-the-art firmware rehosting approaches, we assume that the MMIO addresses are given. We also assume that the user specifies the addresses of the MMIO data registers for the peripherals of interest. When we use the term **memory gets tainted**, we refer to the scenario where the memory gets overwritten by values from a specific MMIO data register, which is selected based on the target board implementation and it is defined as the **Target Data Channel (TDC)**. For example, STM32L475 utilizes a WIFI module for network connection, which is connected via the SPI bus. If our objective is to analyze the data traffic originating from such network connections, the TDC for analyzing the target STM32L475 board would be 0x40003c0c (SPI3 data register). Our work mainly focuses on analyzing all types of data flows from the TDC and generating appropriate function models in the scope of symbolically executing a binary firmware with some path selection heuristics. During the symbolic execution, we collect the records of function calling contexts, monitor the MMIO accessing behaviors, and dynamically model the functions as needed. Also, the values received from other data channels are treated as regular symbols and not being used for function model generation.

Overall Process. In Fig. 1, FIRMMOD starts symbolic execution of the firmware with several function models (initially, empty set). When the overall coverage among all the symbolic execution states does not increase, it pauses the execution and finds out if there exists an opportunity to generate a new function model. By analyzing the peripheral dataflow originating from the low-level layers that interact with the peripherals, FIRMMOD generates function models. Next, if a new function model gets generated, it resumes the symbolic execution from the backup state, which we refer to as the **Entry Snapshot (ES)**. ES is forked at a specific point (e.g. entry function) and it needs to restart the execution from that point to observe the effects of applying the new function model that get generated. Typically, the ES is generated by copying the state when an entry function gets called. A typical entry function is like the **main** function in a bare-metal system or the first task entry for an RTOS system.

Generating API models is based on dynamic Taint Analysis (TA) for both explicit and implicit data transfers. Explicit Taint Analysis (ETA) mainly focuses on taints that happen inside a task context, i.e., loading from an MMIO data register in the context of a task. If a function that gets called by the task has received data from the target peripheral’s data channel, FIRMMOD monitors such direct tainting behavior and keeps tracking of the dataflow. ETA can be conducted without triggering any interrupts if the interrupt version of HAL APIs are not used. The main difference between ETA and Implicit Taint Analysis (ITA) is that ITA requires triggering of the interrupts and monitoring of the dataflows between the tasks and the interrupt handlers.

ITA relies on interrupt modeling to keep track of tainting. For complex devices, the interrupt handlers are complex and achieving high code coverage within such interrupt handlers requires triggering of the interrupt line at the

right time. For example, in Listing 1.1, step 2.b must happen after the RxISR callback is set in step 2.a at line 15 within the task. Even if the right callback is set before the interrupt is triggered, if some other non-symbolic conditions (e.g. constant settings like the peripheral’s mode) are not met, observing the expected tainting (2.c, at line 32) may still not be possible due to low code coverage within the interrupt handler.

```

1 int16_t SPI_WIFI_ReceiveData ( uint8_t *pdata, uint16_t len,
  uint32_t timeout){
2   ...
3   if( HAL_SPI_Receive_IT(&hspi, tmp, 1) != HAL_OK) {... return
  ES_WIFI_ERROR_SPI_FAILED; }
4   wait_spi_rx_event(timeout); // interrupt triggering point
5   pData[0] = tmp[0]; pData[1] = tmp[1]; ...}

```

Listing 1.2. An example function that waits on an event.

Different from ETA, we perform ITA at specific code locations. There are two aspects to know about testing the target peripheral for an ITA process. First, we need to know the specific code location to trigger the interrupt, e.g. `wait_spi_rx_event` as shown in Listing 1.2, which indicates the need for triggering the interrupt line to complete the operation. Second, we need to know which interrupt line should be triggered. This information can be extracted from the interrupt definition documents, e.g. 51 for SPI3 of STM32L475. These two types of information enable us to generate taint propagation models of the target peripheral in a precise way.

ITA aims to generate function models for contexts in which some waiting event is issued. ITA process works as follows. Whenever a wait-event function gets called, we trigger the corresponding interrupt line. Next, we start a new symbolic execution phase to explore the state space of the corresponding interrupt handler. When the interrupt handler returns, we hook the state and switch to another one from the state pool that executes the interrupt handler. After all paths are explored inside the interrupt handler, we terminate the relevant symbolic execution phase, switch back to the original one, and pick one state to continue with. We select the state by preferring those that have read from the TDC and wrote the taints to memory locations that are not on the `IRQ_Handler`’s stack. If there are multiple candidate states to consider, we choose the one from the set that has the largest tainted memory range. This ensures that after exiting from the interrupt handler, the chosen state will be able to propagate the taint information to other parts of the memory within the task.

```

1 static void ParseIP(char* ptr, uint8_t* arr){
2     uint8_t hexnum = 0, hexcnt;
3     while(* ptr) {
4         hexcnt = 1;
5         if(*ptr != '.') {
6             // out of bounds access into arr
7             arr[hexnum++] = ParseNumber(ptr, &hexcnt);}
8         ptr = ptr + hexcnt; ...}
9 uint32_t SOCKETS_GetHostByName( const char * pcHostName ) {
10     uint32_t ulIPAddress = 0; // overflowed local variable ...
11     ES_WIFI_DNS_LookUp( &( xWiFiModule.xWifiObject ), pcHostName, (
12         uint8_t * ) &( ulIPAddress ); ...}
12 ES_WIFI_Status_t ES_WIFI_DNS_LookUp(ES_WIFIObject_t *Obj, const char *
13     url, uint8_t *ipaddress) {
14     AT_ExecuteCommand(Obj, Obj->CmdData, Obj->CmdData);...
15     ptr = strtok((char *)Obj->CmdData + 2, "\r");...
16     ParseIP(ptr, ipaddress);...
17 }

```

Listing 1.3. A buffer overflow example.

Another important component is the Function Explorer (FE), which aims to reduce the path explosion inside functions that process raw data received from the peripherals, e.g., for sanitizing and/or for parsing. Figure 2 shows an example, the `RunJobsDemo` from AWS RTOS [1], where the firmware tries to establish an MQTT connection by receiving data from the SPI bus to establish a WIFI connection. An example function that parses raw data received from a peripheral is the `ParseIP` function shown in Listing 1.3, which parses SPI data. `ParseIP` parses the IP information from raw data pointed by `ptr` and stores the parsed IP address in memory location pointed by `arr`. A failure in the parsing of the IP will cause a failure in establishing a secure connection. However, since the raw data consists of symbolic data, path explosion happens within the `parseIP` function. Similar to the string processing functions, this type of customized parsing library limits the performance of symbolic execution by not exposing flow of the tainted data moving through the upper layers. The goal of FE is to model such functions in a more precise way.

Besides ETA, ITA and FE, we also have other settings. A loop bound manager is used for managing unbounded forking. Furthermore, we have ported some function models from the new S2E for handling the string and memory related functions, e.g. `strlen`. These function models help reduce the size of the state space.

Taint Analysis. An important part of modeling a driver/HAL API is to figure out which part of the memory is modified by the function, especially those memory locations that have been tainted by the peripheral data channel. Most HAL/-driver APIs play the roles of transferring data from the peripheral to the upper layers. Precisely locating the tainted memory regions is essential for precise modeling of an API. Every function may have local variables that are indirectly changed by their callees, or even by the callee’s callees. So, we need to identify all the memory locations that have been tainted by the peripheral data channels. Local memory objects are usually tainted by passing the address as a parameter and by getting overwritten by the lowest level functions, i.e. functions that per-

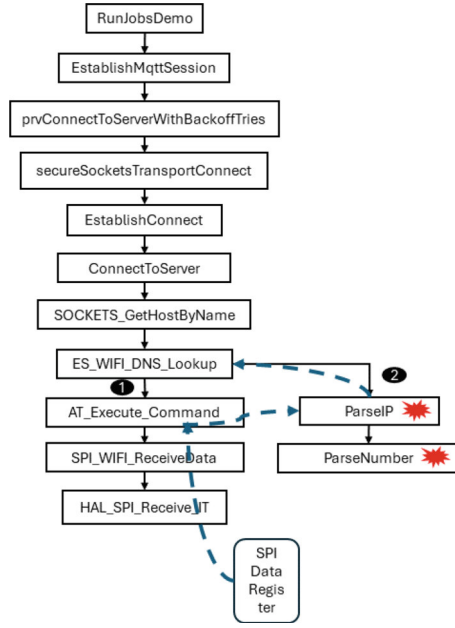


Fig. 2. Peripheral’s raw data processing example from AWS RTOS demo on running jobs [1].

form I/O operations. These functions access data registers and store the received data to the memory objects from the upper stack or a global object. In dynamic analysis, after finding the memory locations that are directly tainted, we need to propagate the information of the tainted memory locations to the upper layers until the analysis identifies the functions whose stack frames host the tainted memory locations. For global objects that get tainted, we need to identify their precise locations.

Algorithm 1 shows how FIRMMOD generates a taint memory map for each function, which summarizes the side effect of the modeled function as a set of memory locations that get tainted by the data channels. The first input of this algorithm, *DTM*, stores the taint records of memory locations that get overwritten by values flowing from the data channels, which get collected by a combination of ETA and ITA. For ETA, it is the first location that writes symbols from the data channel to the memory, i.e., load a symbol from the data channel and store it into the memory address, e.g., line 4 in Listing 1.1. For ITA, since we trigger the interrupt line at specific points and select the target path from the irq handler that have tainted the memory, the memory gets overwritten by the symbols from the target data channel, e.g. line 5 in Listing 1.2. The second input, *RTTM*, stores the modeling related taint memory map, which will be updated by this algorithm. The third input, *CC*, stores the call chain, where the

Algorithm 1. An algorithm for generating the tainted memory map.

```

1: Input ( $DTM$ : Direct Taint Map,  $RTMM$ : Model-related Taint Memory Map,  $CC$ :
   Call chain)
2: Output ( $RTMM$ : Updated Model-related Taint Memory Map)
3:  $DTM, TM : Function \mapsto \mathcal{P}(Memory)$  ▷ Both ETA and ITA
4:  $RTMM : Function \mapsto \mathcal{P}(Memory)$  ▷ Side effects of functions
5:  $TM, RTMM \leftarrow \lambda F. \emptyset$ 
6: for  $F$  in  $Functions(CC)$  do
7:    $TM \leftarrow propagateToCallers(F, DTM[F], TM, CC)$ 
8:   ▷ Propagate the memory addresses bottom-up
9: end for
10: for  $F$  in  $TM.domain$  do
11:   for  $memory$  in  $TM[F]$  do
12:     if  $memory.type \in \{Global, UpperStack\}$  then
13:        $RTTM[F].insert(memory)$ 
14:     end if
15:   end for
16: end for
17: return  $RTMM$ 

```

chain ends with a function that taints some memory addresses with data flowing from a target data channel.

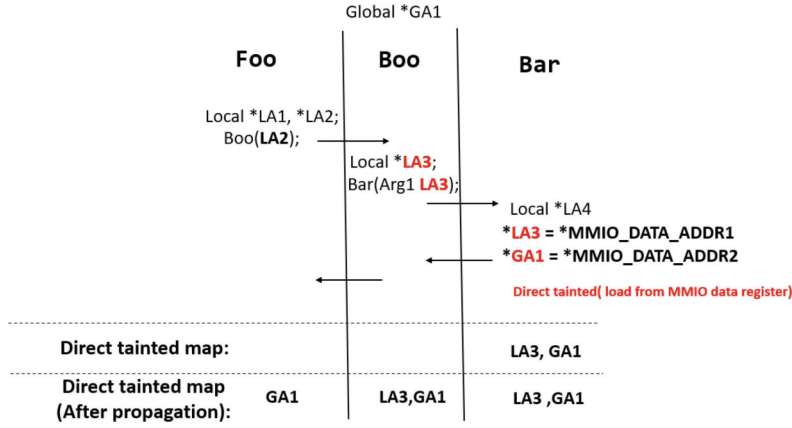


Fig. 3. Propagate the tainted address to upper layers.

The output of Algorithm 1 is a complete taint memory map for each function in the call chain. It provides us with the flexibility of modeling the functions in the call chain that do not modify the tainted memory regions themselves yet their callees do.

Algorithm 2. An algorithm for propagating the taint information to the callers, `propagateCallers`.

```

1: Input( $F$ : Function,  $MAS$ : Memory Address Set,  $TM : Function \mapsto \mathcal{P}(Memory)$ ,
    $CC$ : Call chain)
2: Output ( $TM$ : Taint Map)
3: for address  $a$  in  $MAS$  do
4:   if  $a$  is on  $F$ 's stack frame then
5:     continue
6:   end if
7:    $CF = F.getCaller(CC)$ 
8:   while  $CF$  do
9:     if  $a$  is global then
10:       $TM[CF].insert(a)$ 
11:     else if  $a$  is local then
12:       $TM[CF].insert(a)$ 
13:      if  $a$  is on caller stack then
14:        break  $\triangleright$  stop propagating if the target function frame is reached
15:      end if
16:     end if
17:      $CF = F.getCaller(CC)$ 
18:   end while
19: end for
20: return  $TM$ 

```

The algorithm calls *propagateToCallers*, Algorithm 2, to pass the tainted memory records from the callees to their callers. An example is provided in Fig. 3. The call chain `Foo`, `Boo`, `Bar` ends with `Bar`, which taints local address `LA3` and global address `GA1` directly with target data channel related addresses, `MMIO_DATA_ADDR1` and `MMIO_DATA_ADDR2`, respectively. Algorithm 2 goes through each function in the call chain to propagate their direct taints. So, when it is handling `Bar`, it needs to propagate the taint related memory accesses, `LA3` and `GA1`, to `Boo` and `Foo`. However, for `LA3` the propagation ends at `Boo` as `LA3` is located in the stack frame of `Boo` while propagation of the global address `GA1` is applied to each caller in the call chain, e.g., `Boo` and `Foo`. Normally, there are three types of tainted memory objects for modeling a specific function: local variables, global variables, and variables from the upper stack. For example, `LA3` is from the upper function's frame with respect to the `Bar` function, but it is a local value to the `Boo` function. Algorithm 1 keeps the global addresses and the upper stack addresses in the model (lines 12–13) while ignoring the local variables, which is due to the fact that when a function gets modeled the local variables will not be relevant as they will not be allocated. FIRMMOD can choose to model any of the functions in the call chain, e.g., `Foo`, `Boo`, or `Bar`, as long as we have an entry for these functions in the tainted memory map. When FIRMMOD models a function, it will be able to reflect the function's side effect in terms of propagating taints related to the target data channel.

Function Explorer. FIRMMOD also keeps track of implicit taint propagation through function calls that copy data from tainted buffers to some new buffer. We call such functions Potentially Taint Propagating (PTP) functions. FIRMMOD considers functions that receive arguments that point to tainted buffers as PTPs, and observes them to record the taint propagation, if any.

To ensure that PTP functions receive sufficient coverage, we port the function models of the string library from S2E 2.0 [6] and implement a new analysis module called the Function Explorer, which is designed to find a taint propagation relevant path in a PTP function. The Function Explorer gets activated once a PTP function is reached. As an example, assuming `ES_WIFI_DNS_Lookup` in Fig. 2 is the target function, `ParseIP` can be considered a PTP function due to its `ptr` argument being tainted. It will pause the current executor and start a new one. The new executor will explore the state space of the PTP function starting from the current state, which is similar to the symbolic exploration in ITA. After all the paths of a target function are explored, we select one path as the base state to generate the function model for that function. We formulated a taint related metric to evaluate all the states and select one that yields the highest score in terms of the taint metric. Specifically, the taint metric is the number of tainted bytes propagated within the PTP function. Once the taint model is generated for the PTP function, we restore to the previous executor, pick up one of the backup states forked at the time of invoking the PTP function call, and apply the function model immediately at the PTP callsite.

```

1 // the Prompt array is tainted, but it does not escape the stack frame
2
3 int8_t SPI_WIFI_ResetModule(void){
4     uint8_t Prompt[6]; uint8_t count = 0; ...
5     while (WIFI_IS_CMDDATA_READY()) { // forking...
6         Status = HAL_SPI_Receive(&hspi, &Prompt[count], 1, 0xFFFF);
7         count += 2; ...
8     } ...
9     if((Prompt[0] != 0x15) || (Prompt[1] != 0x15) || ...)
10         return -1;
11     return 0;
12 }

```

Listing 1.4. An example for tainted memory not escaping the stack frame.

Model Generation and Argument Analysis. FIRMMOD can be configured to target specific layers for modeling. We leverage debug information to identify such layers. For example, if a compile unit’s path is suffixed with `hal_spi.c`, the functions of this compile unit are associated with the HAL SPI layer.

We define a modeling boundary so that all the functions above the boundary are not allowed to be modeled. The boundary is used to limit the modeling of functions constrained in the set of peripheral related APIs, e.g. HAL APIs, BSP drivers or I/O helpers. Functions that only taint its local variables is modeled by returning a symbolic value only and without any taint propagation model as a side effect. An example is given in Listing 1.4, where `SPI_WIFI_ResetModule`

allocates a local array `Prompt`, which is used as a buffer to receive data from the WIFI module on the SPI bus. After the `Prompt` buffer is filled, it continues to check each byte and return. Since `Prompt` contains symbolic data from the peripheral, the byte-by-byte checking before returning could fork a lot of states, which would lower the chance for the success path that returns 0 (line 10 in listing 1.4) to be scheduled among all these forked states. If the data flow from the peripheral is detected and the tainted memory does not escape the activation frame of a function that could be modeled, it is safe to model that function with a bypassing model, i.e., no taint related side effects and a symbolic return value, if any. Even though the `Prompt` buffer is tainted with the data register, it has not escaped the stack frame of the `SPI_WIFI_ResetModule` function. So, FIRMMOD models `SPI_WIFI_ResetModule` safely by just symbolizing the return value and without noting any side effects.

FIRMMOD performs Argument Analysis to enrich the function model by capturing the taint propagation through the arguments. Considering the taint information of a function that has a set of taint memory objects from its upper callers' stack or global variables, we need to know the relation between these memory objects and arguments to generate the function model. Based on our observation most STM32 HAL/driver functions that transmit/receive massive data from/to peripherals on I2C or SPI, typically have the following types of parameters: a handler pointer parameter, where the handler abstracts the usage of the peripheral, a data buffer pointer parameter, a size parameter that indicates the length of data to be transmitted or received, and an optional timeout value. So we make the following assumptions for Argument Analysis:

- The tainted memory must be accessible through a pointer that falls into one of the argument range of registers according to the ARM calling conventions.
- One of the arguments optionally indicates the size of the buffer and the value is a configured size value, e.g. an `int` value.
- Only one of the pointer arguments gets tainted for each function.

One of the challenges in model generation is determining the size constraint for the tainted memory region so that taint propagation can be performed in a precise way. Although most driver I/O functions have a parameter to indicate the size of bytes to be transmitted or received, there are some functions that do not have an explicit size argument. To get the most accurate approximation of the size to be written, we leverage debug information to infer the accurate size information. FIRMMOD can also infer the size of the field of a `struct` type using the debug information.

```
// ptr (taint source) and arr (taint dest)
// arr points to a buffer with implicit size
void ParseIP(char* ptr, uint8_t* arr);

// invoke ParseIP model, which taints 4 bytes
// pointed by the 2nd parameter
ParseIP(ptr, NetSettings->IP_Addr);
// uint8_t IP_Addr[4];
```

As the above example shows, without the debug information the model of `ParseIP` would not specify the number of bytes pointed by the second parameter `arr` gets tainted. However, at the time of applying the model of `ParseIP`, FIRMMOD has the concrete value of `arr`, i.e., the value of `NetSettings->IP_Addr`. By querying the debug info, it resolves the address to the field of `NetSettings->IP_Addr` and finds out its type, which is a four-byte array, and the model writes four bytes of symbolic values to the address stored in `arr`. This allows FIRMMOD to propagate taints in a precise way.

FIRMMOS generates function taint models in the format shown below. For each function model, it has the return information and a target taint channel in the form of an argument index to reveal which argument contains the address to be written with the tainted data. Also, it provides the size information by either using an argument index or with an unknown mark to indicate that it would be resolved later at the time of model application.

In the model for the `ParseIP` function, the fields `has_return_value`, `target_taint_channel`, and `size` would be set to `false`, `0`, `1`, and `UNKNOWN`, respectively.

```
Function Model <function name>:
  has_return_value: true/false
  target_taint_channel: <argument index>
  size: UNKNOWN | <argument index>
```

Return-Value Guided Slicing. Another way FIRMMOD deals with the path explosion problem is by applying return value based execution slicing, which eliminates the paths that are of less interest. Given a function $\mathcal{F}$, its return value $\mathcal{R}$, a guided return value $\mathcal{A}$, and the current path constraint $\mathcal{C}$ of state $\mathcal{S}$, we apply the following rule:

$$\text{Terminate state } \mathcal{S} \text{ if } \mathcal{C} \wedge (\mathcal{F}_{\mathcal{R}} = \mathcal{A}) \equiv \text{false}$$

This rule indicates the termination of state $\mathcal{S}$ if the path constraint does not satisfy the guided return value. FIRMMOD can be configured with the guided return values of the APIs of interest. Since most APIs return a 0 value for success, it is straightforward to configure FIRMMOD for success return values. We used this guideline for various layers, which is explained in Sect. 5.

5 Evaluation

We implemented FIRMMOD on top of S2E to analyze ARM cortex M3 firmware and to perform binary level symbolic execution. We applied FIRMMOD to AWS RTOS Demo examples, which utilize the WIFI module and consist of 11 firmware binaries generated to run on the STM32L475 discovery board. Figure 4 illustrates the layers of the analyzed firmware. Main DEMO APP represents the application layer. The CoreMQTT/CoreHTTP layer implements the communication

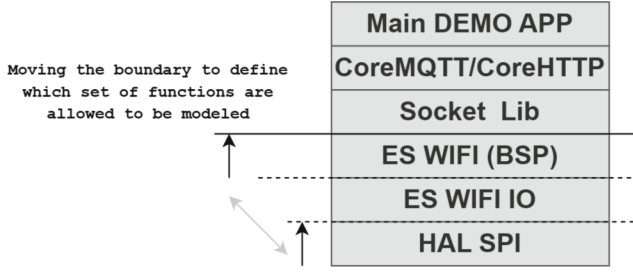


Fig. 4. Layered architecture of the Amazon FreeRTOS Demos.

protocols and interacts with the WIFI module. The Socket Library is being used as an intermediate layer for data transmission.

ES WIFI is a BSP driver and ES WIFI IO is the glue layer defined by the clients that utilize the Hardware Abstraction Layer (HAL) for the SPI bus. ES WIFI leverages two peripherals to send/receive data: SPI and GPIO. It utilizes the GPIO as a signal to indicate whether the data is ready or not. The data received from the network is transmitted over the SPI bus. The ES WIFI layer implements an interface of the AT commands to achieve various communication goals.

We treated the data received from the RNG module as concrete random values based on a timing seed. All other read accesses to MMIO regions return unconstrained symbolic values. We extracted the debug information in DWARF format from the unstripped binaries. Our results have been collected by running FIRMMOD for two hours for each setup on each benchmark on an Ubuntu 16.04 machine with 256GB of RAM.

5.1 RQ1 on the Scope of API Modeling

FIRMMOD leverages two types of API modeling that can be configured for scope. The first one is API taint modeling that it generates automatically. The second one is the API return value modeling that allows to constrain the return values of modeled APIs to success cases. However, the latter requires prior knowledge of the return values that denote success cases. Table 1 shows a summary of the success case return values for various layers we considered for API return value modeling. Our analysis reveals that most of the layers except the Socket layer use an enumerated type for the return value type. Once the success case is determined from the enumerated type, it is straightforward to generate success case API return value models using the debug information in the binary. We only needed to use manual effort for the Socket layer, which does not use an enumerated type for the return values of the API. It should be noted that return value modeling is a one-time effort, which gets reused for all firmware that calls APIs of the modeled library. Table 1 also shows that in most of the cases the success case is represented with a zero return value.

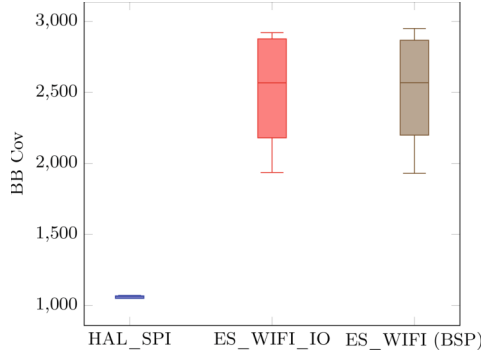
Therefore, we answer RQ1 both on the scope of automated modeling and on the scope of return value modeling. For API taint model generation, we refer the

Table 1. The distribution of zero and nonzero values for representing success cases of API return values in various layers.

Layer	Total # API	# Zero	# NonZero	Manual?
HAL	20	20	0	No
ES WIFI	43	43	0	No
Socket	12	10	2	Yes
MQTT	84	84	0	No
HTTP	18	18	0	No

reader to Fig. 4 for the different scopes we have considered. The three automated modeling scopes we used in our evaluation are defined in terms of the highest (boundary) layer that can be modeled: HAL SPI (only functions of the HAL SPI layer), ES WIFI IO (functions of the ES WIFI IO and HAL SPI layers), and ES WIFI BSP (functions of the ES WIFI BSP, ES WIFI IO, and HAL SPI layers). For return value modeling, we follow a similar approach, but we define different combinations: HAL SPI (only functions of the HAL SPI layer), ES WIFI (functions of the ES WIFI and HAL SPI layers), and HTTP/MQTT (functions of the HTTP/MQTT, Socket, ES WIFI, and HAL SPI layers) for modeling the success case return values of the APIs of the considered layers so that return value based slicing can filter out states with failure cases of such APIs.

Figure 5 shows the basic block coverage for different scopes used for API taint model generation when return value modeling boundary is set to the ES WIFI layer. Modeling only the functions at the HAL SPI achieves the smallest coverage while setting the boundary layer as ES WIFI IO or ES WIFI BSP achieves higher coverage, where moving the boundary to ES WIFI BSP achieves only a slightly higher coverage. So, the higher the boundary among these three layers the higher the basic block coverage gets achieved for the analyzed firmware. It also shows the effectiveness of FIRMMOD’s automatic generation of API taint models.

**Fig. 5.** Basic block coverage (BB cov) for various settings of the boundary layer for API taint modeling.

As mentioned in Sect. 4, FIRMMOD keeps track of memory accesses to determine which memory objects get tainted by the TDC and whether the object is global or local. Also, it monitors flow of tainted data at the time of function calls and returns. We use $\square$, $\blacktriangle$, and $\diamond$ to denote the tainted global objects, the local objects, and arguments or return values, respectively. A summary of the taint types observed is provided in Table 2, which shows the importance of argument modeling and identification of local buffers.

Table 2. Types of taints per layer.

	HAL SPI	ES WIFI IO	ES WIFI	COREMQTT	COREHTTP
global $\square$	2	1	6	3	1
local $\blacktriangle$	4	2	5	16	1
args or ret $\diamond$	6	2	20	16	1

Figure 6 shows the number functions covered at the HTTP/MQTT layer for different scopes of return value modeling. We fixed the API taint model boundary at the ES WIFI IO layer. As the figure shows, modeling of ES WIFI API return values help achieve better coverage compared to not modeling those. However, for *Jobs* and *Core Http Mutual Auth.* demos, modeling the return values of functions at the HTTP/MQTT layer reduces the total coverage due to a path explosion limiting the coverage. So, we think that ES WIFI is the ideal boundary setting for return value modeling for our benchmarks.

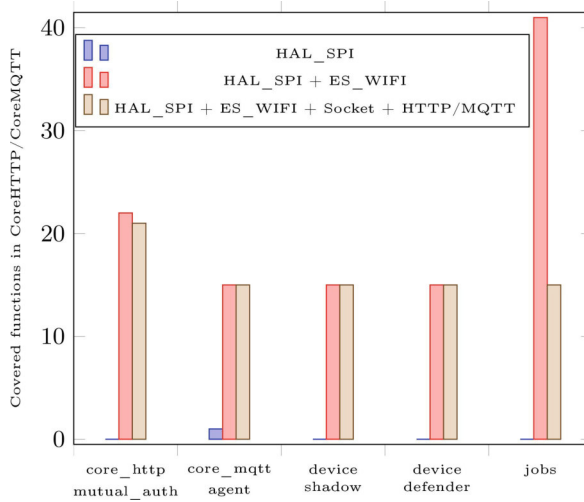


Fig. 6. The impact of the scope of return value modeling.

Table 3. Function reachability for demo application and AWS RTOS demo CoreMQTT/CoreHTTP libraries

Demo Name	Protocol	Demo Entry Function	Reach Demo Entry	Main DEMO App		CoreMQTT/CoreHTTP	
				Reached Functions	Total Functions	Reached Functions	Total Functions
core_http_mutual_auth	CoreHTTP	RunCoreHttpMutualAuthDemo	✓	3	3	22	33
core_http_s3_download	CoreHTTP	RunCoreHttpS3DownloadDemo	✓	2	4	3	45
core_http_s3_download_multithreaded	CoreHTTP	RunCoreHttpS3DownloadMultithreadedDemo	✓	2	7	3	45
core_http_s3_upload	CoreHTTP	RunCoreHttpS3UploadDemo	✓	2	5	3	45
core_mqtt_mutual_auth	CoreMQTT	RunCoreMqttMutualAuthDemo	✗	0	13	0	82
core_mqtt_agent	CoreMQTT	RunCoreMqttAgentDemo	✓	9	12	15	86
device_shadow	CoreMQTT	RunDeviceShadowDemo	✓	1	7	15	82
device_defender	CoreMQTT	RunDeviceDefenderDemo	✓	1	15	15	82
jobs	CoreMQTT	RunJobsDemo	✓	2	6	41	82
greengrass_discovery	CoreMQTT	vStartGreenGrassDiscoveryTask	✗	0	8	0	0
tcp_echo_client	–	vStartTCPEchoClientTasks_SingleTasks	✓	3	3	–	–

Finally, Table 3 shows the coverage of functions reached in the upper layers using the best combination of the API taint model boundary and the return value modeling boundary for each benchmark. For nine out of eleven benchmarks, analysis reaches some functions at the application layer and at the communication protocol (MQTT/HTTP) layers. Regarding the application layer, two of the benchmarks, HTTP Mutual Authentication and TCP Echo Client, even cover all of the included functions. Results of CoreMQTT/CoreHTTP also reveal good coverage as high as 22/33 (66%) in CoreHTTP and 41/82 (50%) in CoreMQTT.

5.2 RQ2: Bug Finding

Our framework is based on symbolic execution of binaries. So, the main mechanism we use for detecting vulnerabilities is observing abnormal cases in the symbolic execution of firmware such as the Program Counter (PC) being overwritten with an address that refers to a non-executable region. Also, if the execution state is trying to access an address defined in terms of an unconstrained symbolic value received from a target data channel (TDC), it is also identified as a potential vulnerability. FIRMMOD has detected two known buffer overflow bugs, one within the `SPI_WIFI_ResetModule` function [2] (within 200 secs) and another one within the `ParseIP` function [3] (within 2500 secs). As Listing 1.4 shows, a local variable `count` will be overwritten with a TDC value if the check on line 4 keeps holding true. Then if the tainted value of `count` is used as an index to the `Prompt` array, which is used as the receiving buffer base address passed to `HAL_SPI_Receive` (line 5), which leads to a memory out of bounds access. FIRMMOD is able to detect such vulnerability based on the defined detection pattern. Similarly, in Listing 1.3, the second argument of `ParseIP`, `arr`, can be filled with symbolic values if the TDC value that gets checked on line 3 keeps holding true. As we observed, the value of TDC at `ptr` can be larger than 1000, but the expected parsed IP address and the size of the buffer pointed by `arr` is only four-bytes. As we show in Sect. 5.3, the state-of-the-art firmware rehosting approach, Fuzzware [13], cannot detect these bugs within 24 h.

5.3 RQ3: FIRMMOD Versus State-of-the-Art

In this section, we first compare FIRMMOD against FIRMSTAT [4], which uses static analysis to generate API taint models. FIRMMOD, on the other hand, uses dynamic symbolic execution to achieve the same goal. Table 4 shows a comparison of FIRMSTAT and FIRMMOD in terms of the number of API taint models generated for various layers. FIRMSTAT is limited to below the MQTT/HTTP layers, possibly because it relies on the HAL data types for inferring taint propagation and the type-based resolution of function pointers.

The reason FIRMSTAT generates models of more functions in some layers is that its scope is independent of what is actually called within the firmware. The reason FIRMMOD generates models for some functions, for which FIRMSTAT

Table 4. Number of functions detected to be tainted

	HAL_SPI	ES_WIFI	ES_WIFI_IO	CoreMQTT
FIRMSTAT	10	17	1	–
FIRMMOD	7	20	3	18

fails is due to FIRMMOD’s ability to provide more precise function pointer resolution as a dynamic analysis approach. Another important advantage of FIRMMOD over FIRMSTAT is that the API taint models in FIRMMOD includes size information for the tainted buffers which get populated with symbolic data lazily to precisely observe the side effect of data-flows from peripherals within the upper layers.

Table 5. Basic Block Coverage for FIRMMOD and Fuzzware

Firmware	FIRMMOD	Fuzzware
core_http_mutual_auth	3114	1426
core_http_s3_download	2609	1390
core_http_s3_download_multithreaded	2546	1428
core_http_s3_upload	2579	1390
core_mqtt_mutual_auth	1963	1419
core_mqtt_agent	3063	1408
device_shadow	2994	1427
device_defender	3006	1426
jobs	3387	1417
greengrass_discovery	1958	1429
tcp_echo_client	2715	1432

We also compared FIRMMOD against Fuzzware [13], a state-of-the-art firmware rehosting approach that combines fuzzing and symbolic execution. We ran Fuzzware for 24 h, which is much longer than the timeout of two hours that was used for FIRMMOD. Table 5 shows the number of basic blocks covered by each tool. As the table shows Fuzzware achieves limited coverage for each benchmark mostly performing initialization steps of the firmware while FIRMMOD achieves up to 2X more coverage. Also, Fuzzware was not able to find the two bugs we found in upper layers showing its inability to achieve sufficient coverage in the upper layers. These show the importance of modeling in firmware analysis. We think that combining the peripheral interaction model generation performed by tools like Fuzzware with the API taint models and return value based slicing performed by FIRMMOD can potentially achieve even better performance, which we plan to pursue as future work.

6 Conclusions

We presented an automated approach, FIRMMOD, for generating taint models of APIs that get employed by various layers of firmware and directly or indirectly interact with some peripherals. FIRMMOD applies the learnt API models to reach deeper parts of the firmware compared to state-of-the-art firmware rehosting approaches that do not leverage API models. This enabled FIRMMOD to detect two buffer overflow vulnerabilities that are missed by standard firmware rehosting approaches that focus on modeling peripheral interaction only. In future work, we will integrate FIRMMOD with fuzzing to achieve goal-based fuzzing of firmware.

Acknowledgments. This work was funded by United States National Science Foundation Award # 1942235.

References

1. <https://github.com/aws/amazon-freertos>
2. <https://github.com/STMicroelectronics/STM32CubeH7/issues/252>
3. <https://tinyurl.com/ms8hy7jm>
4. Bai, K.Y., Yavuz, T.: MMIO access-based coverage for firmware analysis. In: IEEE Conference on Communications and Network Security, CNS 2023, Orlando, FL, USA, 2–5 October 2023, pp. 1–9. IEEE (2023)
5. Cao, C., Guan, L., Ming, J., Liu, P.: Device-agnostic firmware execution is possible: a concolic execution approach for peripheral emulation. In: Annual Computer Security Applications Conference, ACSAC '20, pp. 746–759. Association for Computing Machinery, New York (2020)
6. Chipounov, V., Kuznetsov, V., Candea, G.: S2e: a platform for in-vivo multi-path analysis of software systems. In: Gupta, R., Mowry, T.C. (eds.) ASPLOS, pp. 265–278. ACM (2011)
7. Clements, A.A., et al.: Halucinator: firmware re-hosting through abstraction layer emulation. In: Capkun, S., Roesner, F. (eds.) 29th USENIX Security Symposium, USENIX Security 2020, 12–14 August 2020, pp. 1201–1218. USENIX Association (2020). <https://www.usenix.org/conference/usenixsecurity20/presentation/clements>
8. Feng, B., Mera, A., Lu, L.: P2IM: scalable and hardware-independent firmware testing via automatic peripheral interface modeling. In: Capkun, S., Roesner, F. (eds.) 29th USENIX Security Symposium, USENIX Security 2020, 12–14 August 2020, pp. 1237–1254. USENIX Association (2020). <https://www.usenix.org/conference/usenixsecurity20/presentation/feng>
9. Gustafson, E., et al.: Toward the analysis of embedded firmware through automated re-hosting. In: 22nd International Symposium on Research in Attacks, Intrusions and Defenses, RAID 2019, Chaoyang District, Beijing, China, 23–25 September 2019, pp. 135–150. USENIX Association (2019). <https://www.usenix.org/conference/raid2019/presentation/gustafson>
10. Johnson, E., et al.: Jetset: targeted firmware rehosting for embedded systems. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association (2021). <https://www.usenix.org/conference/usenixsecurity21/presentation/johnson>

11. Li, W., Guan, L., Lin, J., Shi, J., Li, F.: From library portability to para-rehosting: natively executing microcontroller software on commodity hardware. In: 28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, 21–25 February 2021. The Internet Society (2021)
12. Mera, A., Feng, B., Lu, L., Kirda, E.: Dice: automatic emulation of dma input channels for dynamic firmware analysis. In: 2021 IEEE Symposium on Security and Privacy (SP), pp. 1938–1954 (2021). <https://doi.org/10.1109/SP40001.2021.00018>
13. Scharnowski, T., et al.: Fuzzware: using precise MMIO modeling for effective firmware fuzzing. In: 31st USENIX Security Symposium (USENIX Security 22), pp. 1239–1256. USENIX Association, Boston (2022)
14. Zhou, W., Guan, L., Liu, P., Zhang, Y.: Automatic firmware emulation through invalidity-guided knowledge inference. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association (2021). <https://www.usenix.org/conference/usenixsecurity21/presentation/zhou>
15. Zhou, W., Zhang, L., Guan, L., Liu, P., Zhang, Y.: What your firmware tells you is not how you should emulate it: a specification-guided approach for firmware emulation. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22, pp. 3269–3283. Association for Computing Machinery, New York (2022)



Large Language Models Using Retrieval Augmented Generation and Prompt Engineering for AI-Driven Music Source Separation: A Literature Review

Scott Josephson^(✉) and Atif Farid Mohammad

Capitol Technology University, Laurel, MD 20708, USA
{sjosephson, afmohammad}@captechu.edu

Abstract. Music source separation is a critical task in audio processing and is essential for applications such as remixing, audio restoration, and music analysis. Traditional methods, including signal processing techniques like Independent Component Analysis (ICA) and Non-negative Matrix Factorization (NMF), as well as deep learning approaches utilizing Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), face challenges related to extensive data requirements, computational complexity, and limited adaptability to diverse audio contexts. Recent advancements in Large Language Models (LLMs) like GPT-4 introduce new possibilities for addressing these challenges without retraining models. This literature review explores the integration of prompt engineering and Retrieval-Augmented Generation (RAG) with LLMs to perform music source separation. By leveraging LLMs' inherent language understanding capabilities and accessing external databases containing musical scores, lyrics, and instrument profiles, this approach aims to enhance efficiency, flexibility, and accessibility in source separation tasks. The review highlights how this innovative methodology can overcome the limitations of traditional techniques, filling a notable gap in current research and offering transformative potential for both academic inquiry and practical applications in music technology.

Keywords: Music Source Separation · Large Language Models · Prompt Engineering · Retrieval-Augmented Generation · Audio Embeddings · Multimodal Applications

1 Introduction

1.1 Background

Music source separation focuses on extracting instrumental or vocal tracks from a mixed audio signal. This task is essential for various applications, including remixing, audio restoration, music transcription, and analysis [1]. Traditional methods often rely on complex signal processing techniques or retrained machine learning models, which can be computationally intensive and inflexible [2]. Recent advancements in artificial

intelligence, particularly in developing Large Language Models (LLMs) like GPT-4, present new opportunities to address these challenges without retraining models [3]. This literature review examines the evolution of music source separation techniques and the limitations of current methodologies. It explores how prompt engineering and Retrieval-Augmented Generation (RAG) with LLMs can offer innovative solutions.

2 Traditional Methods in Music Source Separation

2.1 Signal Processing Techniques

Early approaches to music source separation utilized signal processing techniques such as Independent Component Analysis (ICA) and Non-negative Matrix Factorization (NMF). ICA assumes statistical independence between sources and non-Gaussian and separates signals based on maximizing this independence [4]. However, its effectiveness is limited when dealing with complex music signals where source independence is not guaranteed.

NMF decomposes a mixed signal into a set of basis spectra and corresponding activations, effectively identifying underlying patterns in the data [5]. NMF has been utilized to factorize spectrograms of audio mixtures into spectral bases and activations, effectively isolating individual sound sources based on their spectral content [6]. NMF's assumption of non-negativity aligns well with the nature of magnitude spectrograms, making it suitable for audio applications. While NMF has been successful in specific contexts, it needs to work on overlapping harmonics and requires prior knowledge about the number of sources.

Sparse Component Analysis (SCA) is another method that exploits the sparsity of source signals in a particular domain, such as the time-frequency domain [7]. SCA assumes that the sources are sparse and only one or a few are active at any given time or frequency. By transforming the mixed signals into a sparse domain, SCA can separate the sources by identifying clusters corresponding to individual sources. This technique is particularly effective when dealing with signals like speech, which exhibit sparsity in the time-frequency representation.

Beamforming techniques have also been applied to music source separation, especially in scenarios involving microphone arrays [8]. Beamforming uses spatial filtering to enhance signals from a specific direction while attenuating others. Steering the beamformer towards the desired source makes isolating it from interfering sources and noise possible. However, beamforming requires knowledge of the array geometry and may not perform well in reverberant environments where reflections can degrade spatial cues.

Despite their successes, traditional signal-processing techniques must be improved in complex auditory scenes. ICA, for instance, relies heavily on the statistical independence of sources and may fail when sources are correlated or when the number of sensors is less than the number of sources [9]. NMF, while effective, can struggle with non-stationary sources and requires careful selection of the number of components. SCA depends on the sparsity of sources, which may not hold in all cases, particularly when sources overlap significantly in time and frequency [10]. These limitations highlight the need for more robust and flexible methods to handle real-world audio mixtures' complexities.

2.2 Deep Learning Approaches

The advent of deep learning introduced models capable of learning complex representations of audio signals. Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have been applied to spectrogram representations of audio for source separation tasks [11]. U-Net architectures, adapted from image segmentation, have shown promise in isolating vocals and instruments from music tracks [12].

Despite their success, these models require large, labeled datasets for training. The MUSDB18 dataset, for example, provides multitrack recordings but is limited in size and diversity [2]. Training models on such datasets can lead to overfitting and poor generalization to unseen audio contexts.

Various architectures and techniques have been explored to improve separation quality and computational efficiency.

2.3 Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs)

Early deep learning approaches employed Convolutional Neural Networks (CNNs) to exploit local structures in spectrograms, capturing time-frequency patterns relevant to different sources [14]. Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, have been used to model temporal dependencies in audio signals, enhancing the separation of sources with temporal coherence [11]. Convolutional Neural Networks (CNNs) have been widely used for music source separation because they capture local time-frequency patterns in spectrograms. An example is a CNN-based monaural audio source separation model, demonstrating that deep CNNs can effectively learn to separate musical instruments from a single-channel mixture [13].

2.4 U-Net Architectures

Initially developed for biomedical image segmentation, U-Net architectures have been adapted for music source separation by treating spectrograms as images [12]. The U-Net's encoder-decoder structure with skip connections allows for effective learning of global and local features, facilitating the separation of vocal and instrumental components.

2.5 Wave-U-Net and Time-Domain Approaches

To address the limitations of spectrogram-based methods, [15] introduced the WaveU-Net architecture, which operates directly on raw audio waveforms. This approach avoids the need for time-frequency transformations and captures phase information more effectively, leading to improved separation quality in some cases.

2.6 Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) have been applied to source separation by framing the problem as a generative task. GANs consist of two neural networks—the generator and the discriminator—trained simultaneously through adversarial processes [16].

Application in Music Source Separation: S. Pascual introduced SEGAN (Speech Enhancement Generative Adversarial Network), which applies GANs for speech enhancement, a task closely related to source separation. While SEGAN focuses on speech, the methodology has inspired music source separation applications [17].

O. Mogren introduced C-RNN-GAN, a continuous RNN GAN for generating sequences, which can be extended to model temporal dependencies in music [18]. N. Takahashi and Y. Mitsufuji specifically addressed music source separation using GANs. They proposed the Multi-scale Multi-band DenseNet (MMDenseNet) with GANs to improve the quality of separated sources by encouraging the generator to produce outputs indistinguishable from real instrument sounds [19].

Benefits of Using GANs: GANs can produce more natural and realistic-sounding separate sources by learning the data distribution of individual instruments or vocals. The discriminator network helps the generator improve by providing feedback on the authenticity of the generated signals.

Challenges: GANs are notoriously difficult to train due to issues like mode collapse and training instability. The adversarial training process can be computationally intensive.

2.7 Deep Clustering and Spectral Masking

Deep clustering involves a network of learning embeddings for each time-frequency bin in the spectrogram, which are grouped according to the source. [20]. This approach allows the separation of overlapping sources by clustering embeddings corresponding to different instruments or vocals.

2.8 End-to-End Learning and TasNet

The TasNet architecture is an end-to-end time-domain audio separation network that learns representations of mixed signals and directly separates sources in the time domain. [21]. This method is significantly improved over spectrogram-based approaches, particularly in speech separation tasks.

2.9 Transformer-Based Models

Transformer architectures, celebrated for their effectiveness in modeling sequential data in natural language processing, have been explored for audio tasks due to their ability to model long-range dependencies. SepFormer, a transformer-based speech separation model, uses self-attention mechanisms to capture global contextual information across the audio sequence [22]. The SepFormer processes audio signals by segmenting them into chunks and applying transformer layers to model intra-chunk and inter-chunk dependencies. This dual-path approach enables the network to handle long audio sequences efficiently while maintaining the ability to model complex relationships between sound sources. While the SepFormer was initially designed for speech separation, the principles

underlying its architecture apply to music source separation. The self-attention mechanisms allow the model to focus on different audio signal parts, capturing intricate relationships between various musical instruments over time. This makes transformer-based models a promising avenue for future research in music source separation.

Transformers have continued to gain attention for music source separation. SoniDo is a generative model featuring a multi-level transformer and hierarchical encoder [23]. It supports various music-related tasks by integrating its intermediate representations into task-specific models with data augmentation. An on-the-fly data augmentation method, “token-out,” is proposed to mitigate overfitting. SoniDo’s effectiveness is demonstrated through benchmarking across diverse tasks, including music tagging, transcription, source separation, and mixing.

2.10 Advances in 2022 and 2023

In recent years, significant progress has been made in deep learning approaches for music source separation, with new models and techniques enhancing performance and efficiency. Hybrid Demucs (HDemucs) was introduced, blending time-domain and frequency-domain processing to enhance the quality of audio separation [24]. HDemucs leverages the strengths of both domains, leading to state-of-the-art performance on benchmarks like MUSDB18. In 2022, further refinements were made to Demucs, incorporating advanced training techniques and architectures that enhanced its ability to separate complex mixtures [25].

Self-supervised and unsupervised learning approaches have been explored to reduce reliance on labeled data. RemixIT is a self-supervised learning framework leveraging mixtures of mixtures, enabling models to learn effectively from unlabeled data [26].

This approach has shown promise in improving separation performance without extensive labeled datasets. Researchers have explored multi-modal approaches incorporating additional contextual information to aid source separation. For example, visual cues from music videos have been used to inform audio separation models [27].

2.11 Challenges and Limitations

Despite advancements in deep learning-based music source separation, several challenges persist, including the need for high-quality, labeled datasets, which are limited and essential for supervised learning approaches. Training and deploying advanced models also require significant computational resources, potentially limiting accessibility for some practitioners. Models may also struggle to generalize to unseen genres, instruments, or recording conditions, and standard evaluation metrics like signal-to-distortion ratio (SDR) may not fully capture perceptual quality, necessitating more comprehensive evaluation methods.

3 Challenges with Traditional Machine Learning Approaches

3.1 Data Requirements

Deep learning models necessitate extensive training data, which can be challenging to obtain due to licensing restrictions and the scarcity of isolated instrument tracks [2]. Limited datasets hinder the models' generalization to different genres and recording conditions.

3.2 Computational Complexity

Training and deploying deep neural networks for source separation is computationally intensive. High-performance hardware and prolonged training times are required, making it impractical for many users [28].

3.3 Lack of Flexibility

Models trained on specific datasets may not perform well when applied to diverse music or audio contexts. Retraining or fine-tuning is often necessary, which is resource-intensive and limits the adaptability of these models [29].

4 Large Language Models and Their Potential Application

4.1 Overview of Large Language Models

Large Language Models like GPT-4 have demonstrated remarkable capabilities in understanding and generating human-like text [3, 30]. These models leverage transformer architectures and are pre-trained on vast amounts of textual data, enabling them to perform a wide range of language tasks without task-specific training.

4.2 Extending LLMs Beyond Text

While LLMs are primarily designed for text-based applications, researchers are exploring their potential in other domains. For instance, LLMs have been used for code generation [31], and there is growing interest in applying them to multimodal tasks that involve both text and non-textual data.

LLMs have been increasingly employed in multimodal applications, where integrating linguistic and non-linguistic data modalities enhances the performance of various tasks. In computer vision, LLMs have been pivotal in tasks like image captioning, where models generate descriptive textual captions for images. Early work by O. Vinyals [32] introduced a neural image caption generator that combined a convolutional neural network (CNN) for image feature extraction with a recurrent neural network (RNN) language model for caption generation. This approach was further advanced by K. Xu [33], who incorporated attention mechanisms to allow the model to focus on specific parts of an image when generating each caption word.

These advancements illustrate the potential of LLMs to serve as a bridge between different data modalities, enhancing the performance of systems that rely on understanding and generating human-like language in conjunction with visual or auditory information. The success of LLMs in these multimodal applications motivates their exploration in other domains, such as audio signal processing, where they can provide semantic context to improve tasks like Music Source Separation.

5 Prompt Engineering

5.1 Concept and Techniques

Prompt engineering involves crafting input prompts that guide LLMs to produce desired outputs [34]. By carefully designing these prompts, users can leverage the model's pre-trained knowledge to perform specific tasks without altering the model's architecture.

5.2 Applications in LLMs

In natural language processing, prompt engineering has improved question-answering, translation, and summarization tasks [35]. For example, providing context-rich prompts can enhance the model's accuracy in generating relevant responses.

5.3 Potential in Music Source Separation

Applying prompt engineering to music source separation involves creating prompts instructing the LLM to perform audio-related tasks. Examples include:

“Analyze the provided mixed audio file and separate it into individual tracks for vocals, guitar, bass, and drums.”

“Given this audio sample, isolate the vocal components and provide a transcript of the lyrics.”

“Separate the instrumental sections from the following song, focusing on the piano and violin parts.”

These prompts utilize the LLM's language understanding capabilities to interpret and act upon audio data.

6 Retrieval-Augmented Generation (RAG)

6.1 Concept and Mechanism

Retrieval-augmented generation combines LLMs with external knowledge sources to enhance their performance on tasks requiring up-to-date or specialized information [36]. The model retrieves relevant data from external databases during generation, which informs and improves its output.

6.2 Implementation in LLMs

RAG has been successfully applied in knowledge-intensive tasks such as open-domain question answering and fact-checking [37]. The LLM can provide more accurate and contextually relevant responses by accessing external documents or databases.

6.3 Application to Music Source Separation

In the context of music source separation, RAG can enable the LLM to access: Audio Embeddings: By converting audio data into embeddings using pre-trained audio models and indexing them in a vector database, the LLM can retrieve similar audio patterns that assist in separation tasks. Lyric and Score Alignment: Accessing lyrics and musical scores databases allow the LLM to align audio signals with textual content, aiding in identifying and isolating specific vocal or instrumental tracks.

Retrieving sound profiles of different instruments helps the LLM distinguish between instruments within a mixed audio signal, improving separation accuracy. For example, when processing a prompt like “Separate the electric guitar and provide its isolated track from this song,” the LLM will generate a query by formulating a search query based on the prompt to retrieve relevant data (e.g., electric guitar sound profiles) from external databases. Use RAG to access and retrieve necessary embeddings or reference tracks. Utilize the retrieved information to guide the separation algorithm, isolating the electric guitar from the mixed audio.

7 Integration of LLMs, Prompt Engineering, and RAG in Music Source Separation

7.1 Leveraging Existing Capabilities

Combining prompt engineering and RAG allows LLMs to perform music source separation effectively without retraining. This approach capitalizes on the model’s existing language understanding and reasoning abilities, augmented by real-time data retrieval.

7.2 Advantages of Traditional Methods

Efficiency: Eliminates the need for extensive training on large datasets.

Flexibility: Can adapt to various audio contexts by accessing diverse external data.

Accessibility: Reduces computational resources required, making the technology more accessible to users without specialized hardware.

7.3 Challenges and Considerations

Data Availability: The effectiveness of RAG depends on the availability and quality of external databases. Licensing and Copyright: Ensuring compliance with intellectual property laws when accessing and using external data is crucial [38].

Model Limitations: LLMs are not inherently designed for audio processing; their ability to interpret audio data relies on how sound prompts and retrieval mechanisms are implemented.

8 Existing Research and Gaps

While there is burgeoning interest in expanding the capabilities of LLMs beyond text, research specifically focusing on using LLMs for music source separation via prompt engineering and RAG is limited. Previous studies have explored cross-modal tasks, such as generating music from text descriptions [39]. However, the direct application of LLMs in audio processing still needs to be explored. This gap presents an opportunity to investigate how LLMs can be effectively utilized in music source separation, leveraging prompt engineering and RAG to overcome the limitations of traditional methods.

9 Conclusion

The literature highlights significant challenges in traditional music source separation methods, including data requirements, computational complexity, and lack of flexibility. Large Language Models, combined with prompt engineering and Retrieval Augmented Generation, offer a promising alternative by leveraging existing capabilities without retraining. LLMs can potentially perform complex audio tasks by crafting effective prompts and integrating external data retrieval, making source separation more accessible and efficient. This innovative approach addresses a notable gap in current research and has the potential to significantly impact both academic and practical aspects of audio processing in music technology.

References

1. Vincent, E., Gribonval, R., Févotte, C.: Performance measurement in blind audio source separation. *IEEE Trans. Audio Speech Lang. Process.* **14**(4), 1462–1469 (2006)
2. Rafii, Z., Liutkus, A., Stöter, F.-R., Mimitakis, S.I., Bittner, R.: MUSDB18 - a Corpus for Music Separation (2017). f10.5281/zenodo.1117371ff. f10.5281/zenodo.1117371ff. f10.5281/zenodo.1117371ff
3. OpenAI: GPT-4 Technical Report (2023). Retrieved from <https://openai.com/research/gpt4>
4. Comon, P.: Independent component analysis, a new concept? *Signal Process.* **36**(3), 287–314 (1994)
5. Lee, D.D., Seung, H.S.: Learning the parts of objects by non-negative matrix factorization. *Nature* **401**(6755), 788–791 (1999)
6. Smaragdis, P., Brown, J.C.: Non-negative matrix factorization for polyphonic music transcription. In: *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, pp. 177–180 (2003)
7. Zibulevsky, M., Pearlmutter, B.A.: Blind source separation by sparse decomposition in a signal dictionary. *Neural Comput.* **13**(4), 863–882 (2001)
8. Van Veen, B.D., Buckley, K.M.: Beamforming: a versatile approach to spatial filtering. *IEEE ASSP Mag.* **5**(2), 4–24 (1988)
9. Cichocki, A., Amari, S.: *Adaptive Blind Signal and Image Processing: Learning Algorithms and Applications*. John Wiley & Sons (2002)
10. Virtanen, T.: Monaural sound source separation by nonnegative matrix factorization with temporal continuity and sparseness criteria. *IEEE Trans. Audio Speech Lang. Process.* **15**(3), 1066–1074 (2007)

11. Huang, P.-S., Kim, M., Hasegawa-Johnson, M., Smaragdis, P.: Joint optimization of masks and deep recurrent neural networks for monaural source separation. *IEEE/ACM Trans. Audio Speech Lang. Process.* **23**(12), 2136–2147 (2015)
12. Jansson, A., Humphrey, E.J., Montecchio, N., Bittner, R., Kumar, A., Weyde, T.: Singing voice separation with deep U-Net convolutional networks. In: 18th International Society for Music Information Retrieval Conference, pp. 745–751 (2017)
13. Nugraha, A.A., Liutkus, A., Vincent, E.: Multichannel music separation with deep neural networks. In: 2016 24th European Signal Processing Conference (EUSIPCO), pp. 1748–1752. IEEE (2016)
14. Chandna, P., Miron, M., Janer, J., Gómez, E.: Monoaural audio source separation using deep convolutional neural networks. In 2017 25th European Signal Processing Conference (EUSIPCO), pp. 291–295. IEEE (2017)
15. Stoller, D., Ewert, S., Dixon, S.: Wave-U-Net: a multi-scale neural network for end-to-end audio source separation. In: 19th International Society for Music Information Retrieval Conference, pp. 334–340 (2018)
16. Goodfellow, I., et al.: Generative adversarial nets. In: *Advances in Neural Information Processing Systems*, pp. 2672–2680 (2014)
17. Pascual, S., Bonafonte, A., Serrà, J.: SEGAN: speech enhancement generative adversarial network. *Interspeech* **2017**, 3642–3646 (2017)
18. Mogren, O.: C-RNN-GAN: Continuous Recurrent Neural Networks with Adversarial Training (2016). arXiv preprint [arXiv:1611.09904](https://arxiv.org/abs/1611.09904)
19. Takahashi, N., Mitsufuji, Y.: Multi-scale multi-band DenseNets for audio source separation. In: 2017 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA), pp. 21–25. IEEE (2017)
20. Hershey, J.R., Chen, Z., Le Roux, J., Watanabe, S.: Deep clustering: discriminative embeddings for segmentation and separation. In: 2016 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), pp. 31–35. IEEE (2016)
21. Luo, Y., Mesgarani, N.: Conv-TasNet: Surpassing ideal time–frequency magnitude masking for speech separation. *IEEE/ACM Trans. Audio Speech Lang. Process.* **27**(8), 1256–1266 (2019)
22. Subakan, C., Ravanelli, M., Cornell, S., Bronzi, M., & Zhong, J.: Attention is all you need in speech separation. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pp. 21–25. IEEE (2021)
23. Liao, W., et al.: Music Foundation Model as Generic Booster for Music Downstream Tasks (2024). arXiv preprint [arXiv:2411.01135](https://arxiv.org/abs/2411.01135)
24. Défossez, A.: Hybrid Spectrogram and Waveform Source Separation (2021). arXiv preprint [arXiv:2111.03600](https://arxiv.org/abs/2111.03600)
25. Défossez, A.: Hybrid Demucs v3: high fidelity music source separation. In: *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR 2022)* (2022)
26. Tzinis, E., Adi, Y., Ithapu, V.K., Xu, B., Smaragdis, P., Kumar, A.: Remixit: continual self-training of speech enhancement models via bootstrapped remixing. *IEEE J. Sel. Top. Signal Process.* **16**(6), 1329–1341 (2022)
27. Gan, C., Wang, Y., Torralba, A.: Look, listen, and act: towards audio-visual embodied navigation. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13582–13591 (2022)
28. Stöter, F.-R., Liutkus, A., Ito, N.: The 2018 signal separation evaluation campaign. In: *International Conference on Latent Variable Analysis and Signal Separation*, pp. 293–305 (2018). Springer

29. Uhlich, S., Giron, F., Mitsufuji, Y.: Deep neural network-based instrument extraction from music. In: 2017 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), pp. 546–550 (2017)
30. Brown, T.B., et al.: Language models are few-shot learners. *Adv. Neural. Inf. Process. Syst.* **33**, 1877–1901 (2020)
31. Chen, M., et al.: Evaluating Large Language Models Trained on Code (2021). arXiv preprint [arXiv:2107.03374](https://arxiv.org/abs/2107.03374)
32. Vinyals, O., Toshev, A., Bengio, S., Erhan, D.: Show and tell: a neural image caption generator. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 3156–3164 (2015)
33. Xu, K., et al.: Show, attend and tell: neural image caption generation with visual attention. In: Proceedings of the 32nd International Conference on Machine Learning, pp. 2048–2057 (2015)
34. Reynolds, L., McDonell, K.: Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm (2021). arXiv preprint [arXiv:2102.07350](https://arxiv.org/abs/2102.07350)
35. Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G.: Pre-train, Prompt, and Predict: a Systematic Survey of Prompting Methods in Natural Language Processing (2021). arXiv preprint [arXiv:2107.13586](https://arxiv.org/abs/2107.13586)
36. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, pp. 9459–9474 (2020)
37. Izacard, G., Grave, E.: Leveraging passage retrieval with generative models for open domain question answering. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics, pp. 874–880 (2021)
38. Floridi, L., Cowls, J.: A unified framework of five principles for AI in society. *Harvard Data Sci. Rev.* **1**(1), 5 (2019)
39. Yang, L.-C., Chou, S.-Y., Yang, Y.-H.: MidiNet: a convolutional generative adversarial network for symbolic-domain music generation. In: Proceedings of the 18th International Society for Music Information Retrieval Conference, pp. 324–331



Beyond Supervision: HyMoBY-Swin Hybrid Self-guided and Adaptive Learning Transformer for Multiclass Retinal Disease Diagnosis

Muhammad Hammad Malik¹, Ghulam Mudassir², Yingying Ren^{1,3},
and Da-Wei Ding^{1,3}(✉)

¹ School of Automation and Electrical Engineering, University of Science and Technology
Beijing, Beijing 100083, China
dingdawei@ustb.edu.cn

² School of Computing, The University of Buckingham, Buckingham MK181EG, UK

³ Key Laboratory of Knowledge Automation for Industrial Processes, Ministry of Education,
University of Science and Technology Beijing, Beijing 100083, China

Abstract. Early detection of eye diseases, such as Diabetic Retinopathy (DR), Cataract, and Glaucoma, as well as the ability to accurately differentiate these from Normal conditions, is critical for preventing irreversible vision loss and improving patient outcomes. However, accurate and automated classification from fundus images remains a challenge due to the scarcity of labeled data and the limitations of conventional supervised learning. To address this, we propose a novel hybrid learning framework that integrates Momentum Bootstrap Your Own Latent (MoBY) with a Swin Transformer backbone for precise and robust multi-class eye disease classification. The proposed model first undergoes self-supervised pretraining to extract discriminative features from unlabeled images, followed by supervised fine-tuning on limited labeled data for task-specific adaptation. HyMoBY-Swin employs the Swin Transformer backbone for deep feature extraction. These features are refined through contrastive learning by maximizing similarity between augmented views of the same image while minimizing overlap with unrelated samples. Supervised fine-tuning further enhances generalization across both labeled and unlabeled data. Extensive experiments on the OIH dataset, comprising 4,215 fundus images, demonstrate that HyMoBY-Swin achieves a robust test accuracy of 92.43%, with consistently strong performance across all four categories. Diabetic retinopathy demonstrated the highest precision and sensitivity, both at 0.98, along with a leading ROC-AUC score of 0.99, followed by cataract (0.96), glaucoma (0.93), and normal cases (0.92). Ablation studies reveal that HyMoBY-Swin significantly outperforms its individual components and other state-of-the-art baselines, demonstrating the effectiveness of the combined contrastive learning and transformer-based architecture. Furthermore, t-SNE visualizations confirm the model's ability to clearly differentiate between disease categories, highlighting its strong generalization capacity across complex clinical scenarios. These results establish HyMoBY-Swin as an accurate and generalizable solution for real-world ophthalmic settings, offering significant clinical utility by reducing reliance on large annotated datasets and enhancing diagnostic efficiency.

Keywords: Fundus Image Classification · Hybrid Learning · Self-Supervised Learning · Supervised Learning · Contrastive Learning · HyMoBY-Swin

1 Introduction

Visual impairment worldwide stems from retinal diseases which include diabetic retinopathy (DR), glaucoma, and cataracts, since these conditions affect more than 450 million people and boost global blindness statistics [1]. Timely medical care becomes crucial for these diseases to stop vision loss in its a permanent state. Fundus photography has become the main non-invasive instrument for retinal condition diagnosis and screening since it provides detailed pictures of vascular structures along with the optic disc and macula [2]. Machine-based fundus image analysis keeps encountering obstacles for accurate disease classification because of changes in image capture conditions along with individual patient differences and overlapping disease indicators [3].

Convolutional Neural Networks (CNNs) and other deep learning approaches demonstrate good performance levels for diagnosing retinal diseases. The reliance of these models upon extensive annotated data sets proves challenging because expert annotation requirements make them difficult to acquire [4]. A lack of sufficient medical dataset labeling stands as the main obstacle to deep learning model deployment in real-world clinical applications, especially in areas which have few ophthalmic specialists to work. The restricted availability of medical data requires new scalable learning systems to achieve population generalization together with high accuracy results.

The classification of fundus images relies mainly on fully supervised deep learning models that need extensive expert-labeled datasets for operation. The notable success of CNN architectures like ResNet, EfficientNet and DenseNet exists despite their inability to work effectively in limited labeled data conditions because they use supervised learning [5]. Supervised deep learning models exhibit difficulties in adapting between different imaging domains because it leads to reduced clinical application effectiveness [4].

The integration of self-supervised learning (SSL) with transformer-based architectural methods provides potential solutions to these limitations. Self-supervised models leverage a huge amount of unlabeled data to learn valuable feature representations as they do not need human-provided annotations [6, 7]. However, medical applications require supervised fine-tuning using limited labeled data since this process helps refine representations for accurate classification. The feature learning abilities of Momentum Contrastive Learning (MoCo) achieve maximum performance through contrastive learning that differentiates alike from different image pairs, which enhances feature representation quality [7, 8]. The Swin Transformer system, which utilizes hierarchical self-attention algorithms delivers top-quality results in medical imaging analysis through its ability to recognize local and global contextual relationships [9].

The present research lacks an effective hybrid learning framework that integrates self-supervised contrastive pretraining with supervised fine-tuning, particularly for retinal disease classification. Most self-supervised methods only work on natural image datasets [8], but researchers have not yet fully developed their use for fundus imaging. The